Part 2 · Scale

Chapter 1:
The Internet at Scale

How routers, ISPs, and shared paths make the Internet scale

Yaxiong Xie

CSE589 · Modern Network Concepts · Fall 2026

Four questions organize the Internet core

By the end of this part, you should be able to answer four questions.

1

Core structure

Who connects the Internet core?

2

...

Routing + forwarding

3

...

Packet switching

4

...

Performance

See how routers, ISPs, content networks, and IXPs form the Internet core.

Four questions organize the Internet core

By the end of this part, you should be able to answer four questions.

1 ✓	2	3	4
Core structure	**Routing + forwarding**	…	…
Who connects the Internet core?	Which path—and how is each hop executed?	**Packet switching**	**Performance**

Choose an end-to-end route, then execute it one hop at a time.

Four questions organize the Internet core

By the end of this part, you should be able to answer four questions.

1 ✓
Core structure
Who connects the Internet core?

2 ✓
Routing + forwarding
Which path—and how is each hop executed?

3
Packet switching
How do packets share link capacity?

4
...
Performance

Explain how packet and circuit switching share capacity.

Four questions organize the Internet core

By the end of this part, you should be able to answer four questions.

1 ✓

Core structure

Who connects the Internet core?

2 ✓

Routing + forwarding

Which path—and how is each hop executed?

3 ✓

Packet switching

How do packets share link capacity?

4

Performance

What determines delay, loss, and throughput?

Connect link properties and contention to delay, loss, and throughput.

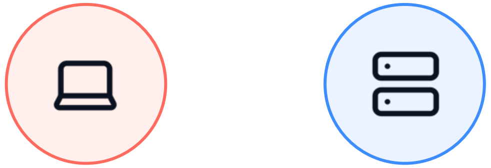
Part 1 · Core structure

01

From one router to a network of networks.

Core structure | Routing + forwarding | Packet switching | Performance

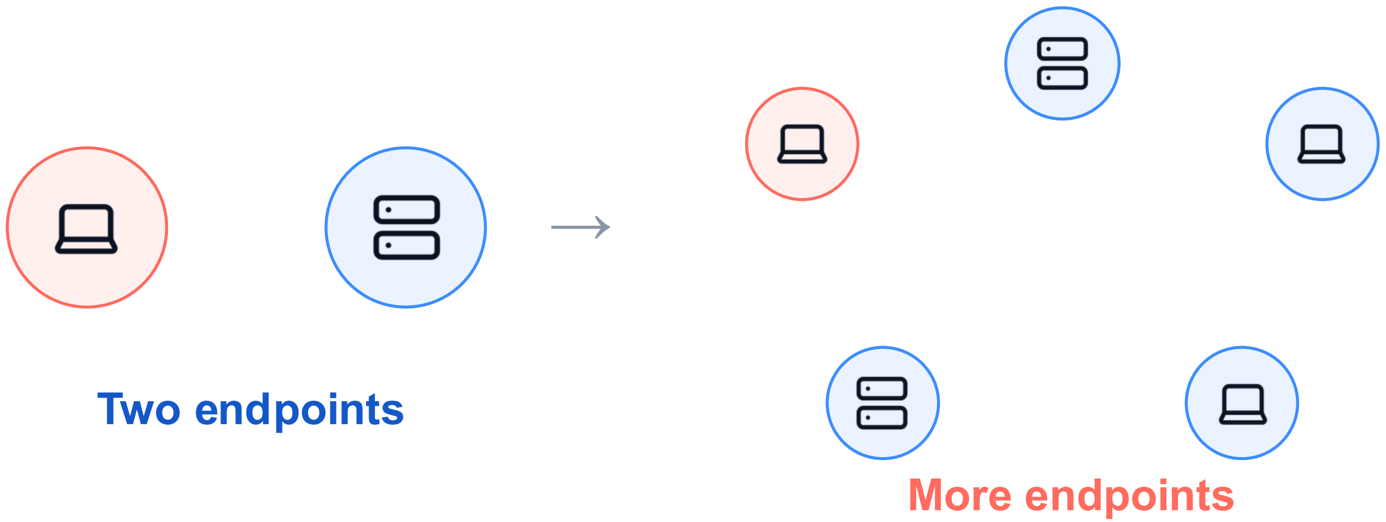
How can every endpoint reach every other endpoint?



Two endpoints

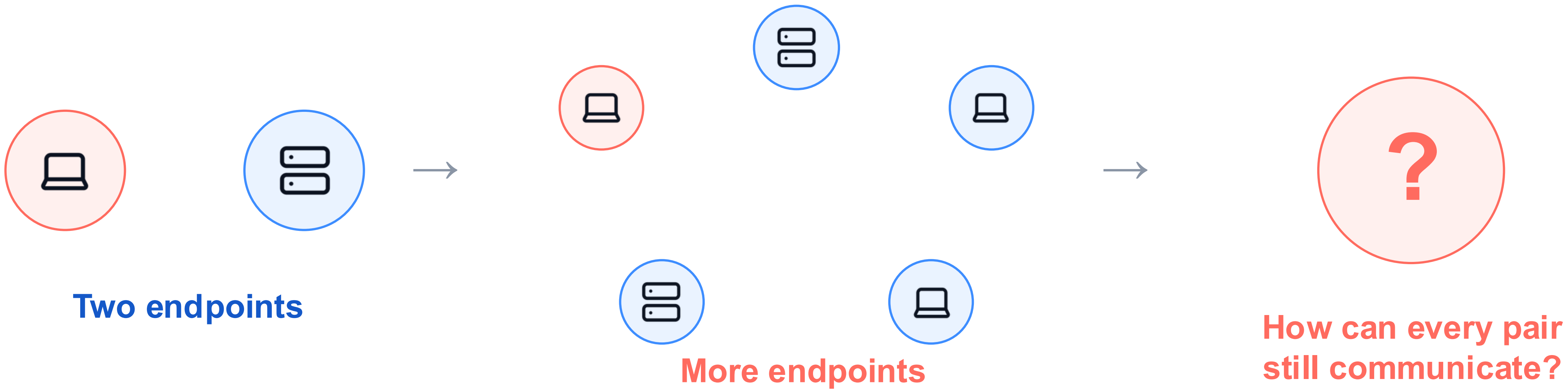
1 Direct links | 2 Scaling problem | 3 Shared routers | 4 Network core

How can every endpoint reach every other endpoint?

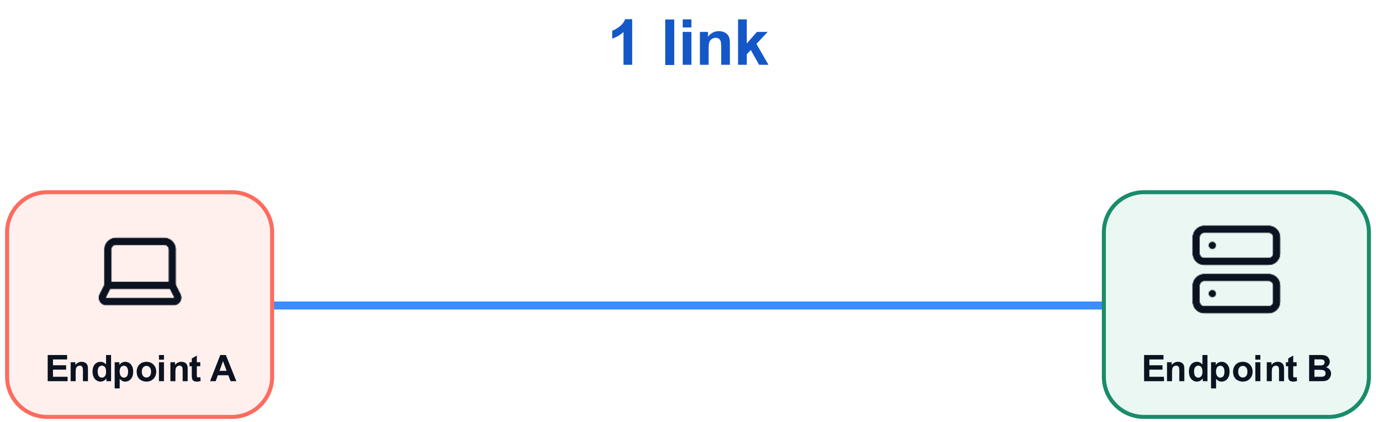


1 Direct links
2 Scaling problem
3 Shared routers
4 Network core

How can every endpoint reach every other endpoint?

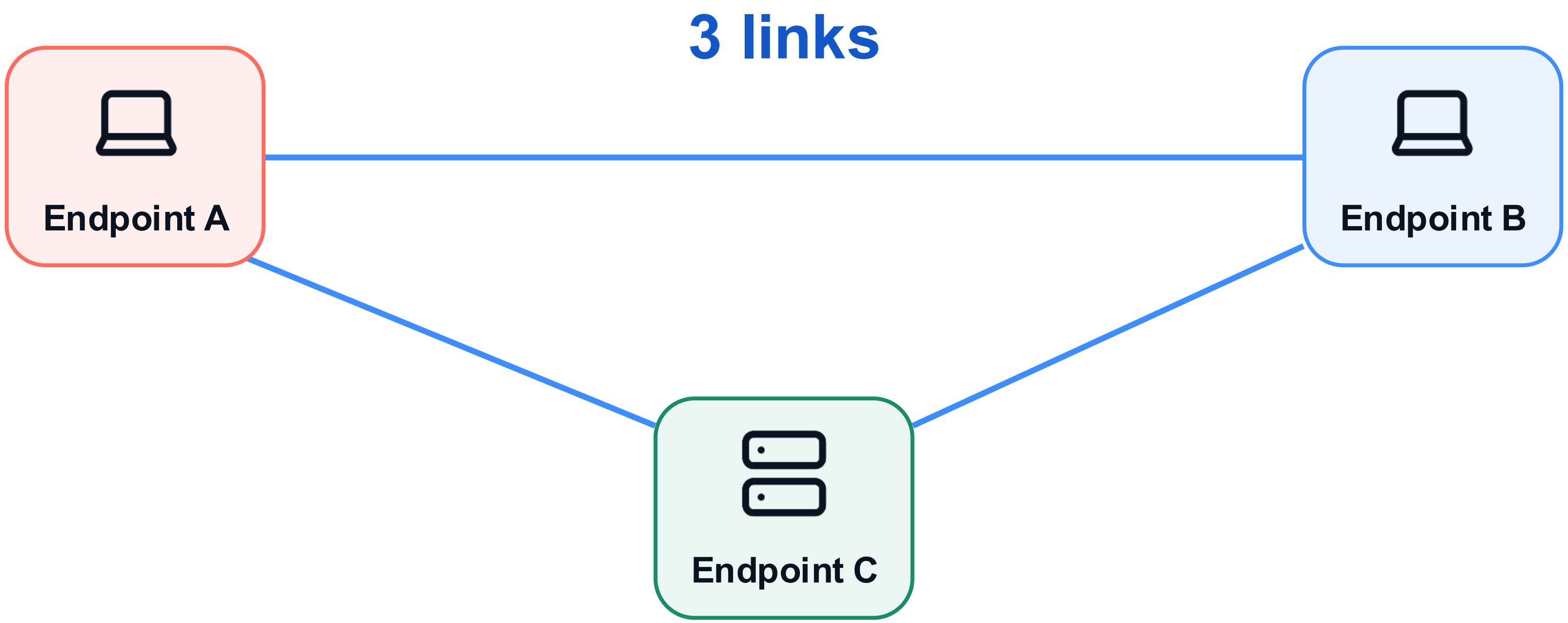


Start simple: two endpoints need one link



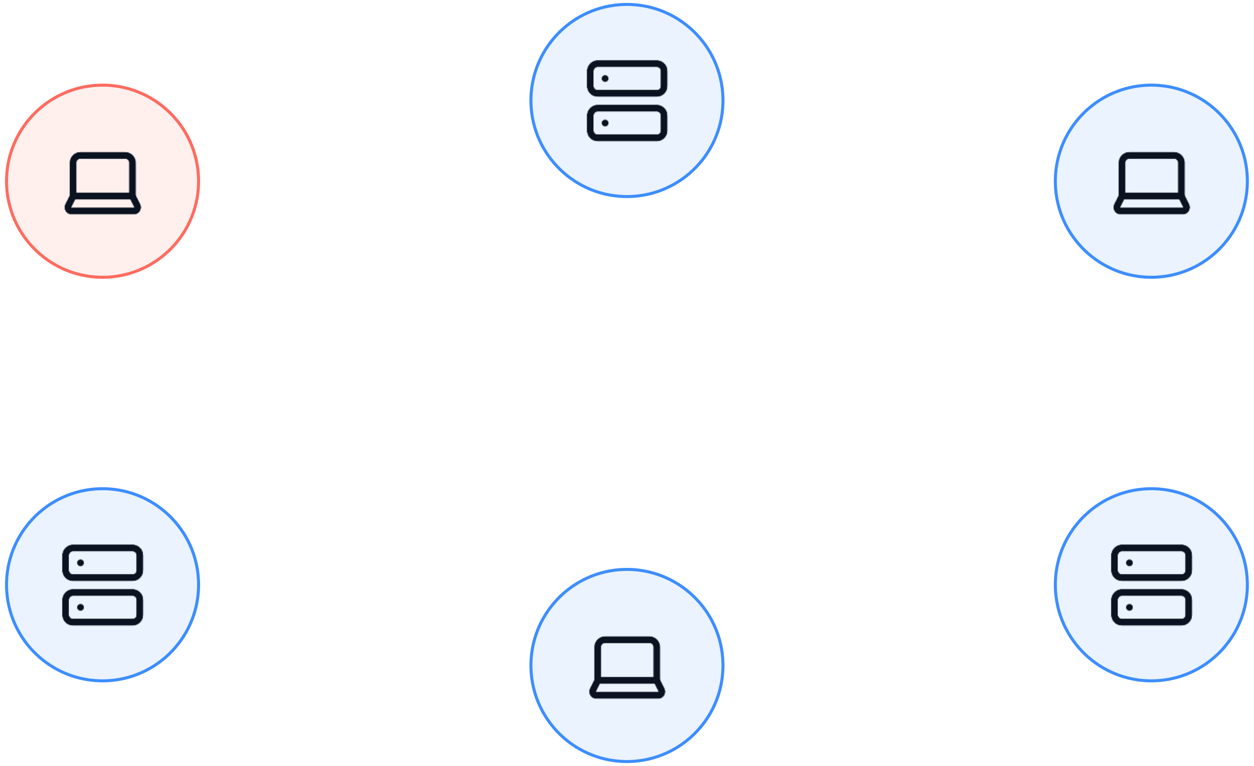
Direct connection is simple at tiny scale.

Add one endpoint: now three links are needed



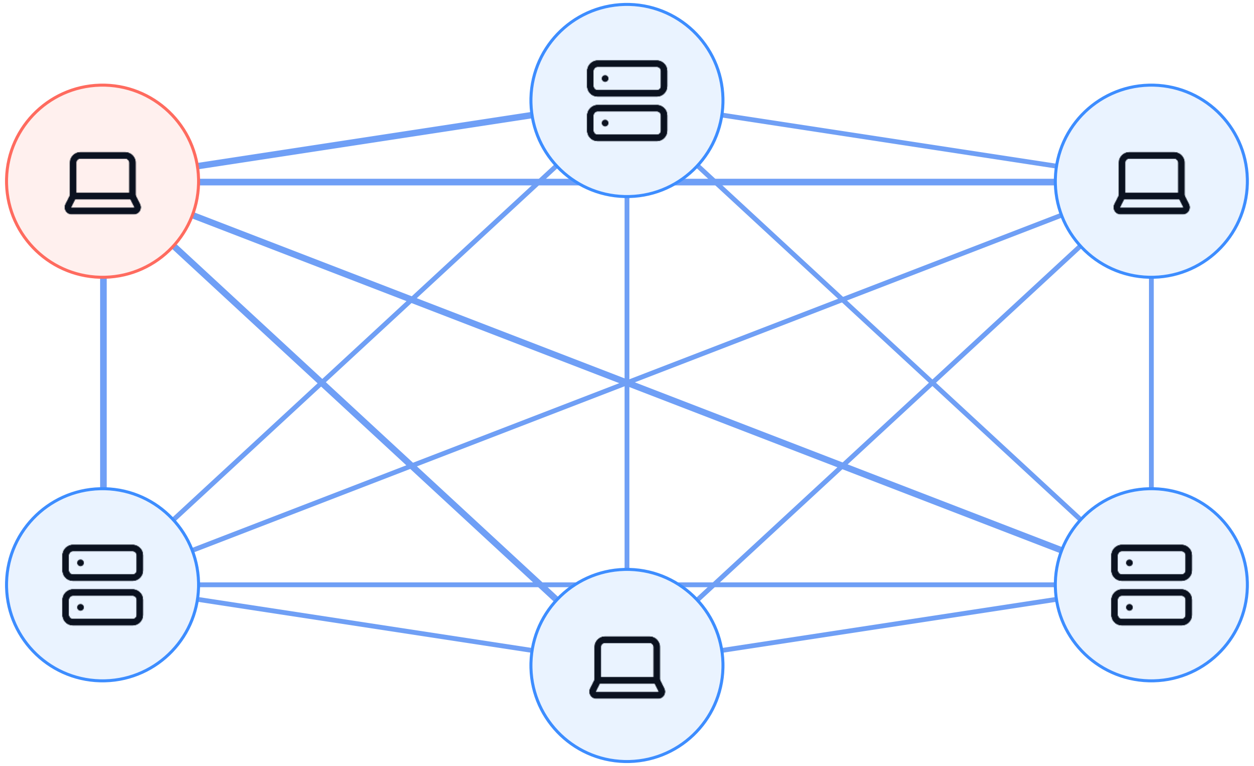
Every new endpoint creates more pairwise connections.

Direct connections grow quadratically



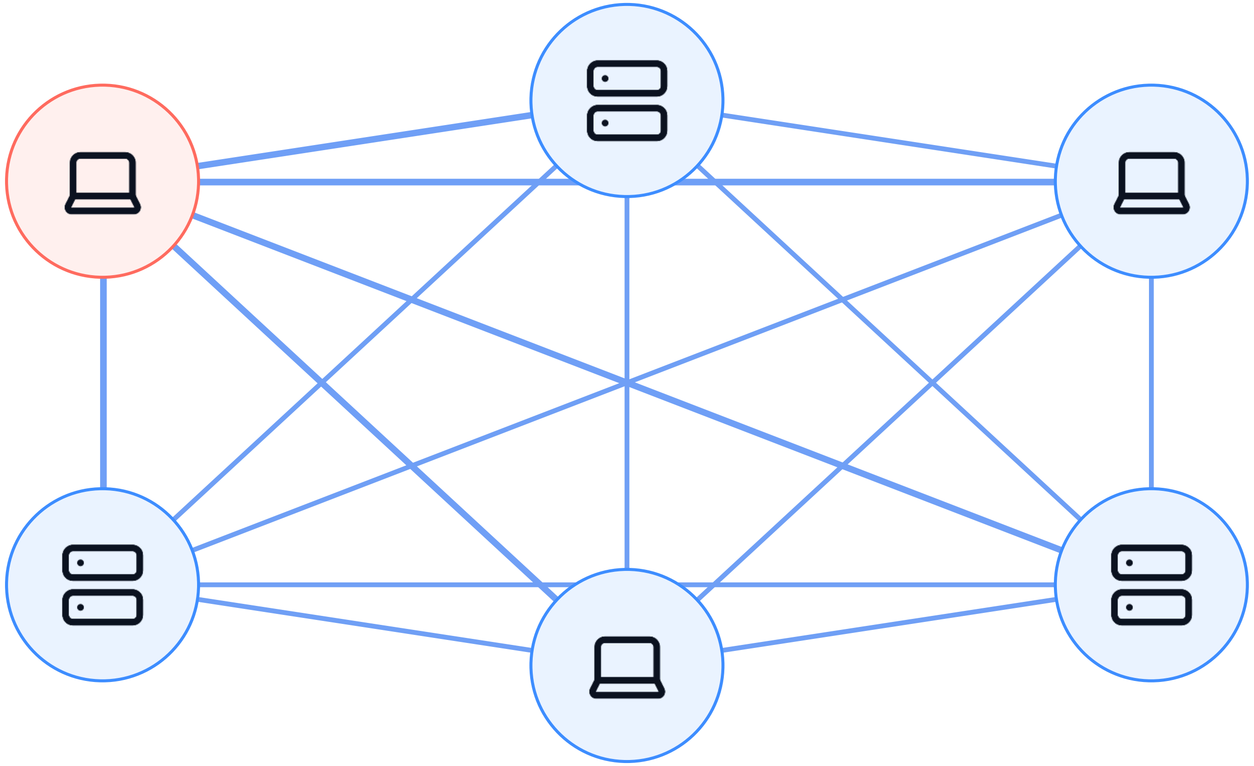
1 Direct links | 2 Scaling problem | 3 Shared routers | 4 Network core

Direct connections grow quadratically



1 Direct links | 2 Scaling problem | 3 Shared routers | 4 Network core

Direct connections grow quadratically



Every pair needs one link

$$E = \frac{N(N-1)}{2}$$

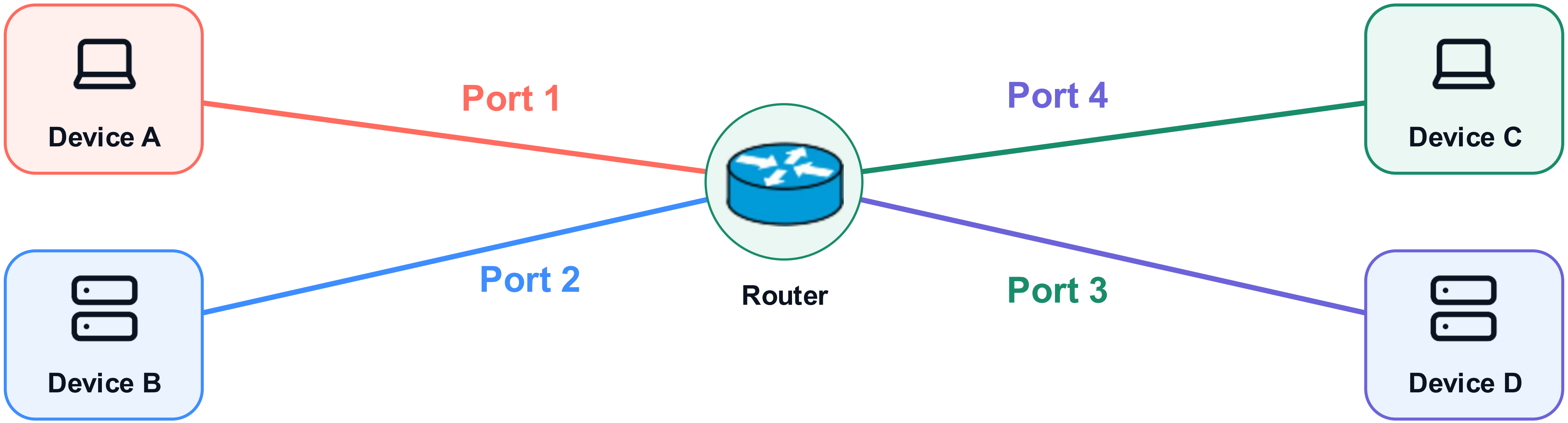
E = number of direct links

The number of direct links grows quadratically with N.

One router connects several devices through separate ports

1 Direct links | 2 Scaling problem | 3 Shared routers | 4 Network core

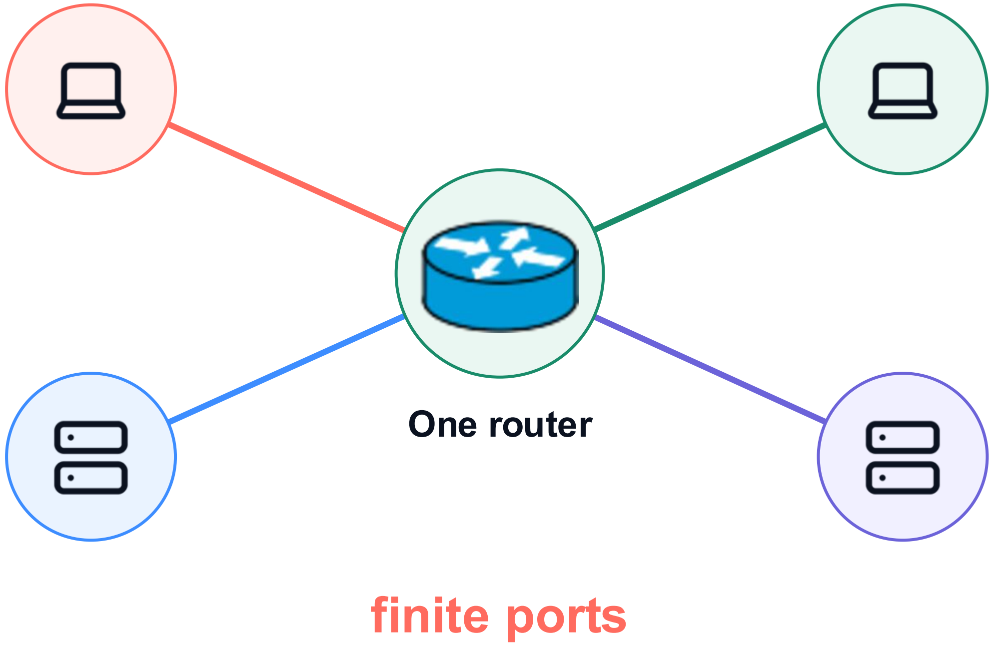
One router connects several devices through separate ports



But a router has a finite number of ports—and a network must serve far more devices.

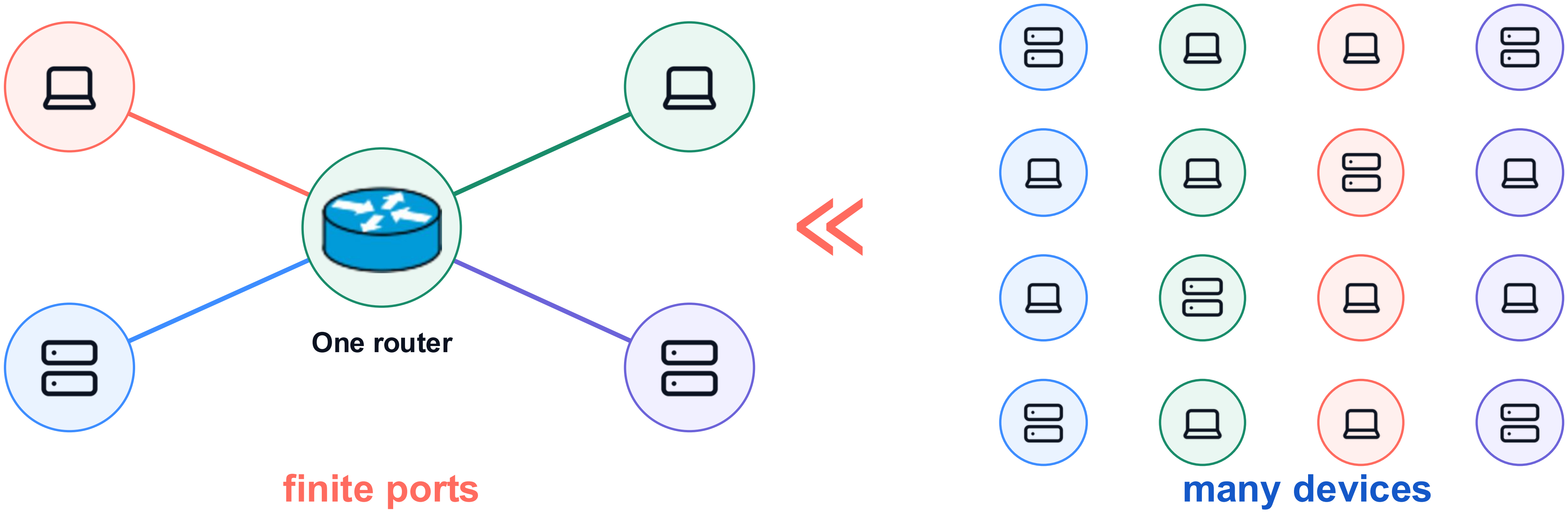
1 Direct links | 2 Scaling problem | 3 Shared routers | 4 Network core

One router is not enough for a large network



1 Direct links | 2 Scaling problem | 3 Shared routers | 4 Network core

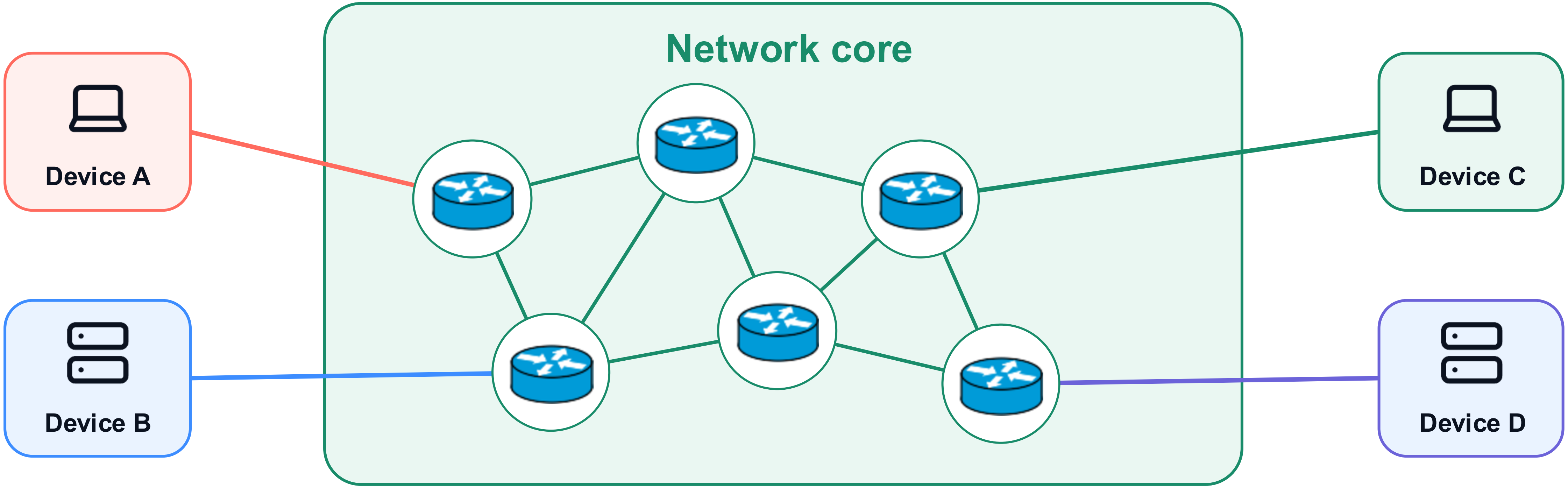
One router is not enough for a large network



To scale, routers must connect to other routers.

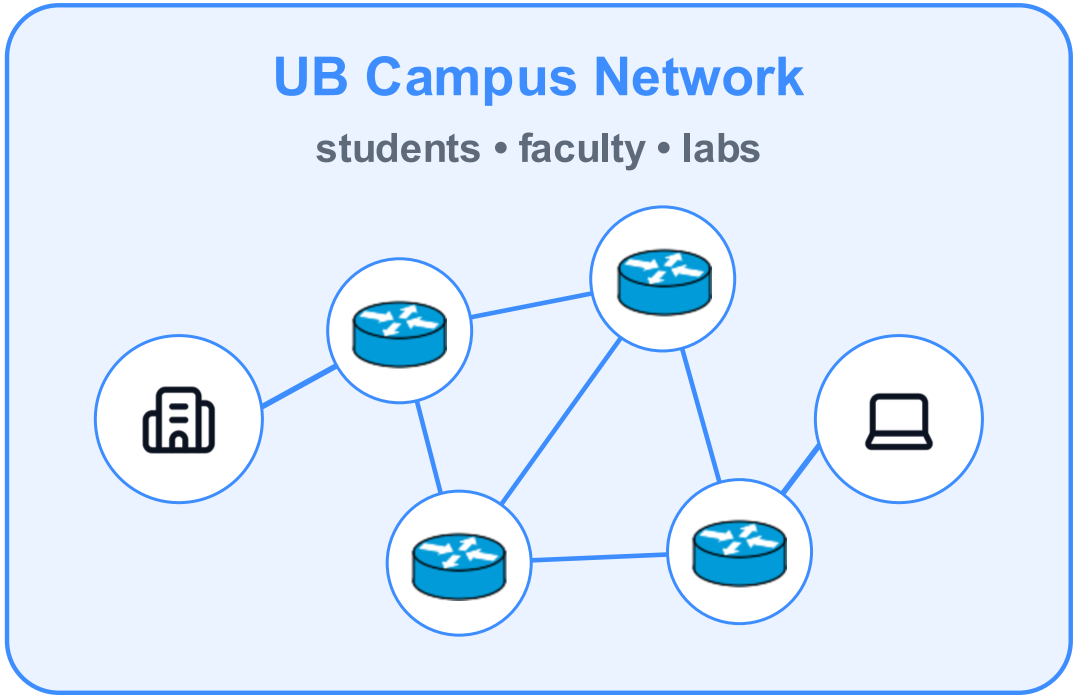
1 Direct links | 2 Scaling problem | 3 Shared routers | 4 Network core

Multiple routers interconnect to form the network core



Edge devices attach to routers; interconnected routers form the network core.

A network or ISP serves only part of the users



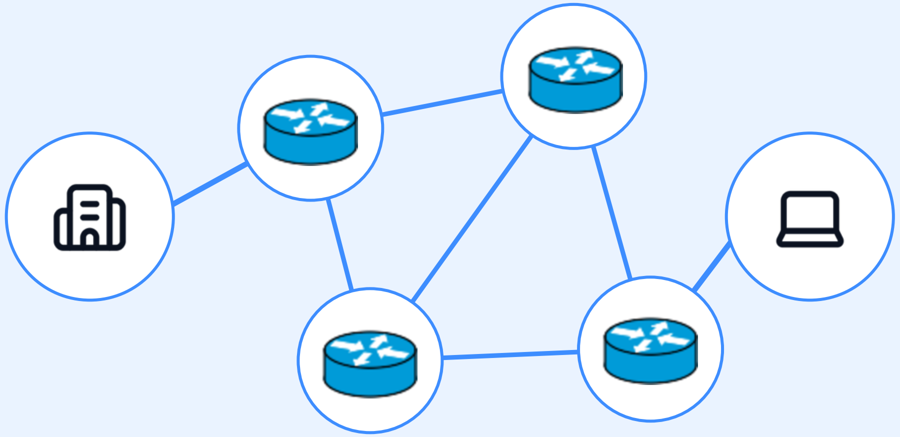
Each network covers a limited community or region—not everyone.

Core structure | Routing + forwarding | Packet switching | Performance

A network or ISP serves only part of the users

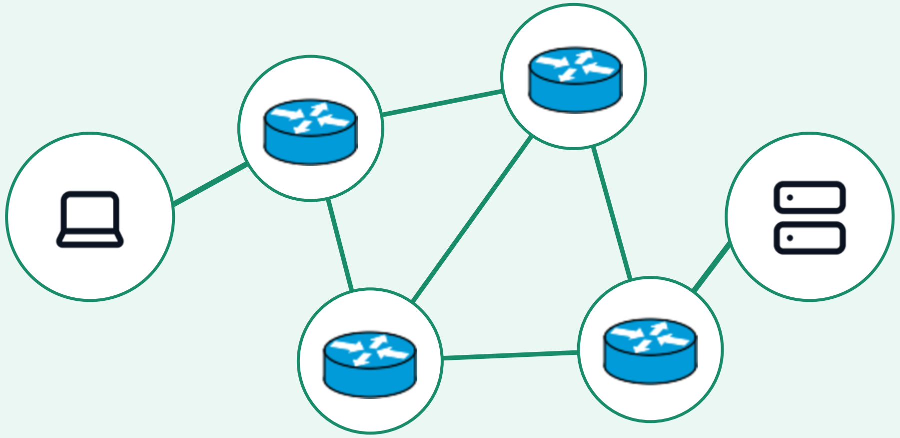
UB Campus Network

students • faculty • labs



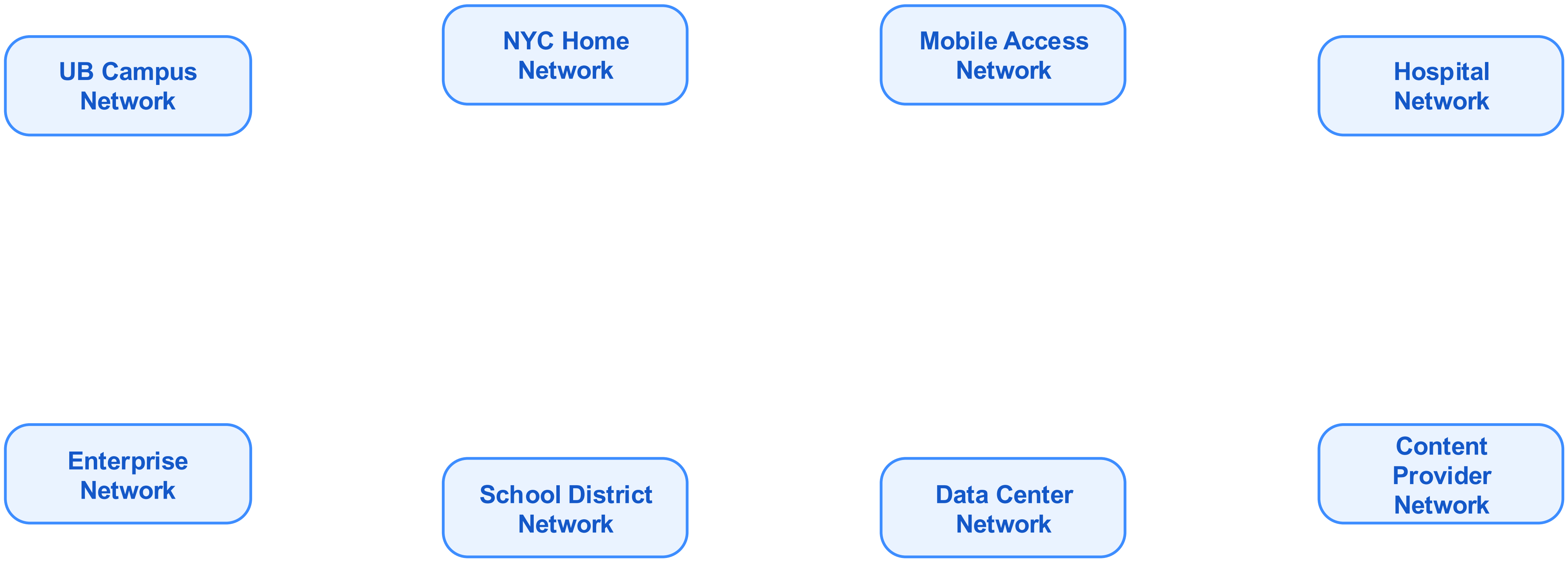
AT&T New York Network

homes • campuses • mobile users



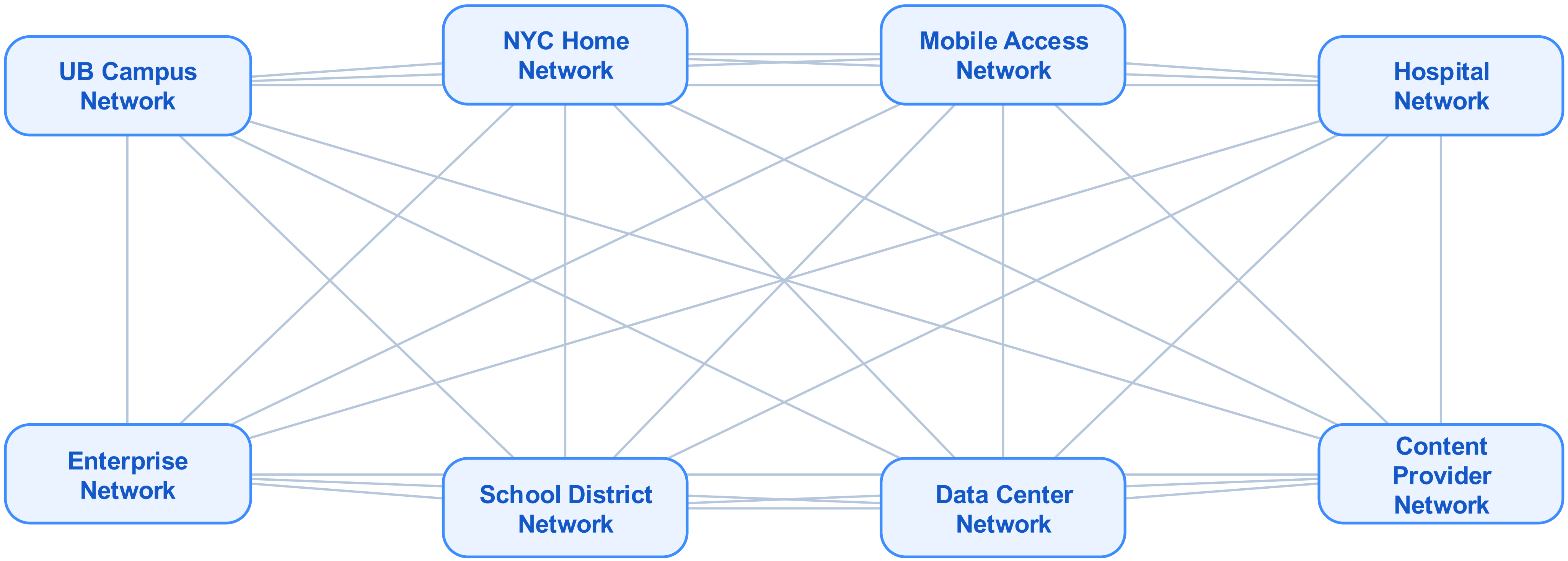
Each network covers a limited community or region—not everyone.

Directly connecting every small network does not scale



The same scaling problem appears again—now between networks.

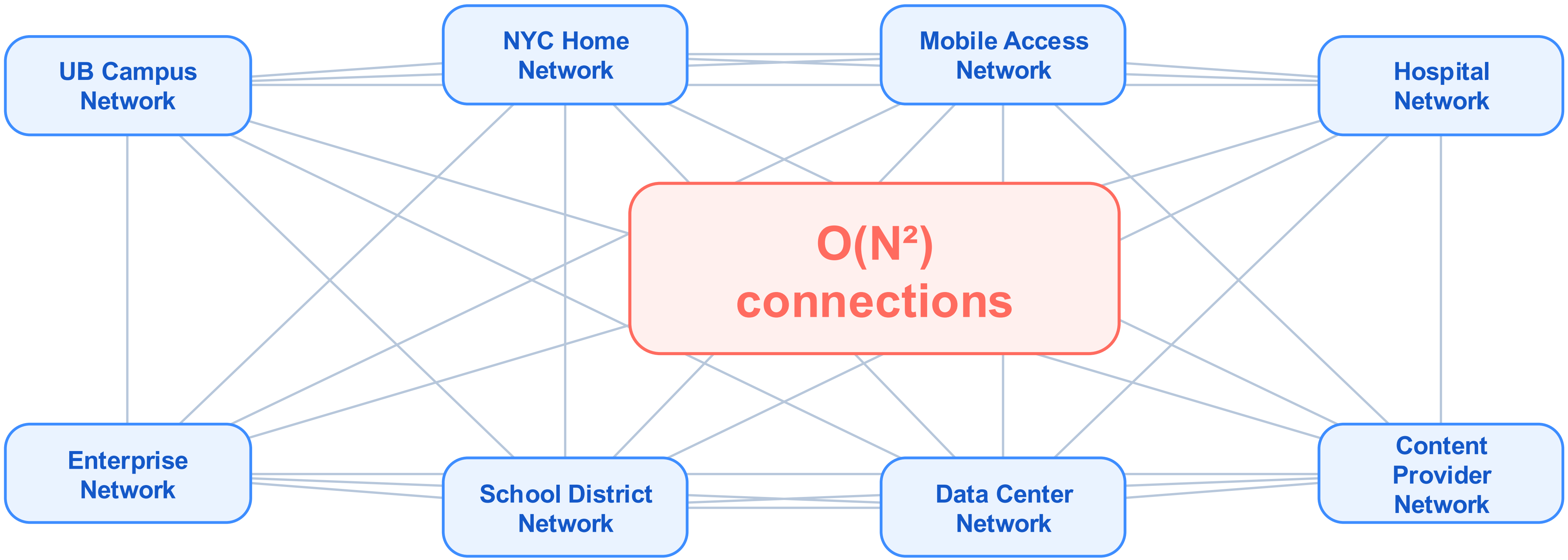
Directly connecting every small network does not scale



The same scaling problem appears again—now between networks.

Core structure | Routing + forwarding | Packet switching | Performance

Directly connecting every small network does not scale



The same scaling problem appears again—now between networks.

Core structure | Routing + forwarding | Packet switching | Performance

Local ISPs begin as separate access networks

Local ISP A

Local ISP B

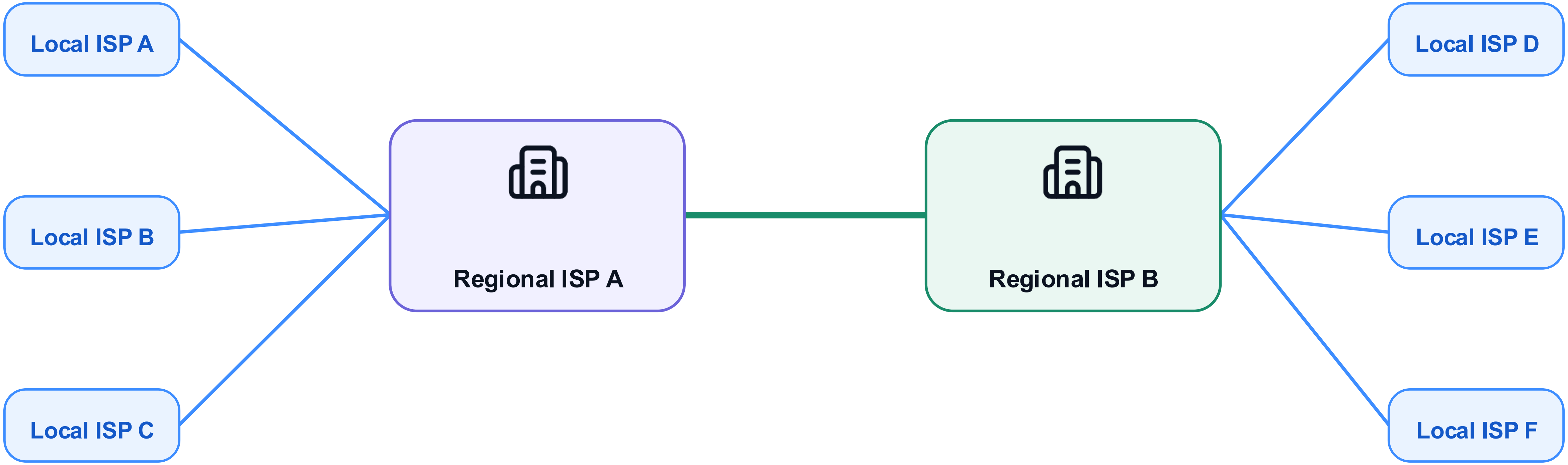
Local ISP C

Local ISP D

Local ISP E

Local ISP F

Regional ISPs aggregate many local ISPs



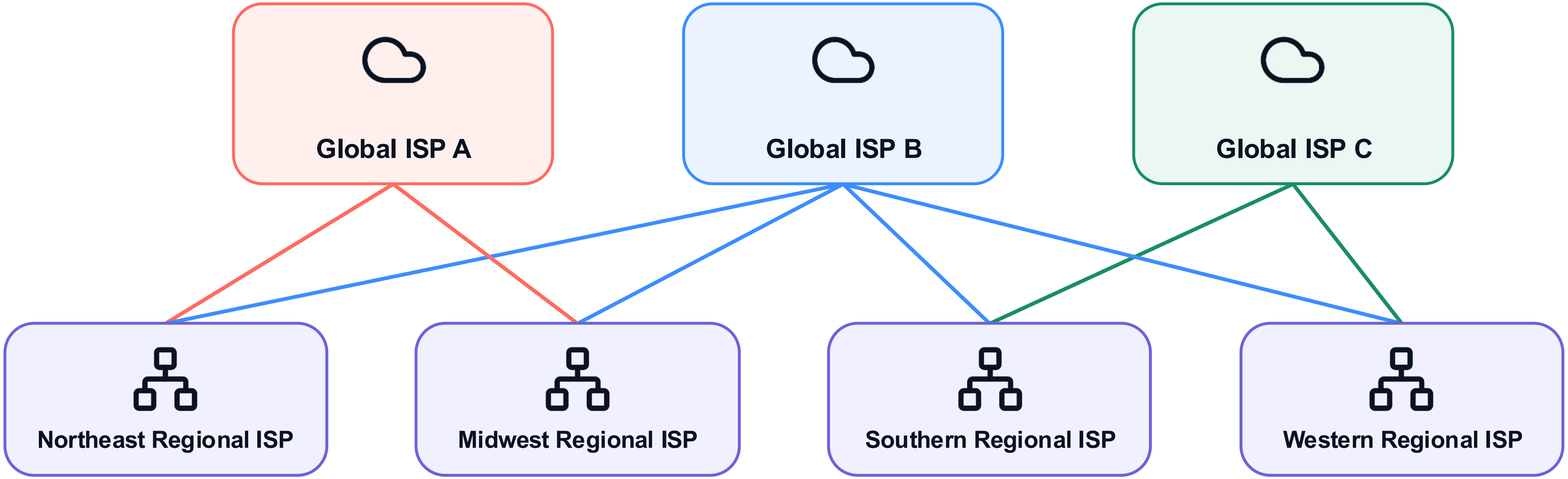
Hierarchy begins when regional ISPs aggregate many local ISPs.

Core structure | Routing + forwarding | Packet switching | Performance

Regional ISPs first serve separate geographic regions

- Northeast Regional ISP
- Midwest Regional ISP
- Southern Regional ISP
- Western Regional ISP

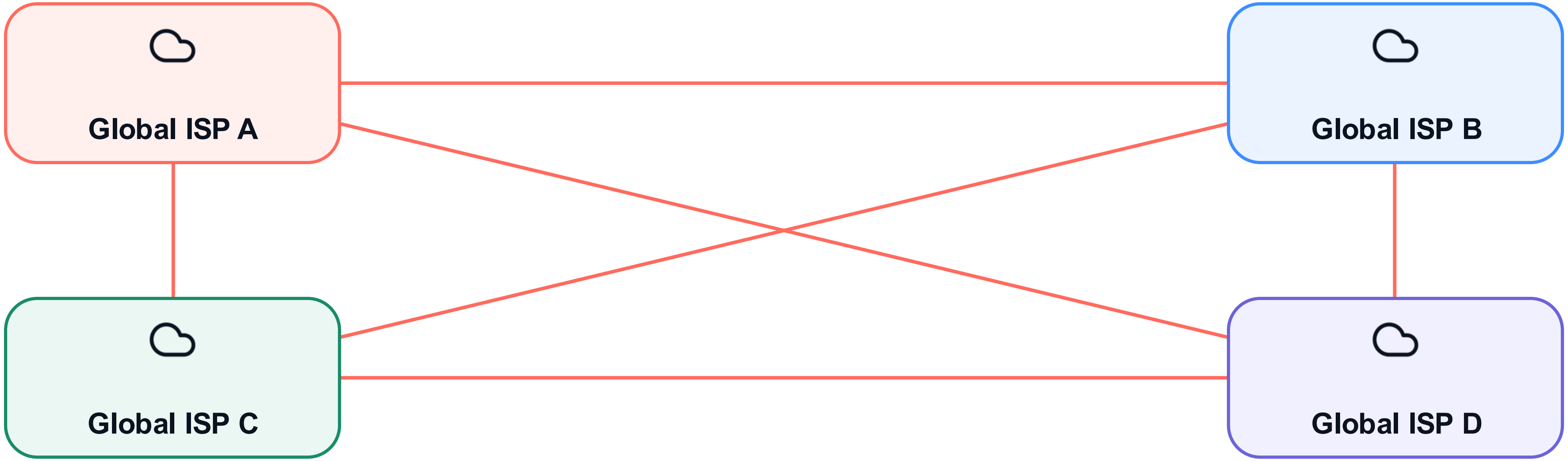
Regional ISPs connect through global providers



Regional ISPs connect upward to multiple global providers.

But global ISPs also need to exchange traffic

If they rely only on bilateral private links…



Does every pair need its own private link?

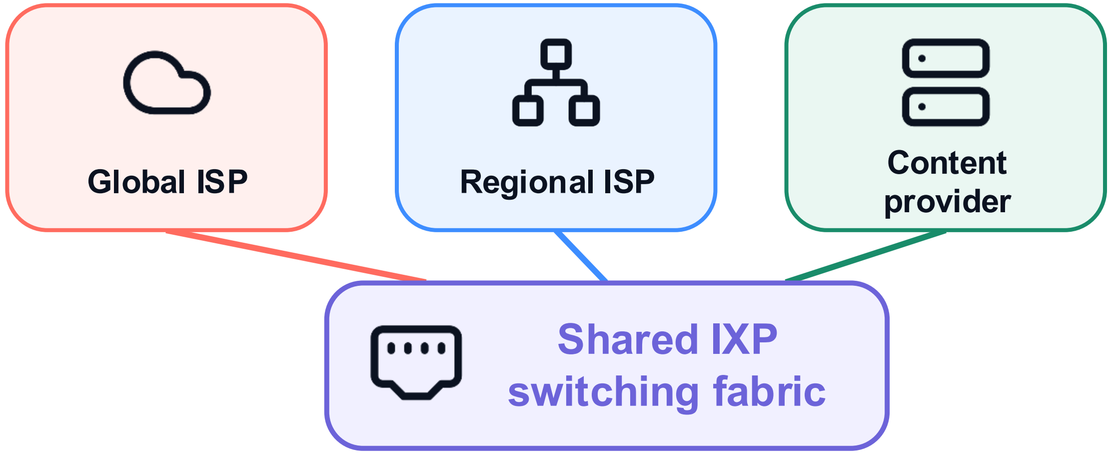
The same scaling problem appears whenever many independent networks need to exchange traffic.

An IXP is where independent networks exchange traffic

Example: an IXP facility in London

IXP = Internet Exchange Point

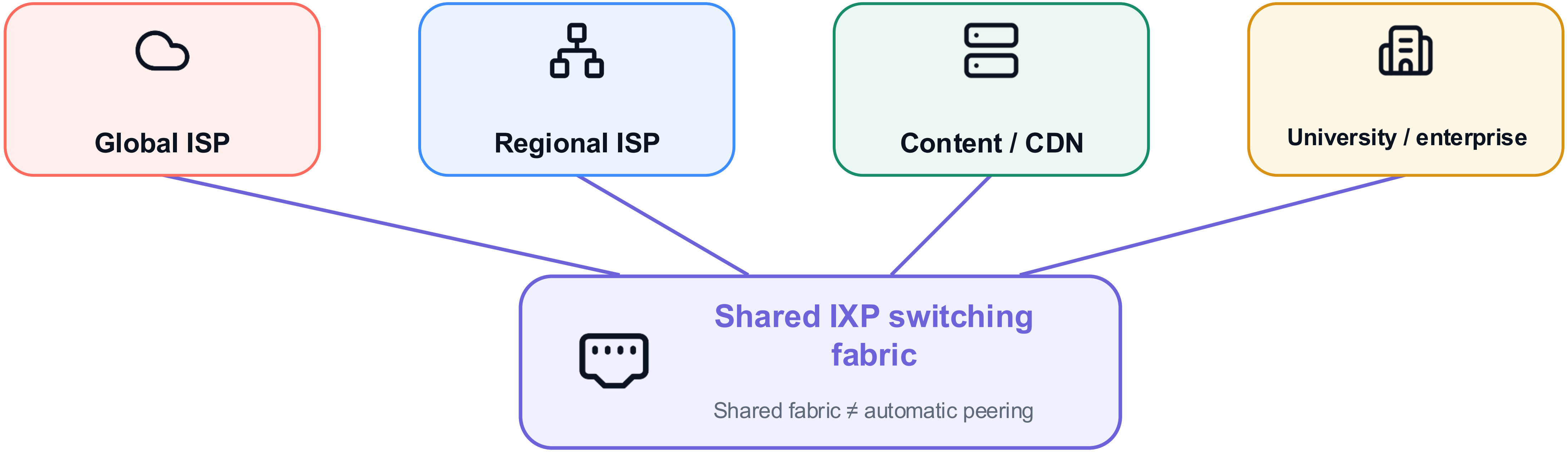
A physical facility where independently operated networks connect and exchange traffic.



IXP is the meeting place; peering is the traffic-exchange relationship.

An IXP connects networks—not only ISPs

Participants are independently operated IP networks

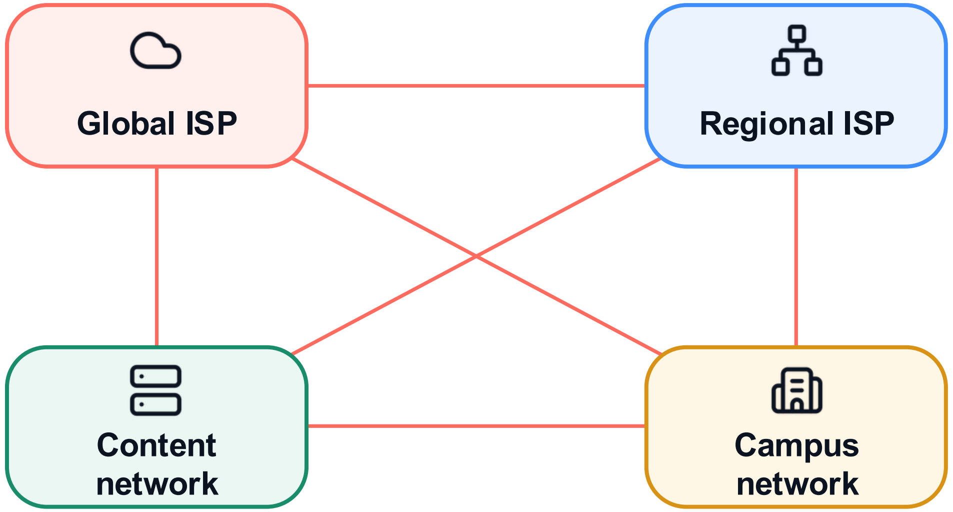


Global and regional ISPs can peer here—if both connect and their policies allow it.

Core structure | Routing + forwarding | Packet switching | Performance

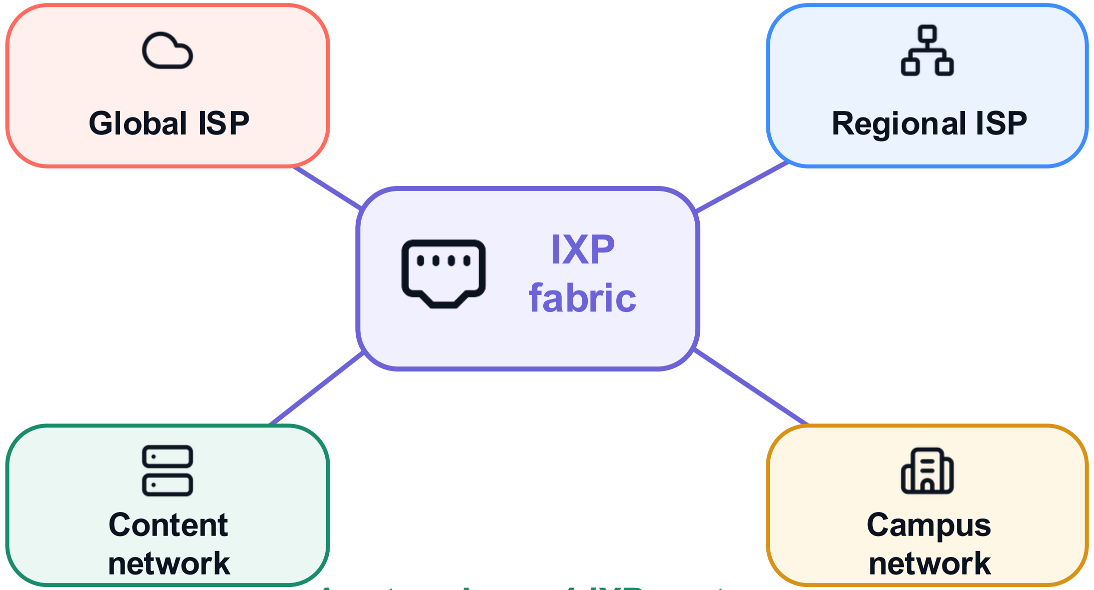
A shared IXP fabric scales better than pairwise links

Private cross-connects



4 networks → 6 private links
Pairwise links grow as N(N−1)/2

Shared IXP fabric

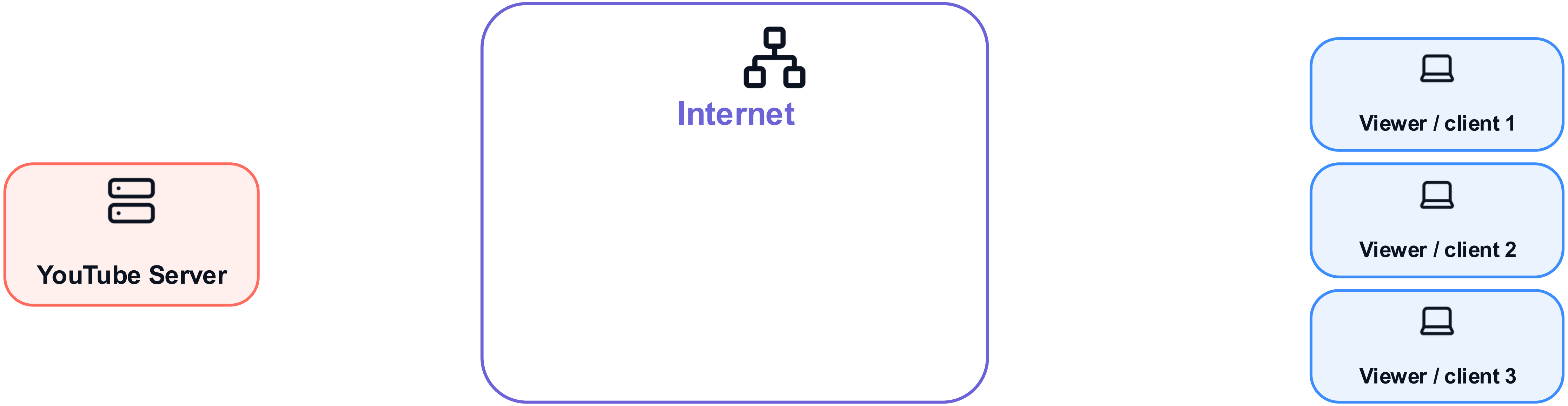


4 networks → 4 IXP ports
One connection can support many peerings

Private links still help heavy pairs; an IXP scales many possible peerings.

The same video crosses the Internet once per viewer

Ordinary delivery is unicast: one server-to-client stream per viewer

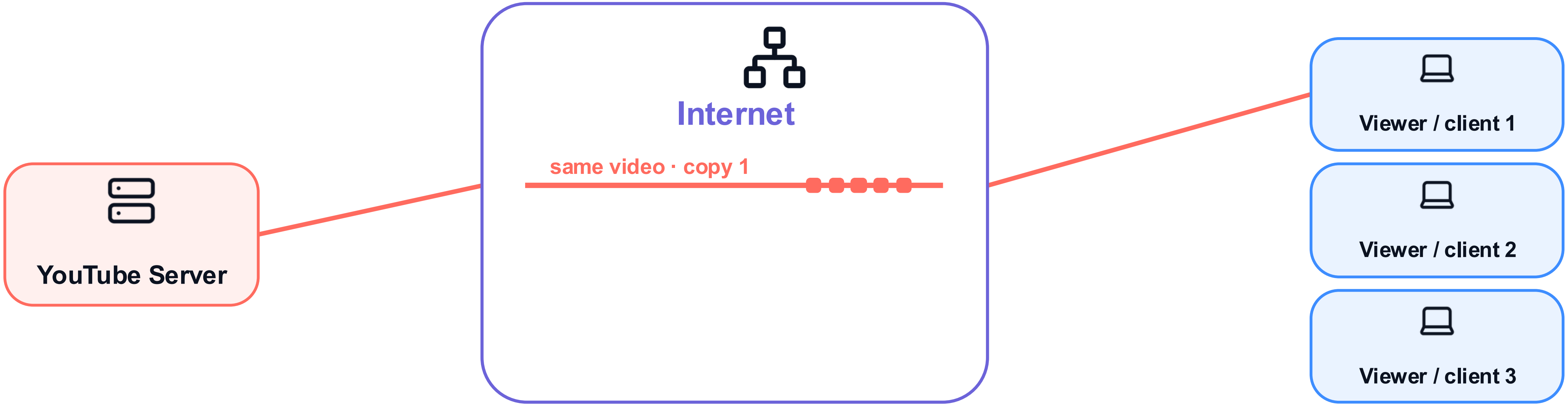


Three viewers request the same video. How many streams cross the Internet?

Core structure | Routing + forwarding | Packet switching | Performance

The same video crosses the Internet once per viewer

Ordinary delivery is unicast: one server-to-client stream per viewer

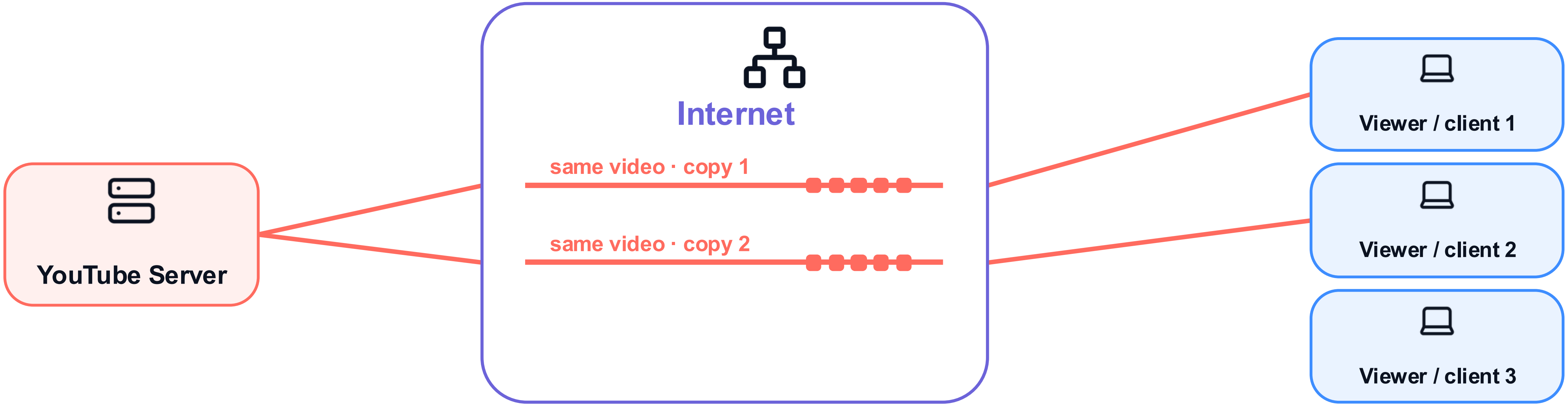


1 viewer → 1 independent stream

Core structure | Routing + forwarding | Packet switching | Performance

The same video crosses the Internet once per viewer

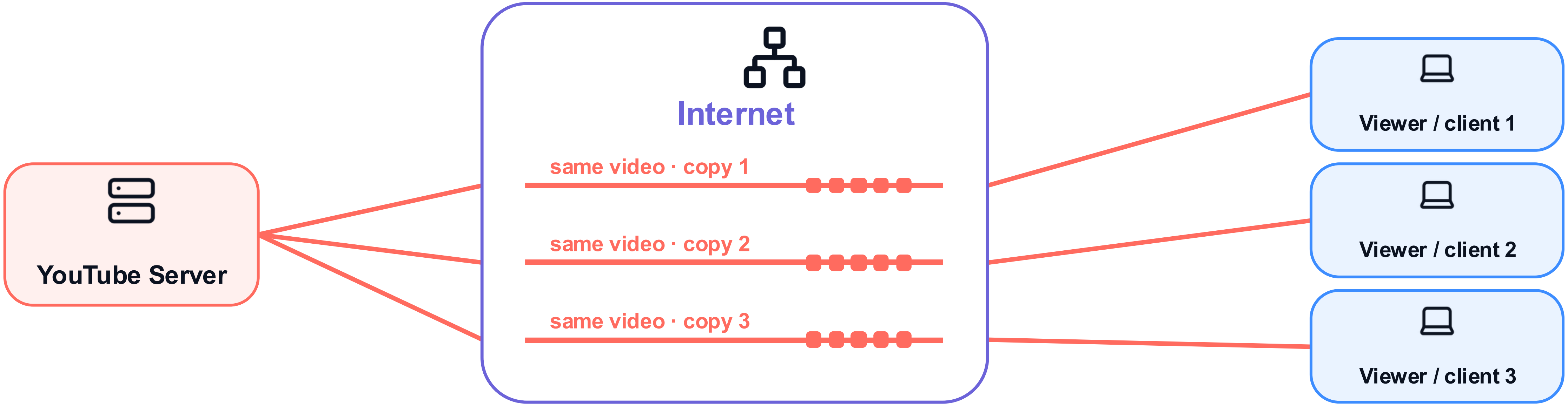
Ordinary delivery is unicast: one server-to-client stream per viewer



2 viewers → 2 independent streams

The same video crosses the Internet once per viewer

Ordinary delivery is unicast: one server-to-client stream per viewer

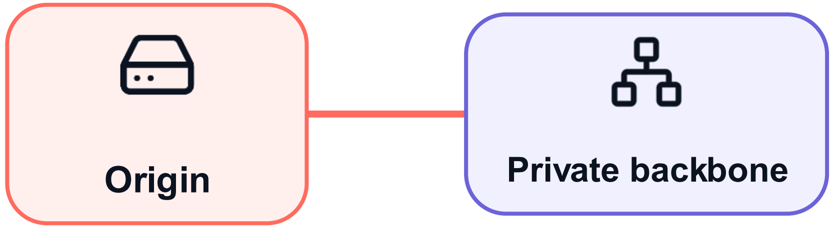


3 viewers → 3 independent streams

Same content × 3 unicast streams = the video crosses shared Internet paths three times.

A content network moves delivery capacity closer to users

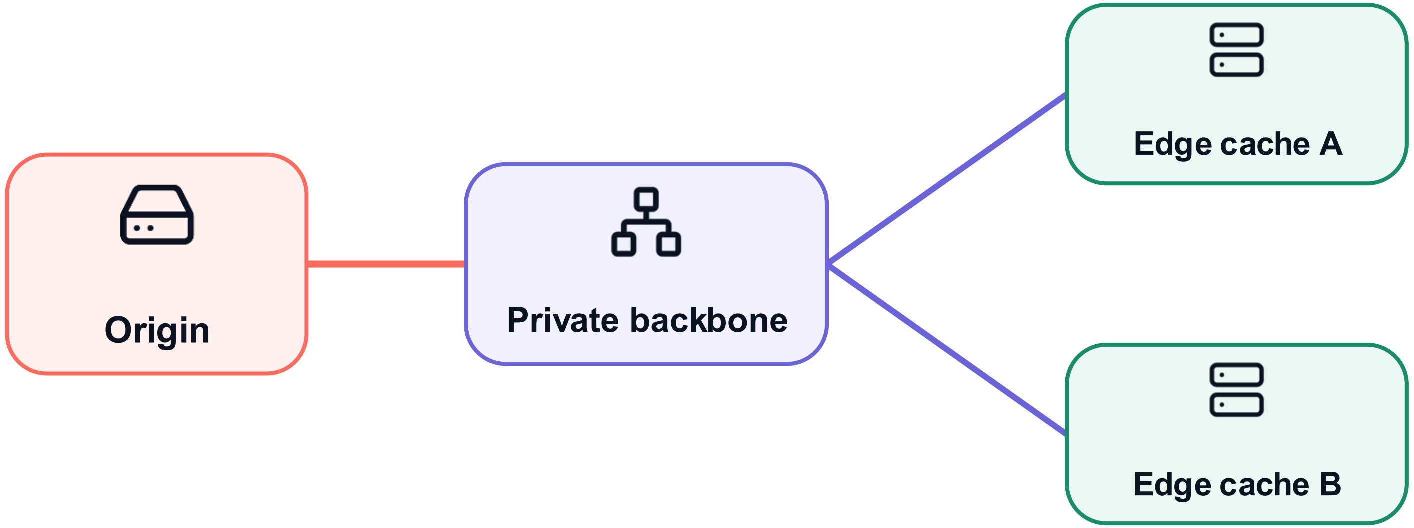
Step 1 · Carry content from the origin on a private backbone



One origin still needs a way to reach many regions.

A content network moves delivery capacity closer to users

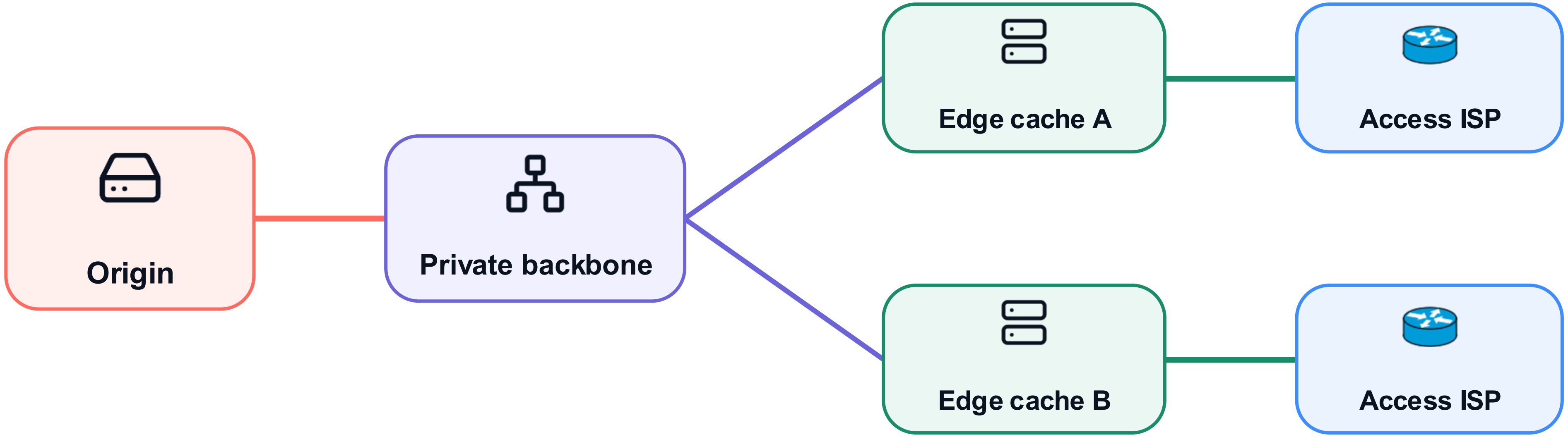
Step 2 · Place copies at distributed edge caches



Popular video objects can be copied to multiple edge locations.

A content network moves delivery capacity closer to users

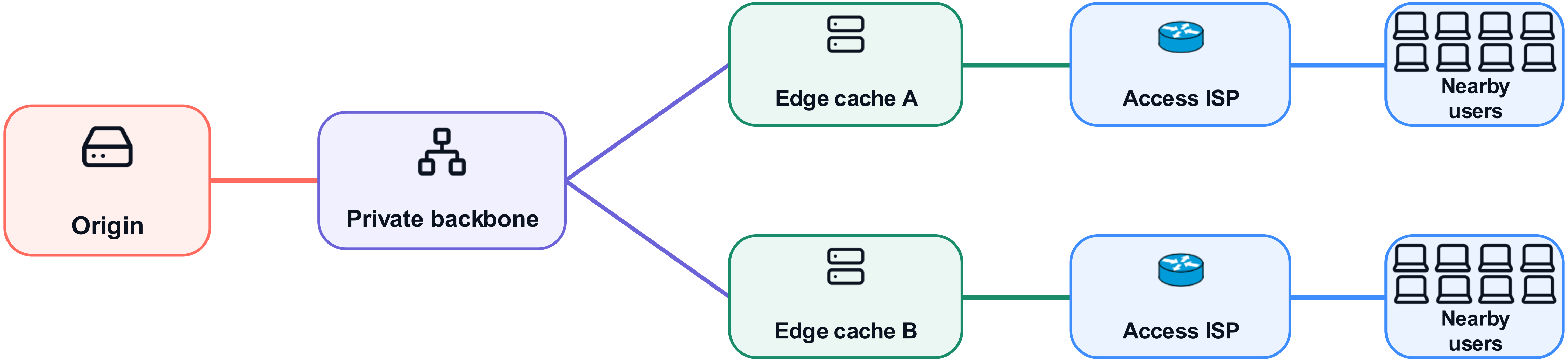
Step 3 · Interconnect those caches near access ISPs



Each edge cache connects where users enter the Internet.

A content network moves delivery capacity closer to users

Step 4 · Serve viewers from a nearby copy whenever possible



Cache hit: serve nearby · Cache miss: fetch or refill from the origin

Nearby copies shorten delivery paths and reduce repeated long-haul traffic.

Content networks can be service-owned or shared CDNs

Service-owned delivery networks

- **Google / YouTube**
  video, search, apps, and popular cached objects
- **Netflix Open Connect**
  a purpose-built network for streaming delivery
- **Meta**
  feeds, photos, video, messaging, and application traffic

Shared CDN / cloud-edge networks

- **Amazon CloudFront**
  global edge locations and regional edge caches
- **Microsoft Azure Front Door**
  cloud CDN and global application delivery
- **Akamai**
  distributed delivery for many content owners
- **Cloudflare**
  CDN, edge delivery, performance, and security

Different owners; the same design goal: put delivery capacity close to demand.

A content network is a network role—not another ISP tier.

ISPs provide reach; content networks deliver the service

ISP: access + reach

Connects users to other networks

Content network: service + delivery

Operates the service, edge sites, and origin

1 ISP carries the request

2 Networks interconnect

3 Content network answers

Connectivity and service are different roles—even when both networks meet at the same IXP.

The ISP supplies connectivity; the content network supplies the service.

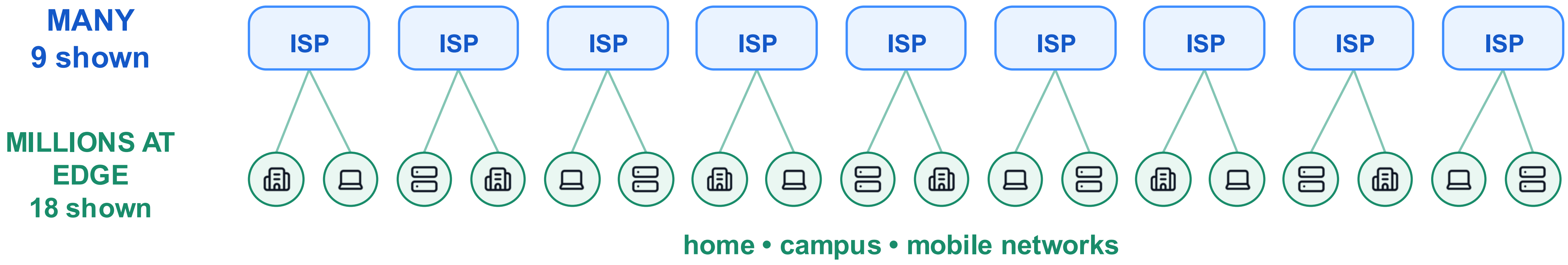
Millions of edge networks sit at the bottom

MILLIONS AT EDGE
18 shown

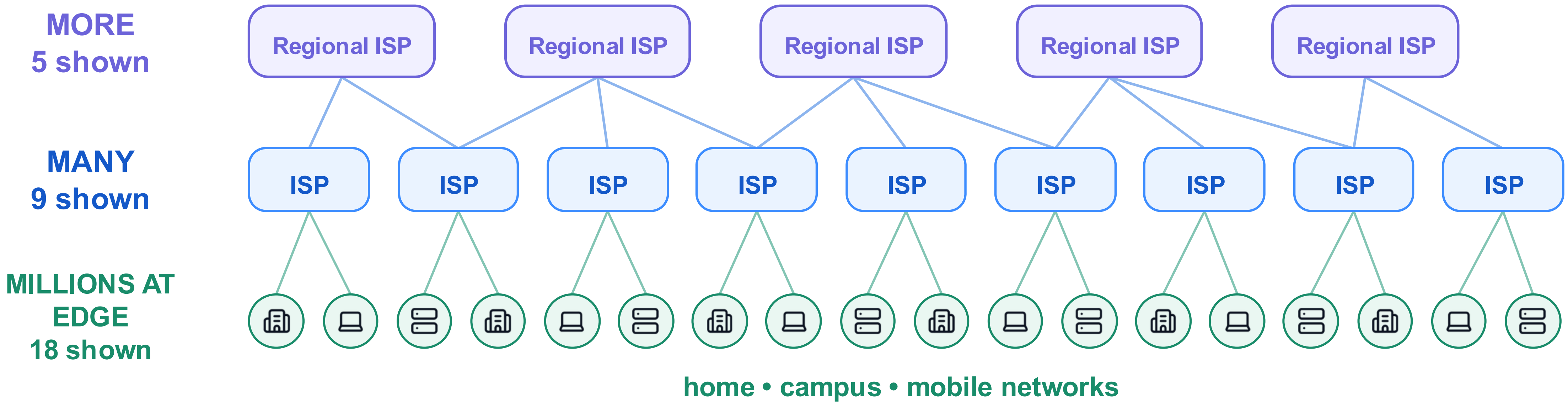
home • campus • mobile networks

Core structure | Routing + forwarding | Packet switching | Performance

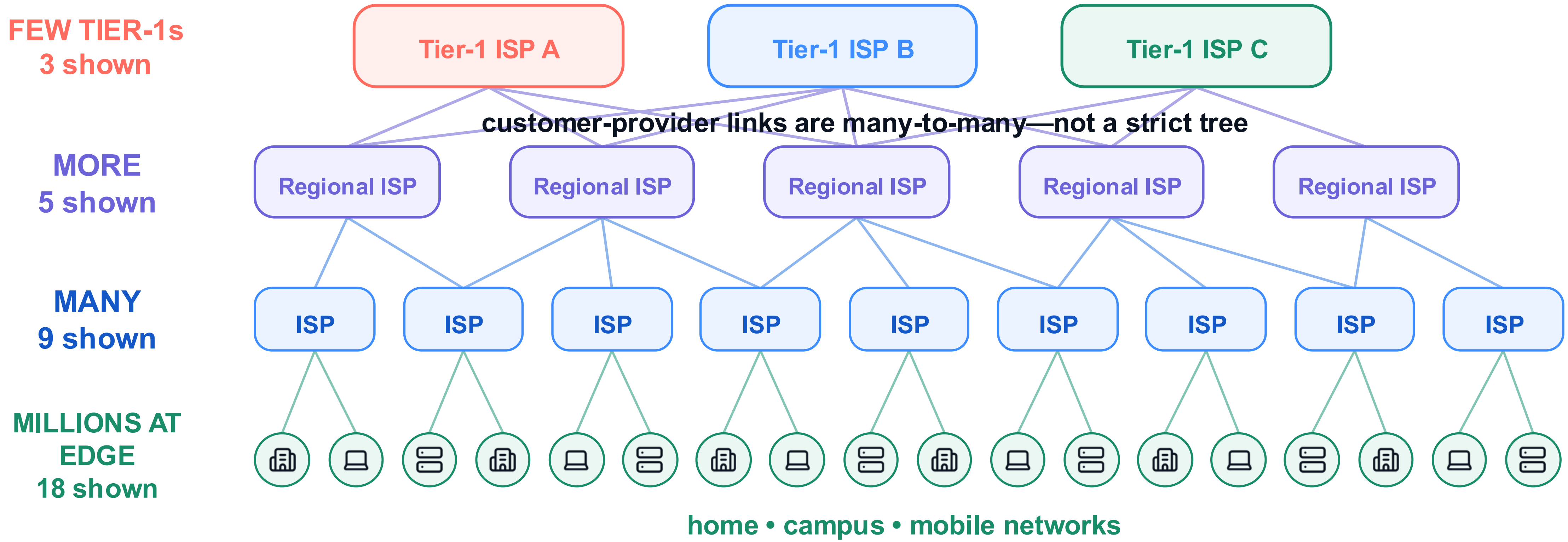
Access ISPs aggregate many edge networks



Regional ISPs aggregate many access ISPs



The ISP hierarchy is many-to-many, not a strict tree



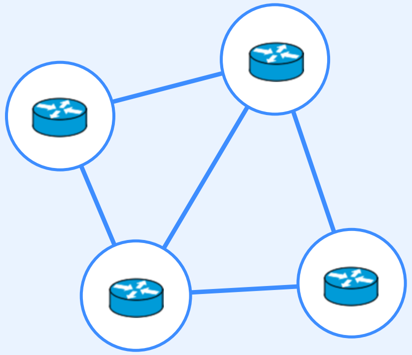
One Tier-1 serves many regional ISPs; a regional ISP can also connect to multiple upstreams.

Core structure | Routing + forwarding | Packet switching | Performance

Independent networks can operate on their own

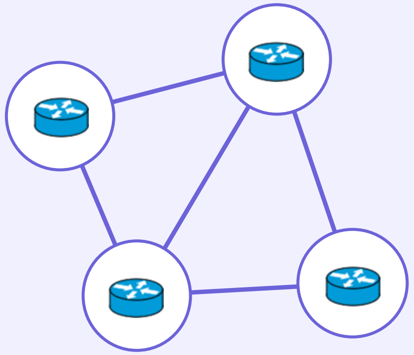
UB campus network

one independently operated network



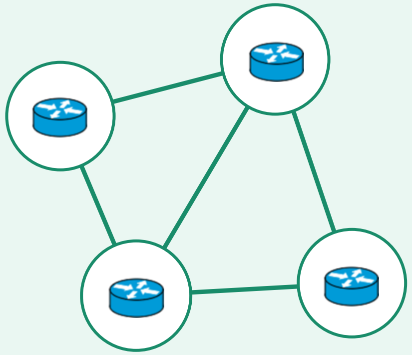
Regional ISP network

one independently operated network



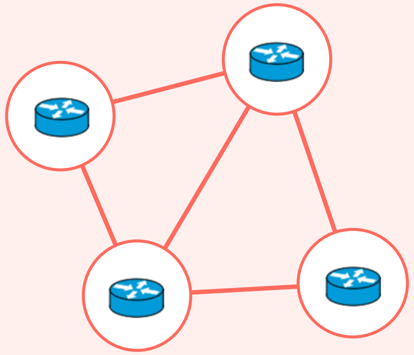
Tier-1 ISP network

one independently operated network

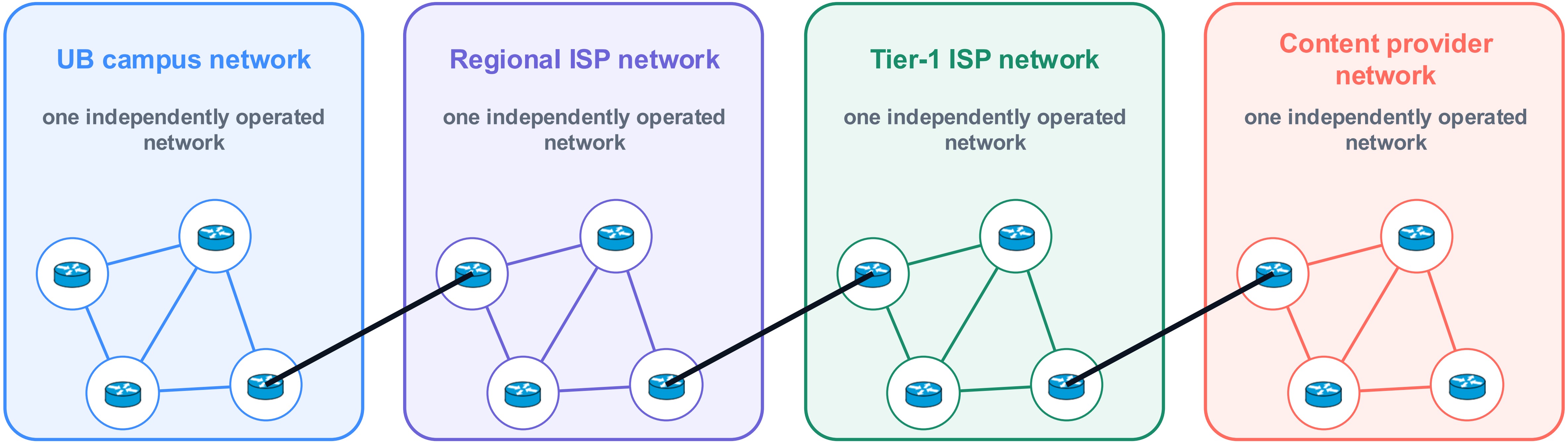


Content provider network

one independently operated network



The Internet is many complete networks joined at gateways

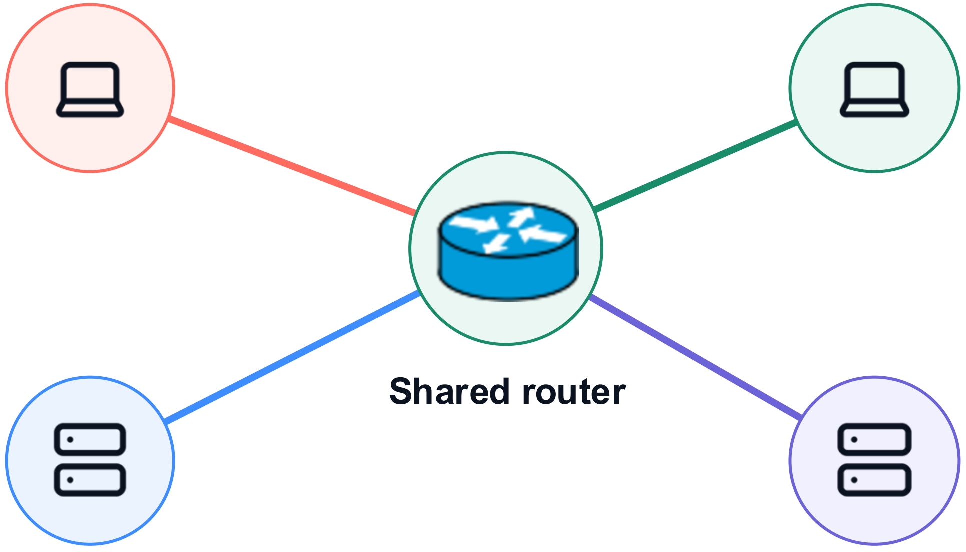


gateway-to-gateway links join the networks

The Internet is not one network—it is many complete networks joined at their gateways.

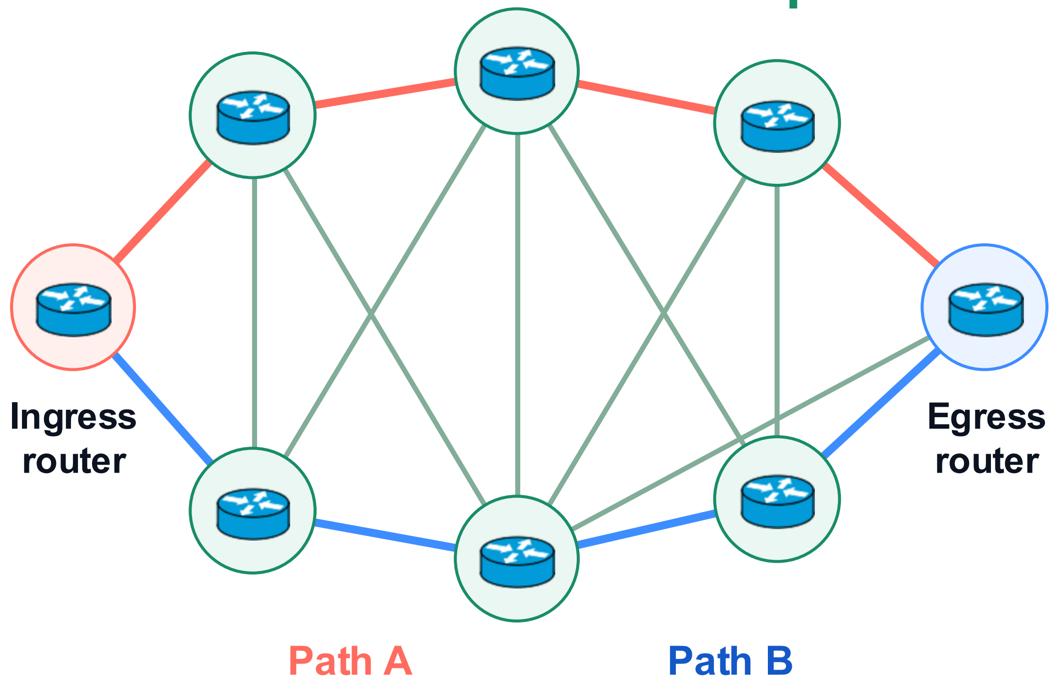
Shared routers reduce links; a meshed core adds paths

1 Fewer endpoint links



4 router links < 6 pairwise links

2 More router-to-router paths



Edge links are reduced; links among core routers are added intentionally for path diversity.

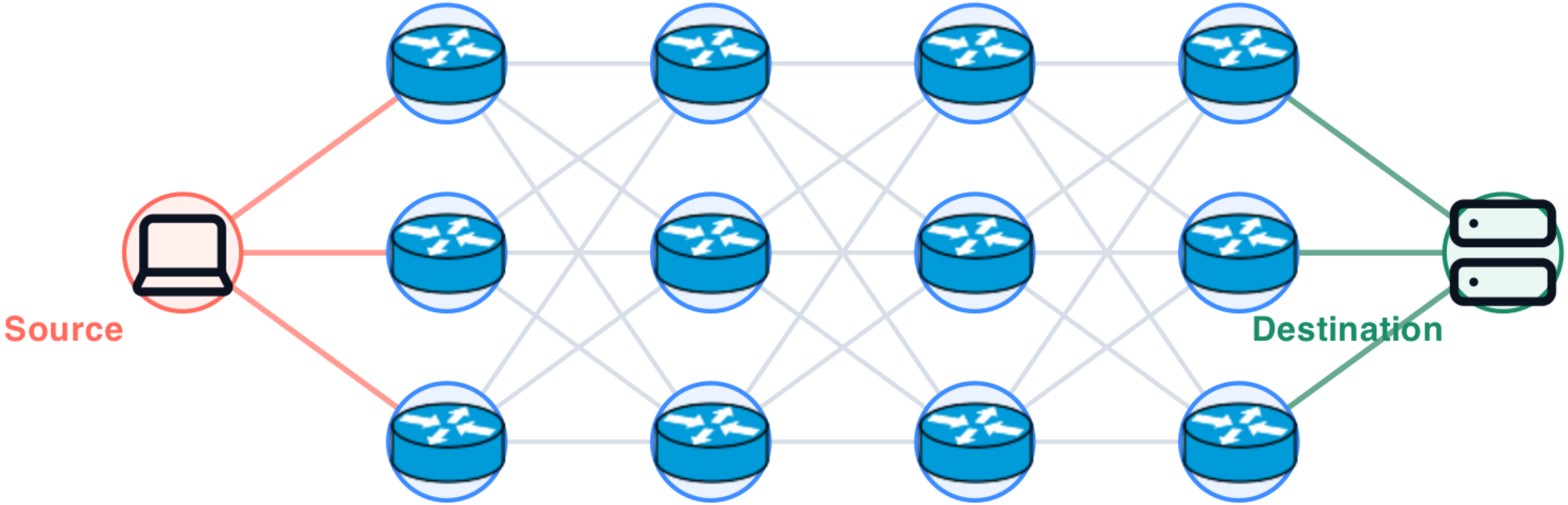
Part 2 · Routing + forwarding

02

Routing chooses. Forwarding executes.

Core structure | Routing + forwarding | Packet switching | Performance

Many possible paths make routing a difficult choice



Which route is best?

More possible paths make the routing decision harder.

Optimal depends on the metric

Shortest

Fewest hops or least distance

Lowest cost

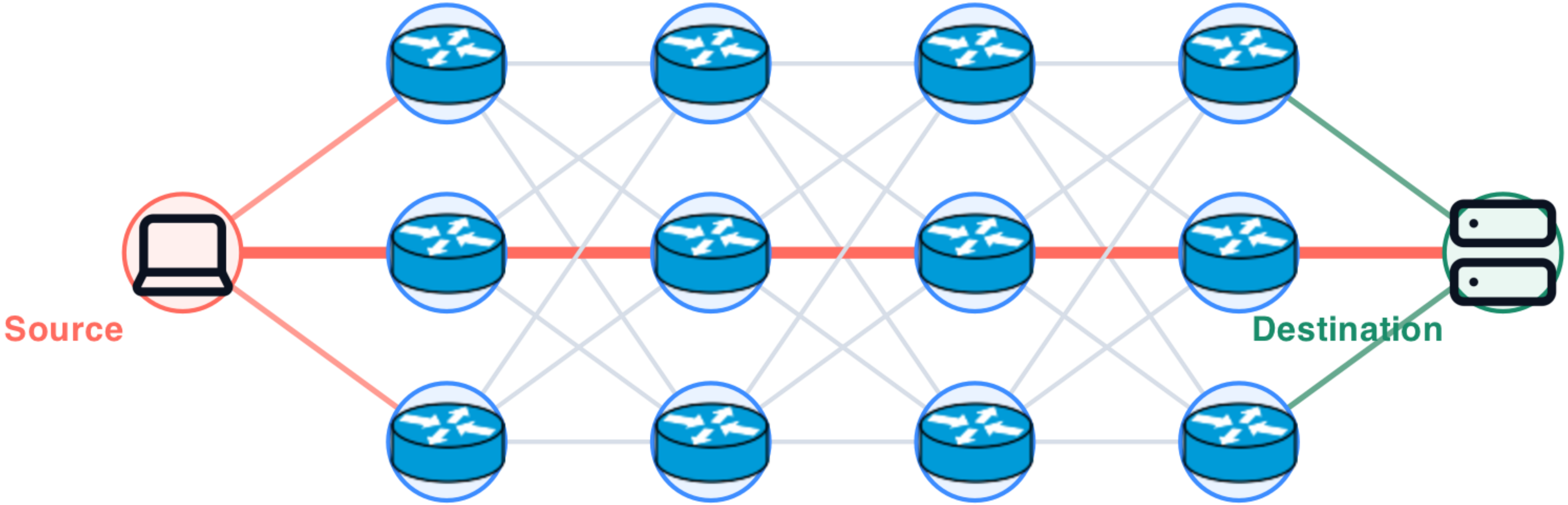
Cheapest links or agreements

Policy compliant

Paths allowed by operators

A route is optimal only relative to a stated objective and policy.

Routing selects one path through a complex topology

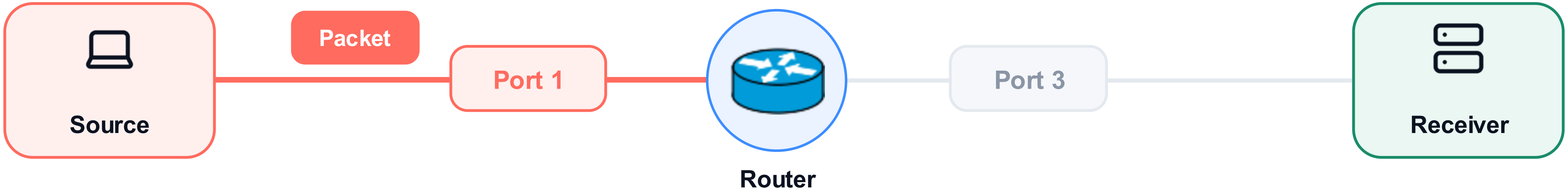


Routing is global: it chooses one path through the full topology.

Core structure | Routing + forwarding | Packet switching | Performance

Step 1 · The packet arrives on an input port

FORWARDING = receive → choose → send

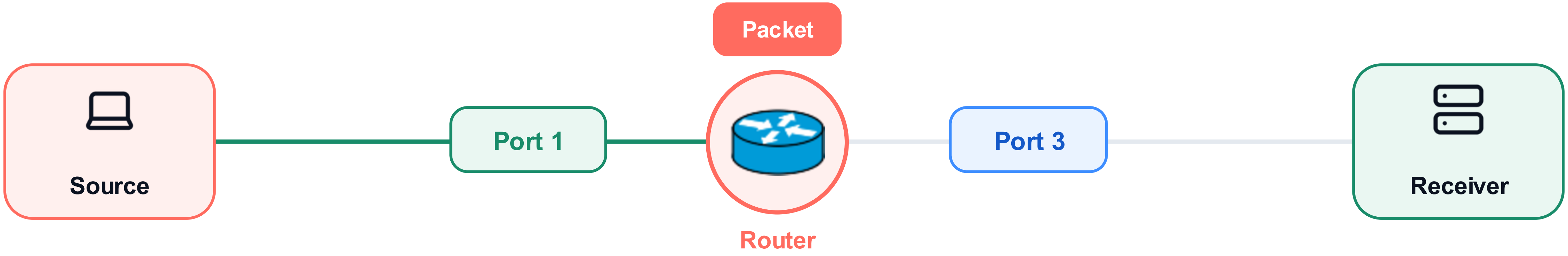


The router first receives one packet on Port 1.

Core structure | Routing + forwarding | Packet switching | Performance

Step 2 · The router chooses one output

FORWARDING = receive → choose → send

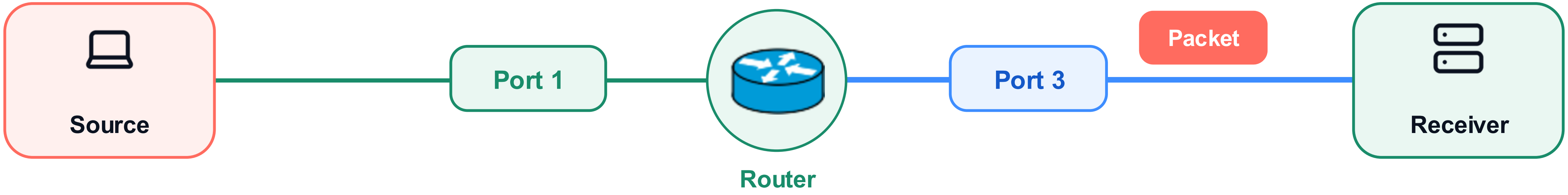


The router uses its forwarding information to choose Port 3.

Core structure | Routing + forwarding | Packet switching | Performance

Step 3 · The packet leaves through the selected output

FORWARDING = receive → choose → send

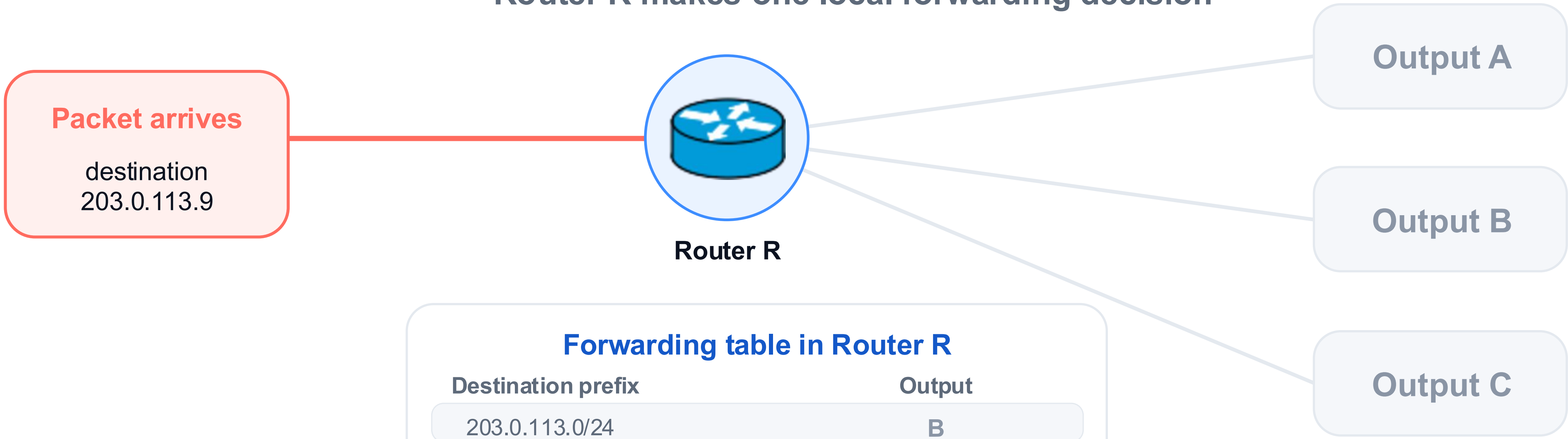


Forwarding is local: one router moves one packet from Port 1 to Port 3.

Core structure | Routing + forwarding | Packet switching | Performance

Step 1 · Read the packet's destination

Router R makes one local forwarding decision



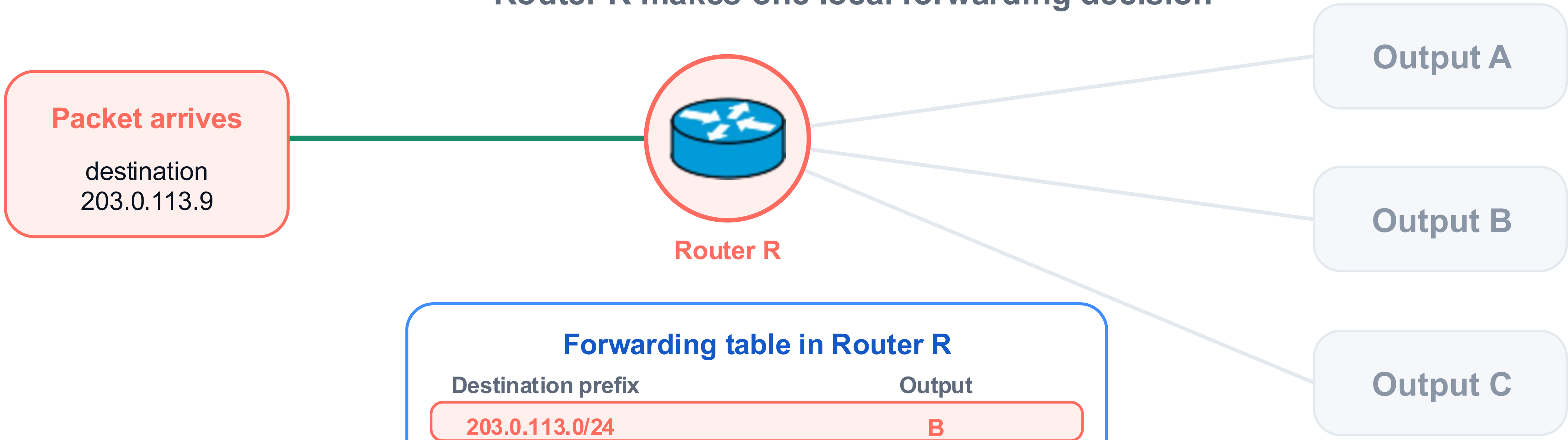
Forwarding table in Router R

Destination prefix	Output
203.0.113.0/24	B
198.51.100.0/24	A
0.0.0.0/0	C

The destination address is the lookup key.

Step 2 · Match the destination to a table entry

Router R makes one local forwarding decision



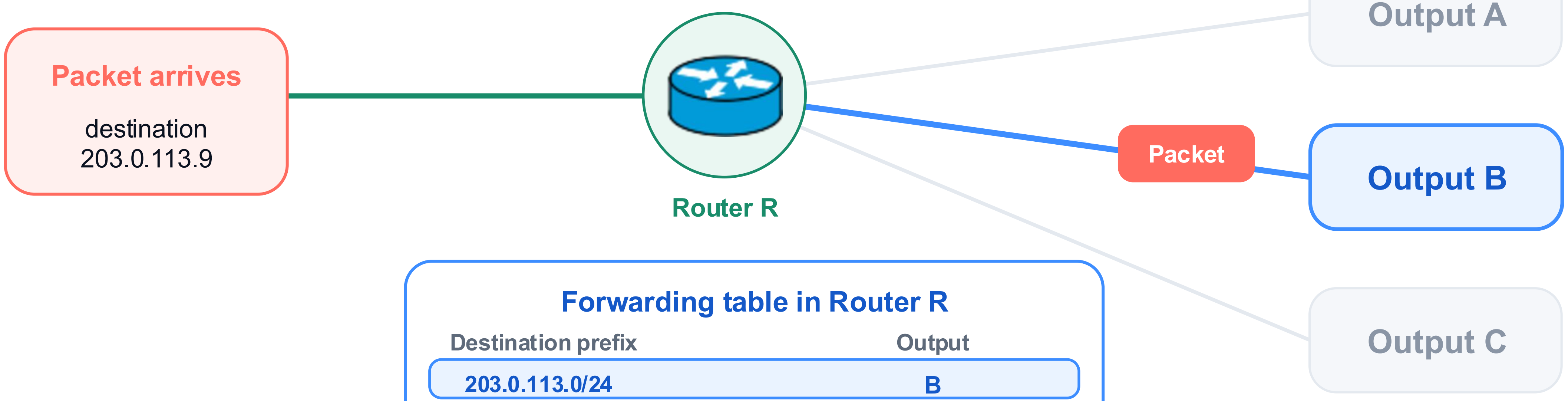
Forwarding table in Router R

Destination prefix	Output
203.0.113.0/24	B
198.51.100.0/24	A
0.0.0.0/0	C

203.0.113.9 matches 203.0.113.0/24, so the table selects B.

Step 3 · Send the packet through output B

Router R makes one local forwarding decision



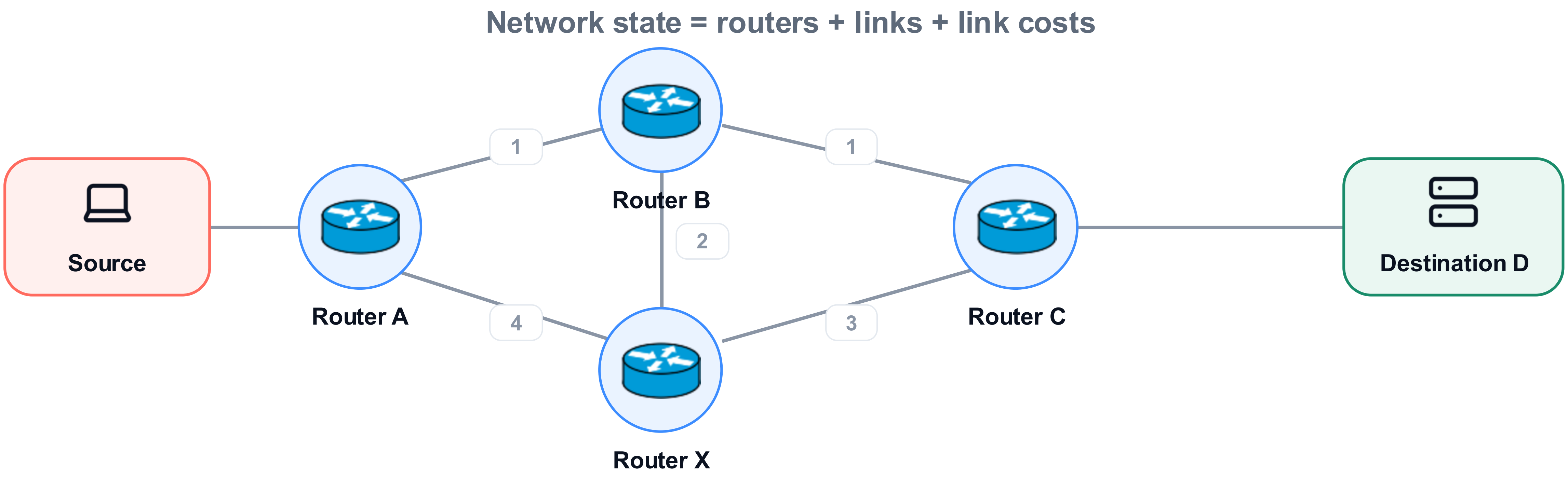
Forwarding table in Router R

Destination prefix	Output
203.0.113.0/24	**B**
198.51.100.0/24	A
0.0.0.0/0	C

Forwarding reads one local rule and activates one output.

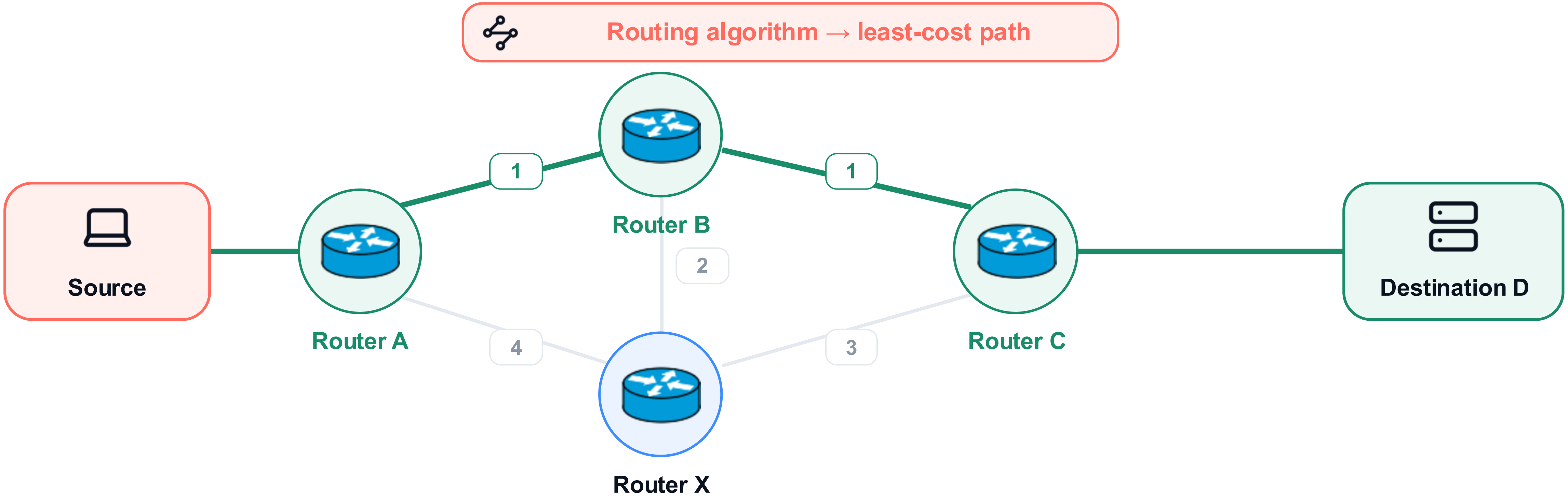
Core structure | Routing + forwarding | Packet switching | Performance

Routing starts with a network-wide topology



Routing reasons about the network as a whole.

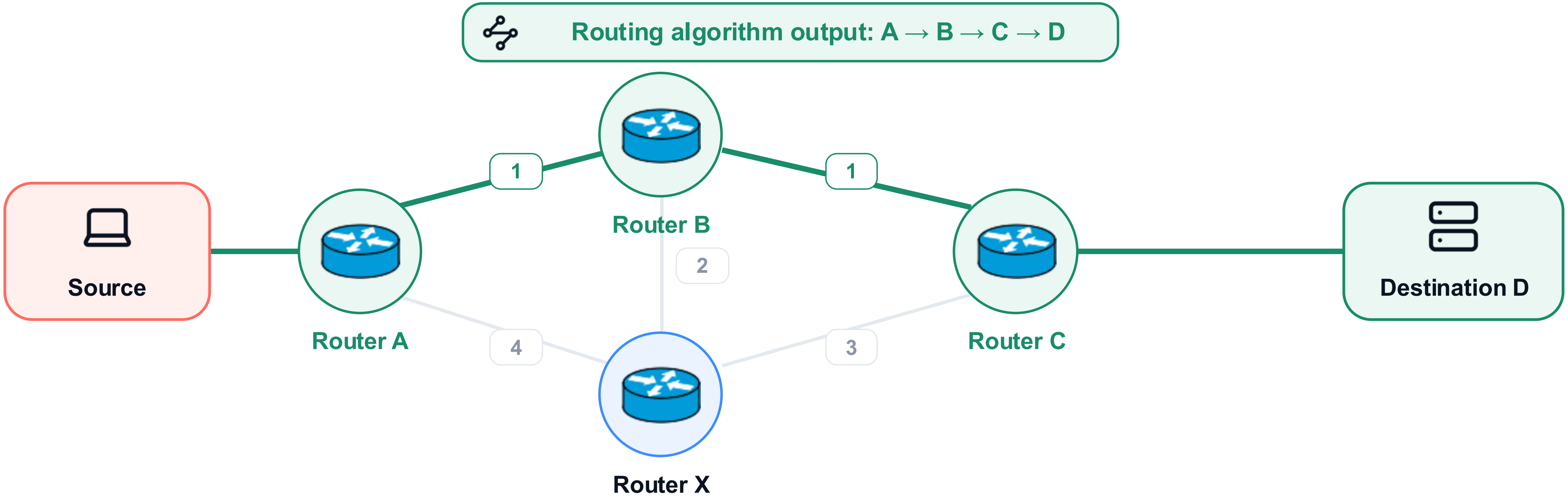
The routing algorithm selects one path through the topology



For this example, the least-cost route is Source → A → B → C → Destination.

Core structure | Routing + forwarding | Packet switching | Performance

For destination D, each router stores its next hop



Router A
Destination: D | Next hop: B

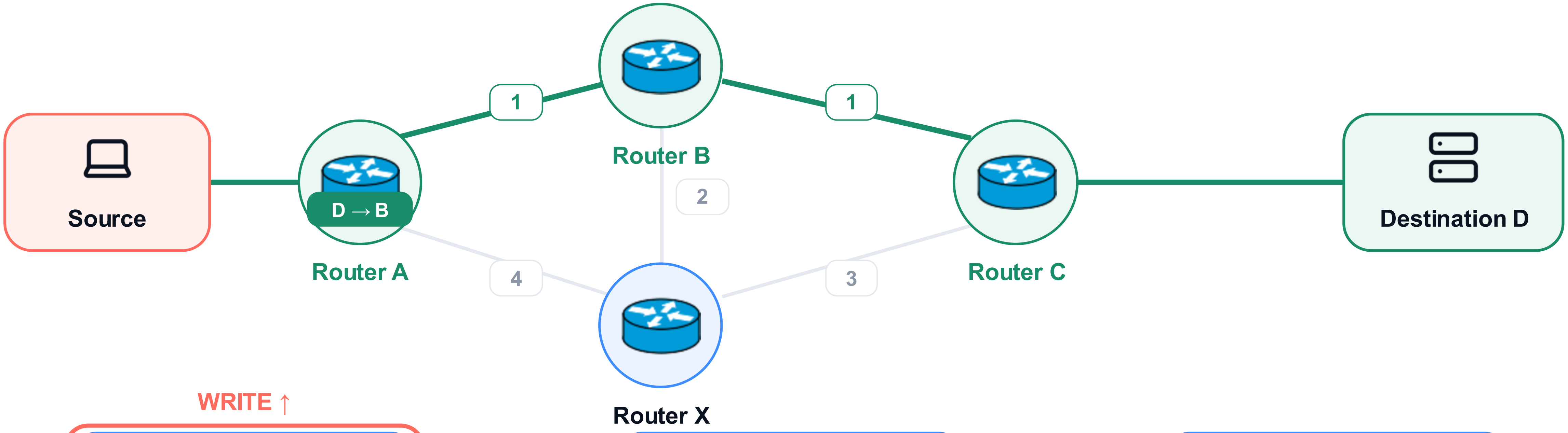
Router B
Destination: D | Next hop: C

Router C
Destination: D | Next hop: D

The algorithm computed three next-hop rules. Next, install one rule in each router.

Install Router A's rule: destination D → next hop B

Control plane writes local rule 1 of 3



WRITE ↑

Router A
Destination: D | Next hop: B

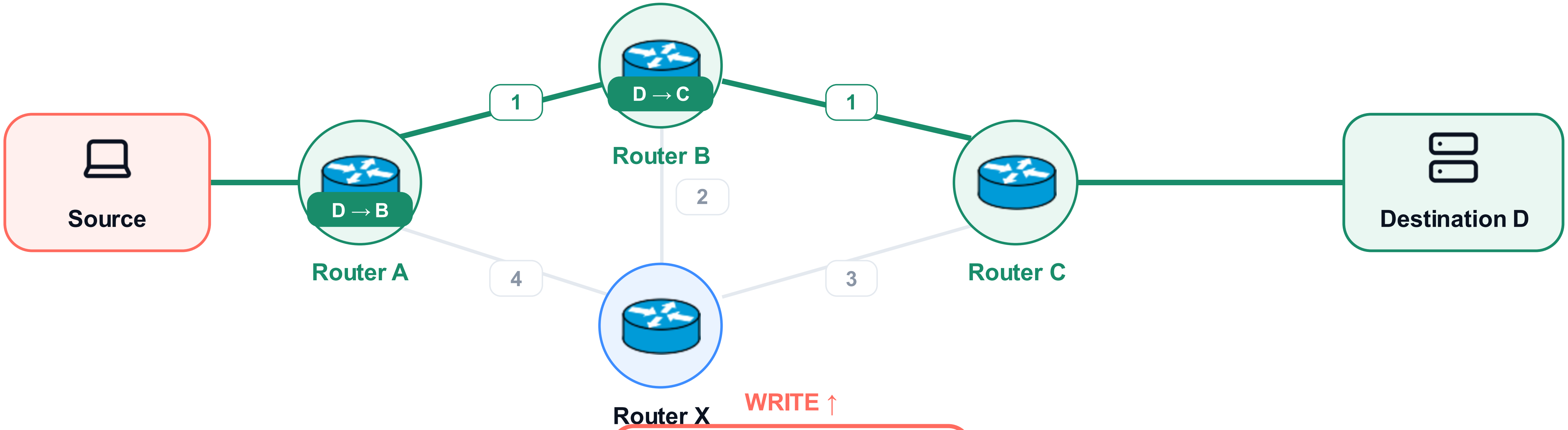
Router B
Destination: D | Next hop: C

Router C
Destination: D | Next hop: D

Router A now has the local rule it needs for destination D.

Install Router B's rule: destination D → next hop C

Control plane writes local rule 2 of 3



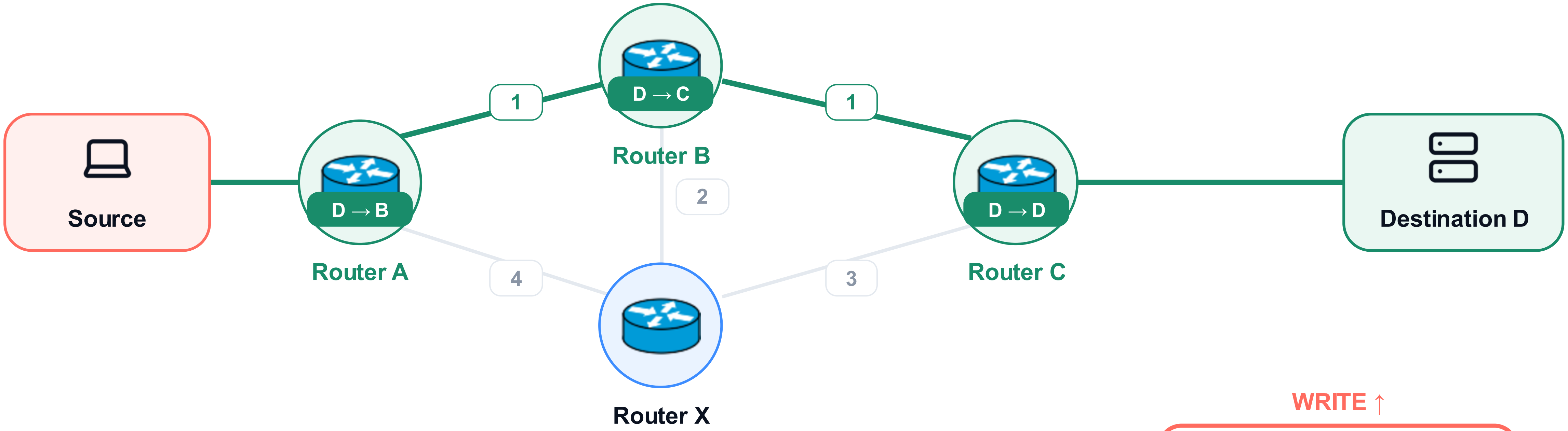
WRITE ↑

Router A	Router B	Router C
Destination: D | Next hop: B	Destination: D | Next hop: C	Destination: D | Next hop: D

Router B now has the local rule it needs for destination D.

Install Router C's rule: destination D → next hop D

Control plane writes local rule 3 of 3



Router A
Destination: D | Next hop: B

Router B
Destination: D | Next hop: C

WRITE ↑

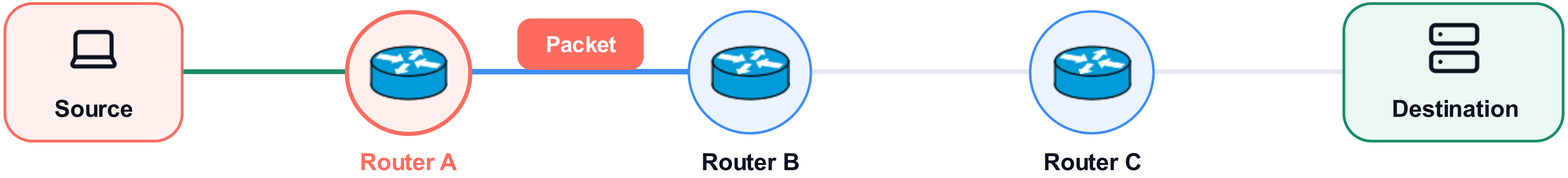
Router C
Destination: D | Next hop: D

The end-to-end path is now encoded as three local forwarding rules.

Core structure | Routing + forwarding | Packet switching | Performance

Router A forwards the packet to Router B

Each router consults only its own forwarding table



Router A table
D → next hop B

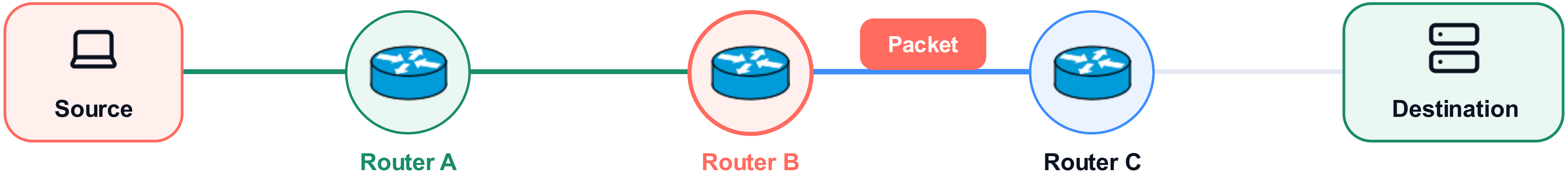
Router B table
D → next hop C

Router C table
D → local delivery

One table lookup changes only Router A's local action.

Router B forwards the packet to Router C

Each router consults only its own forwarding table



Router A table
D → next hop B

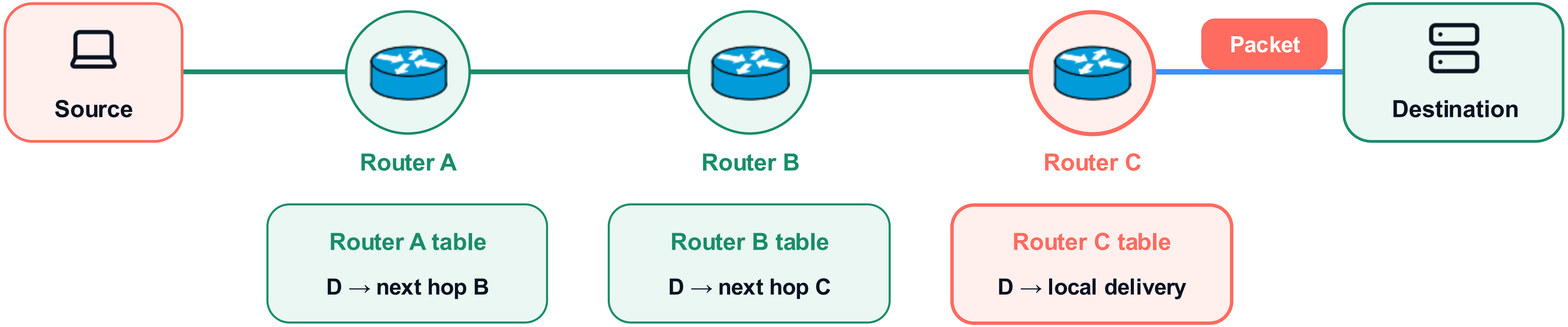
Router B table
D → next hop C

Router C table
D → local delivery

The packet reaches Router B, which makes a new local decision.

Router C forwards the packet to the destination

Each router consults only its own forwarding table

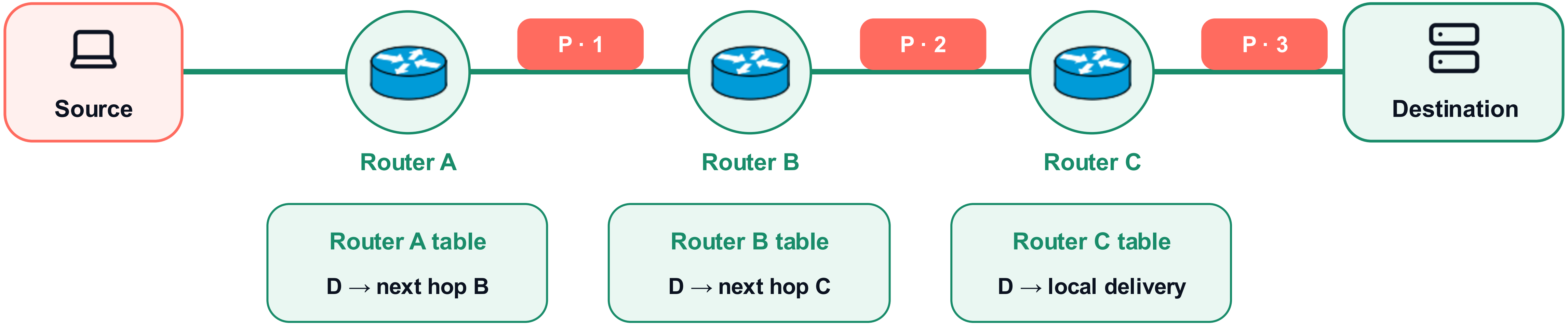


Repeated local decisions carry the packet across the selected route.

Routing chooses the path; forwarding moves each packet

routing (global) → forwarding tables (local state) → forwarding (per packet)

the same packet shown at three successive hops



Routing selected the path; repeated local forwarding actions realize the end-to-end route.

Part 3 · Packet switching

03

Packets share links on demand.

Two switching models allocate resources differently

Packet switching

Share links on demand

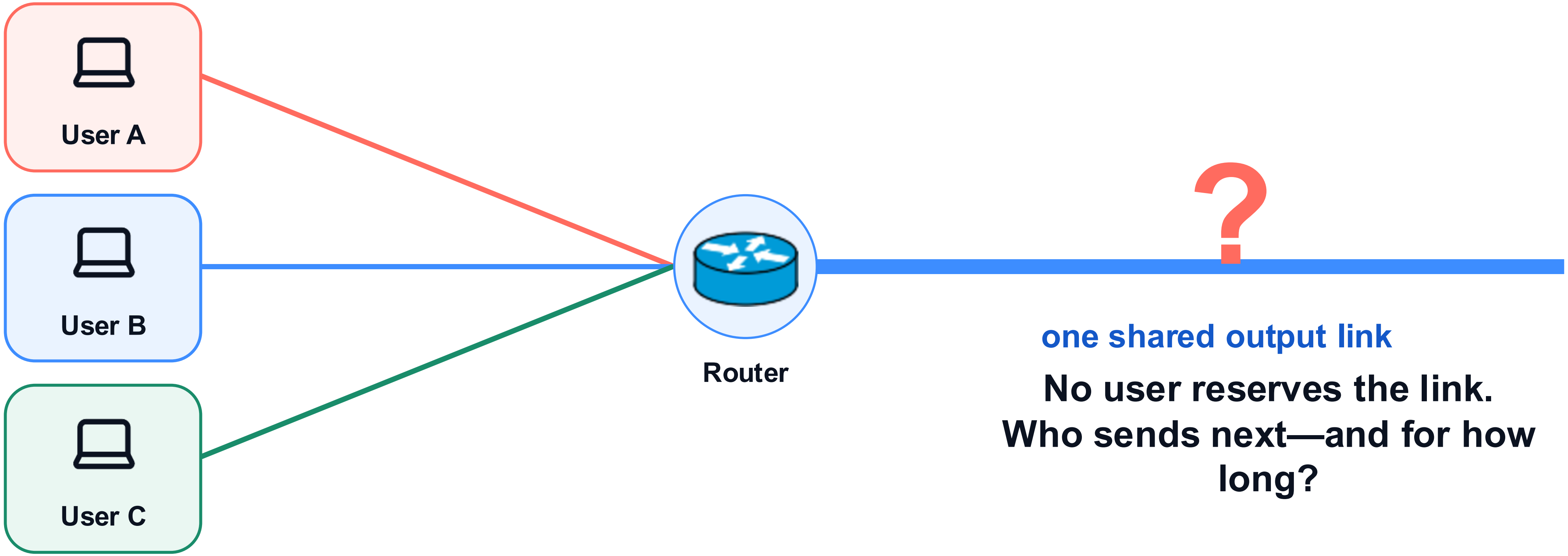
Circuit switching

Reserve a path before sending

Both models move data, but they allocate link capacity differently.

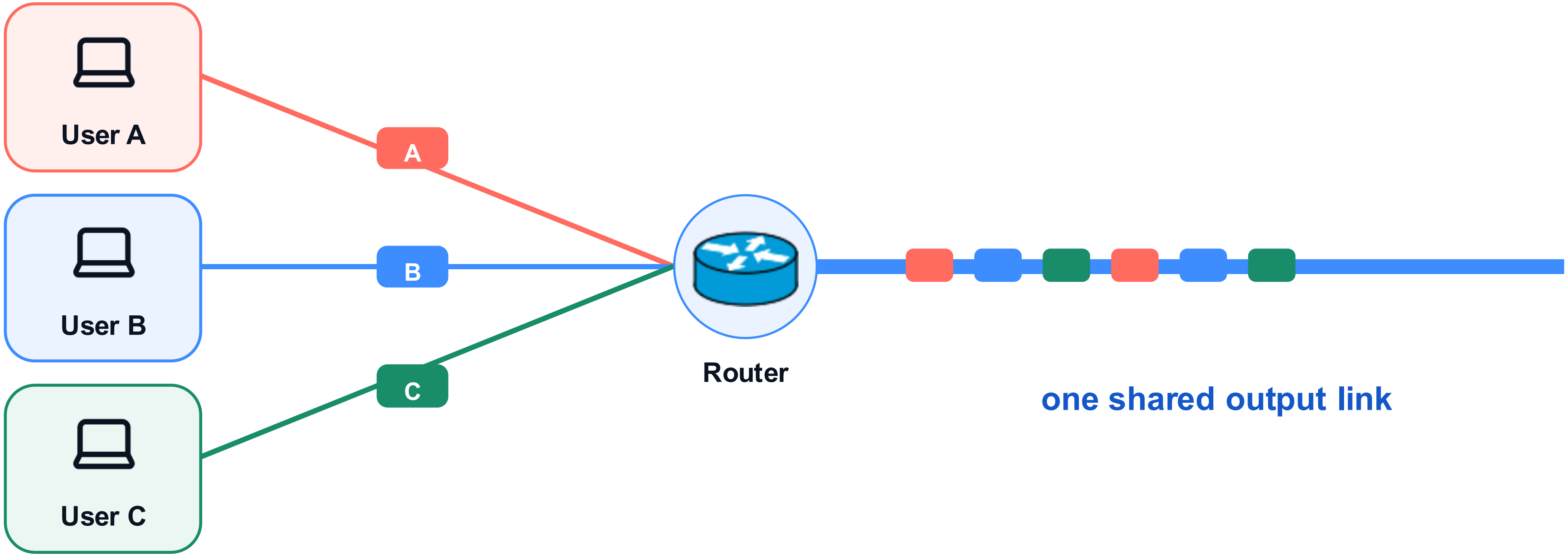
Core structure | Routing + forwarding | Packet switching | Performance

How can several users share one outgoing link?



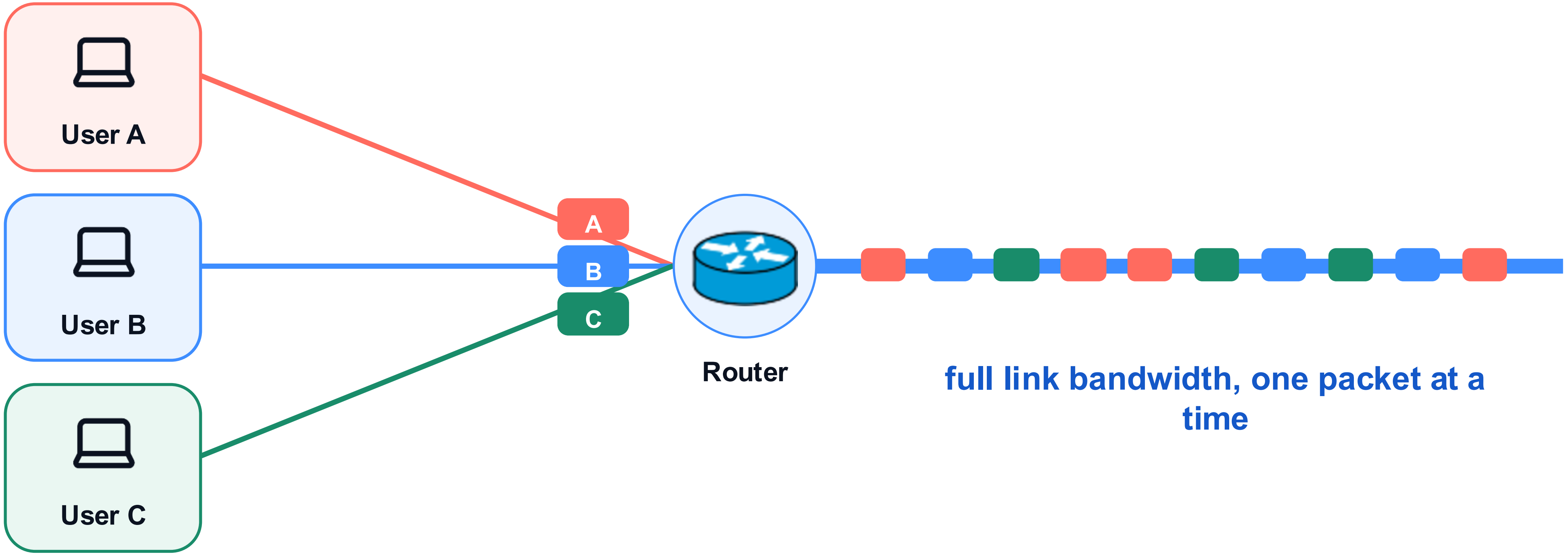
Question: can the router interleave packets from all three users?

Packets from several users share one outgoing link



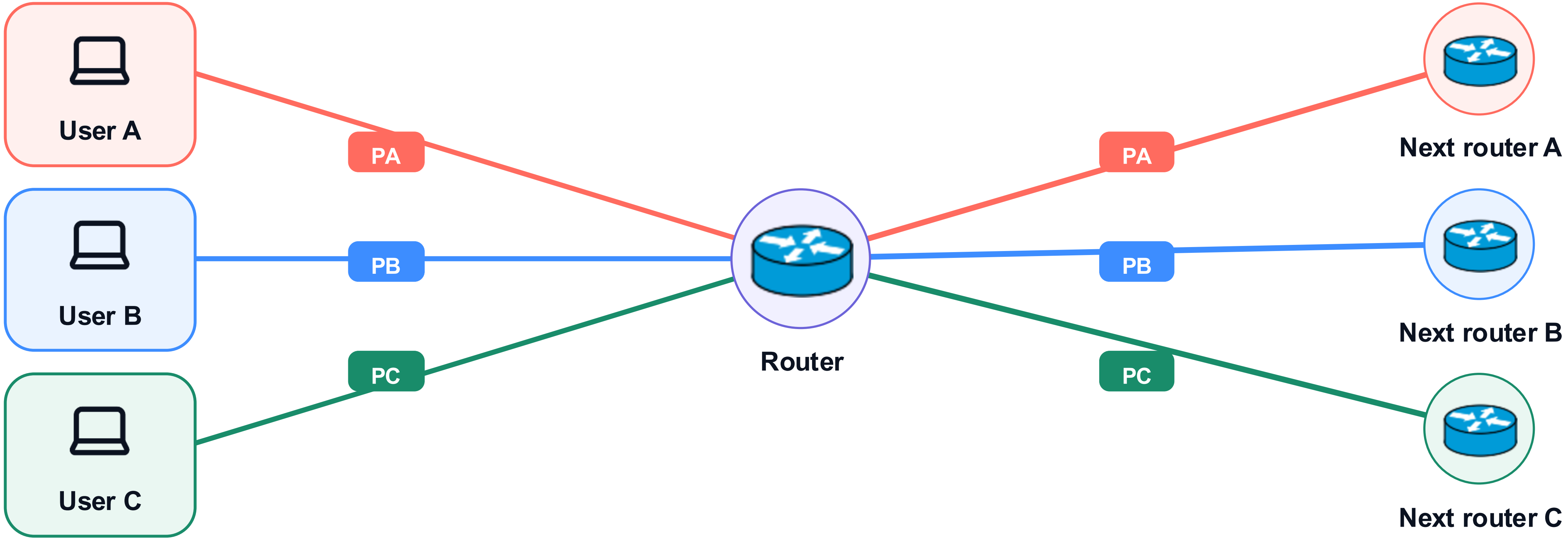
Packets from different users take turns on the same link.

Packets from many users share the same links



Packet switching interleaves users without reserving fixed slices of the link.

Three packets can take three different next hops

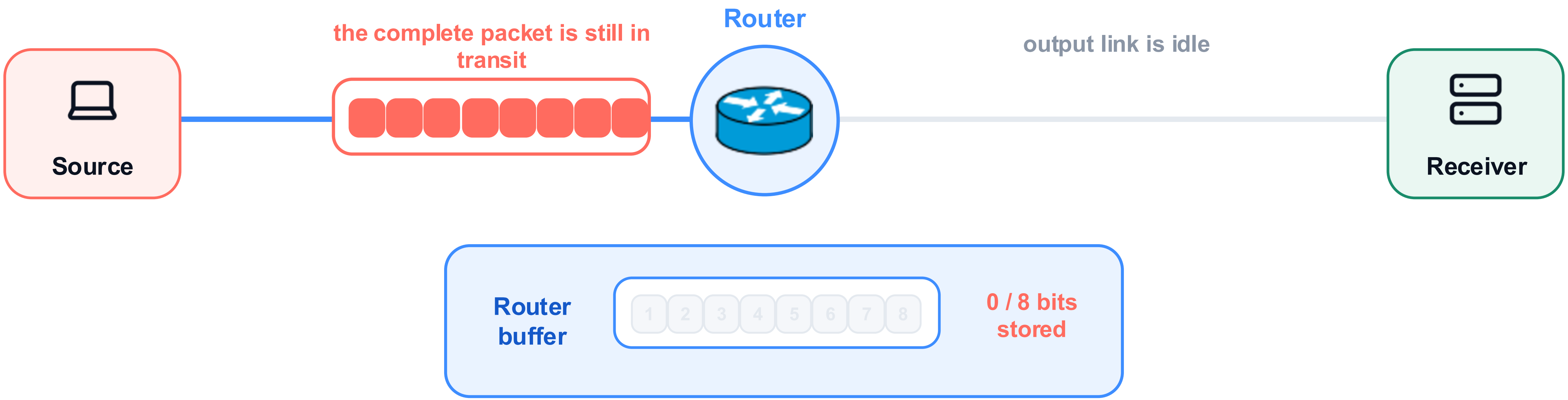


The router reads each destination and can send different packets to different next routers.

The packet is still traveling toward the router

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

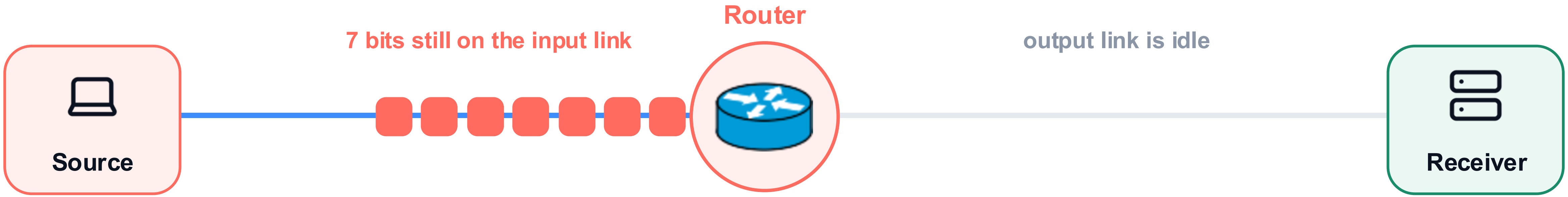


The router has received 0 / 8 bits, so it cannot transmit.

Bit 1 arrives and is stored in the router buffer

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet



Router buffer

1	2	3	4	5	6	7	8

1 / 8 bits stored

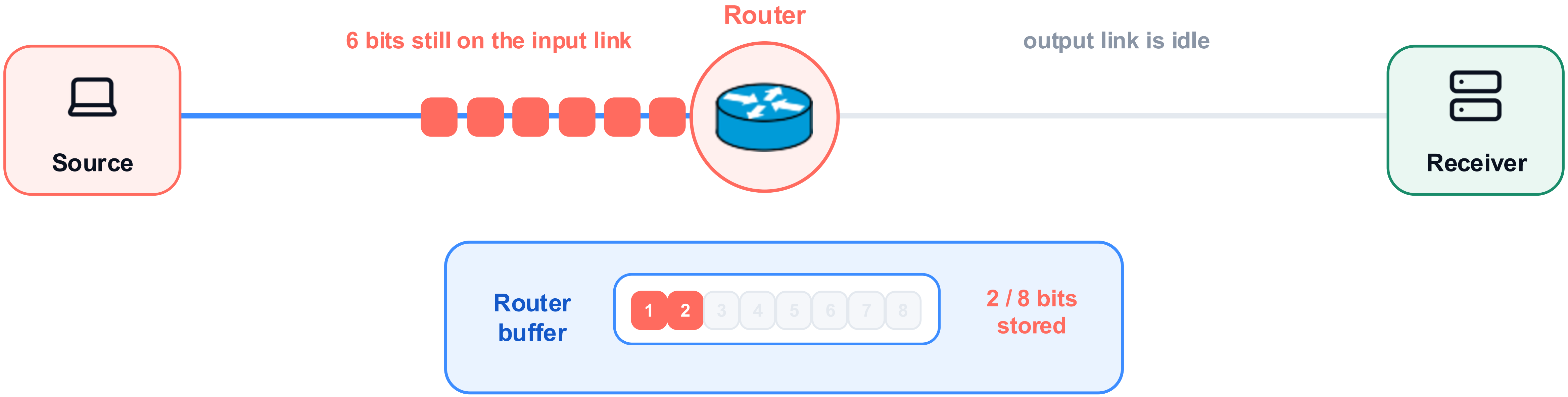
The output stays empty: store-and-forward is still waiting for bit 8.

Core structure | Routing + forwarding | Packet switching | Performance

Bit 2 arrives and is stored in the router buffer

1 RECEIVE + STORE ALL 8 BITS → **2 TRANSMIT**

one square = one bit · eight bits = one packet

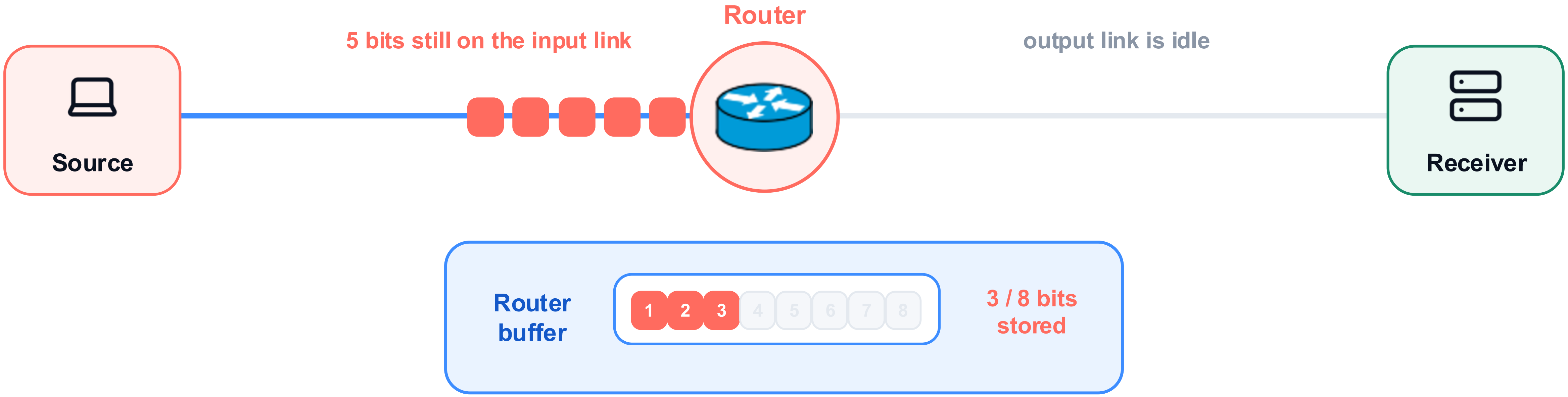


The output stays empty: store-and-forward is still waiting for bit 8.

Bit 3 arrives and is stored in the router buffer

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet



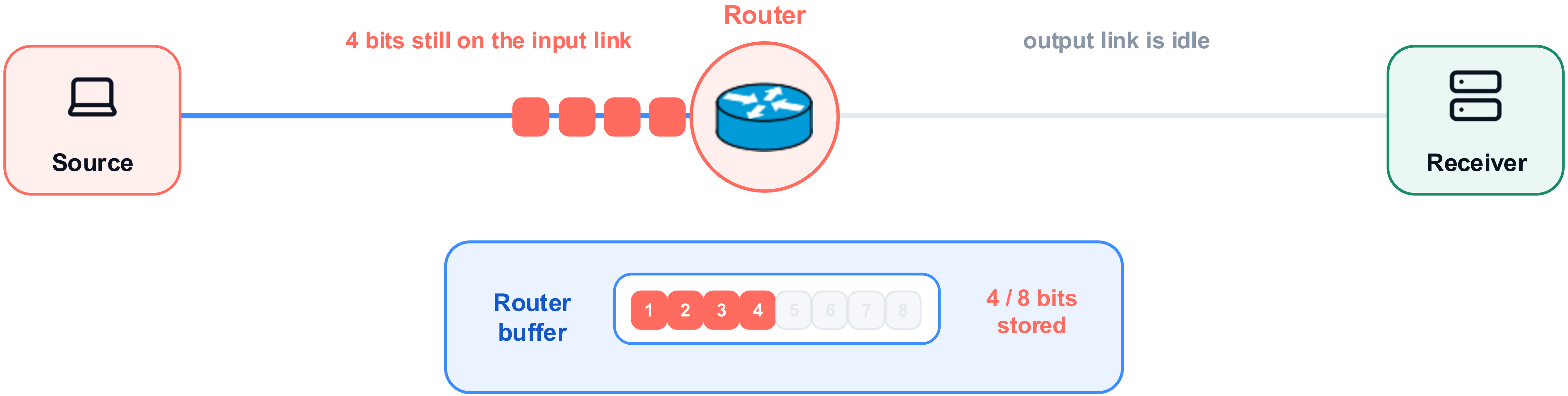
The output stays empty: store-and-forward is still waiting for bit 8.

Core structure | Routing + forwarding | Packet switching | Performance

Bit 4 arrives and is stored in the router buffer

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet



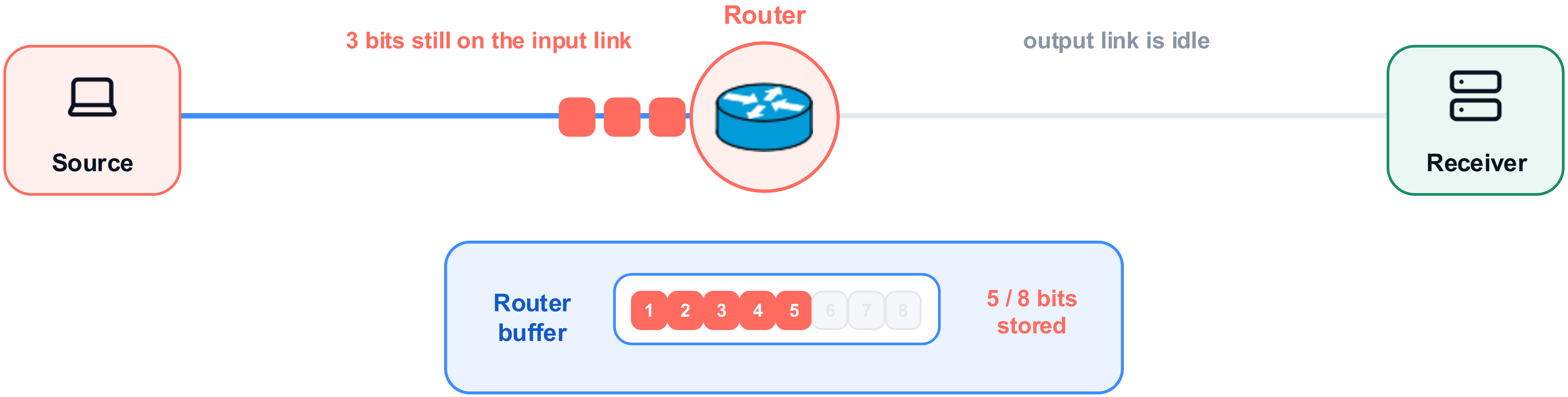
The output stays empty: store-and-forward is still waiting for bit 8.

Core structure | Routing + forwarding | Packet switching | Performance

Bit 5 arrives and is stored in the router buffer

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

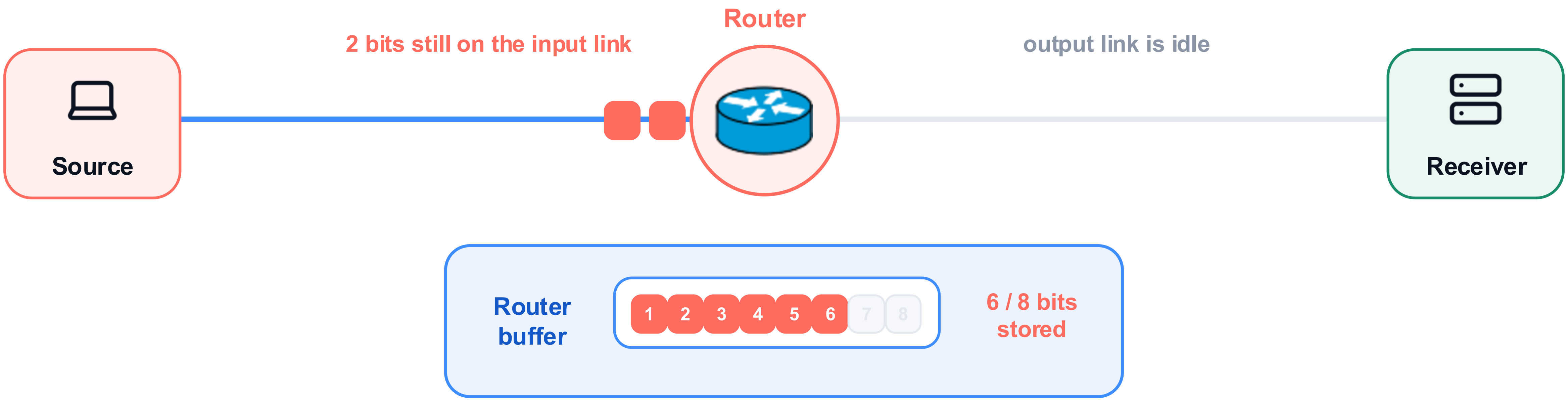


The output stays empty: store-and-forward is still waiting for bit 8.

Bit 6 arrives and is stored in the router buffer

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

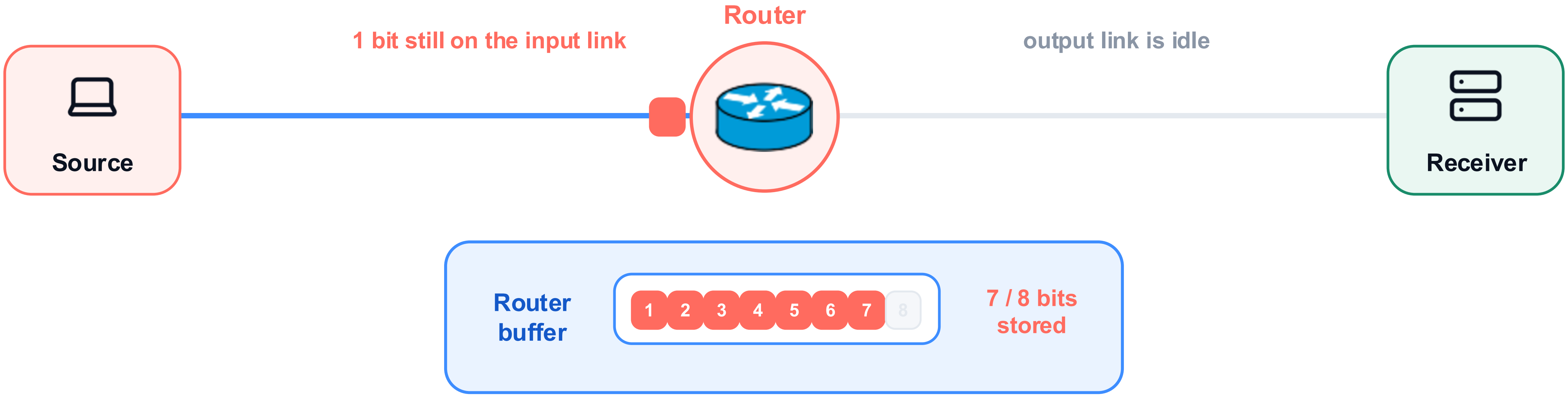


The output stays empty: store-and-forward is still waiting for bit 8.

Bit 7 arrives and is stored in the router buffer

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

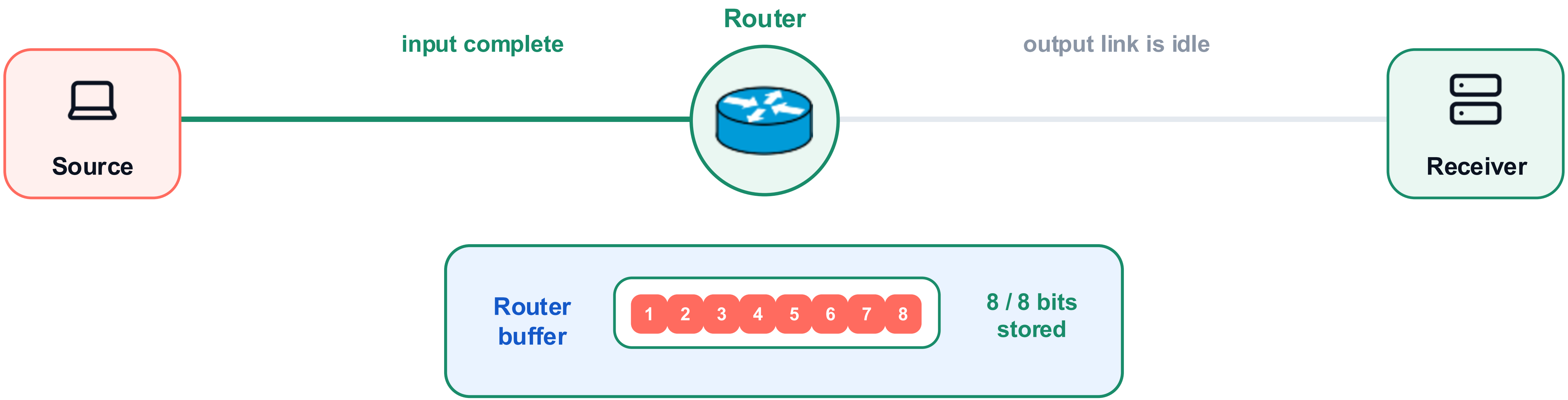


The output stays empty: store-and-forward is still waiting for bit 8.

All 8 bits are buffered; transmission may now begin

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet



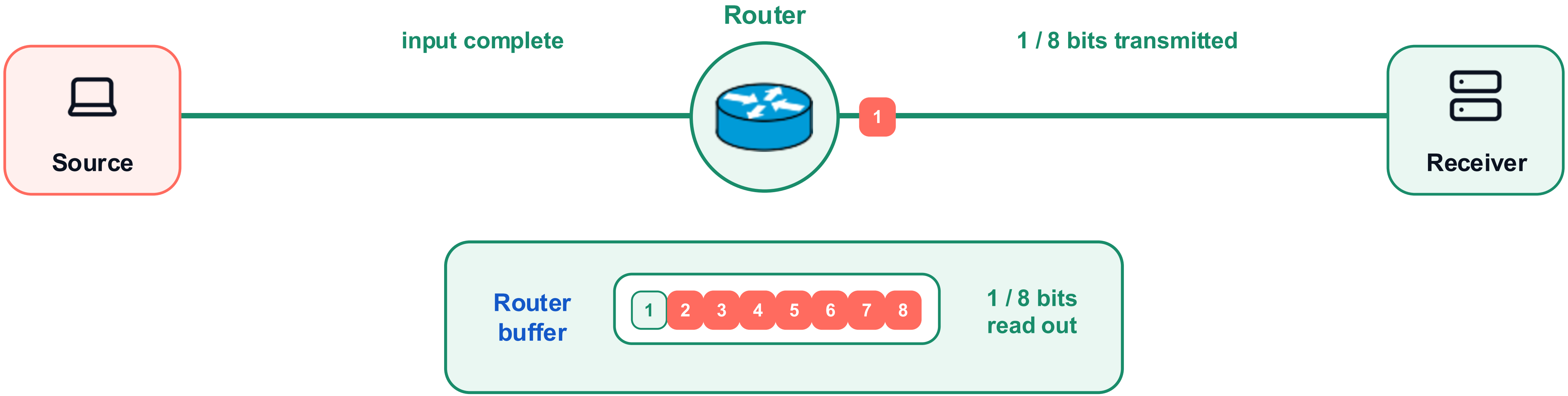
Only after bit 8 arrives may the router start transmitting on the next link.

Core structure | Routing + forwarding | Packet switching | Performance

The router transmits bit 1 onto the next link

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

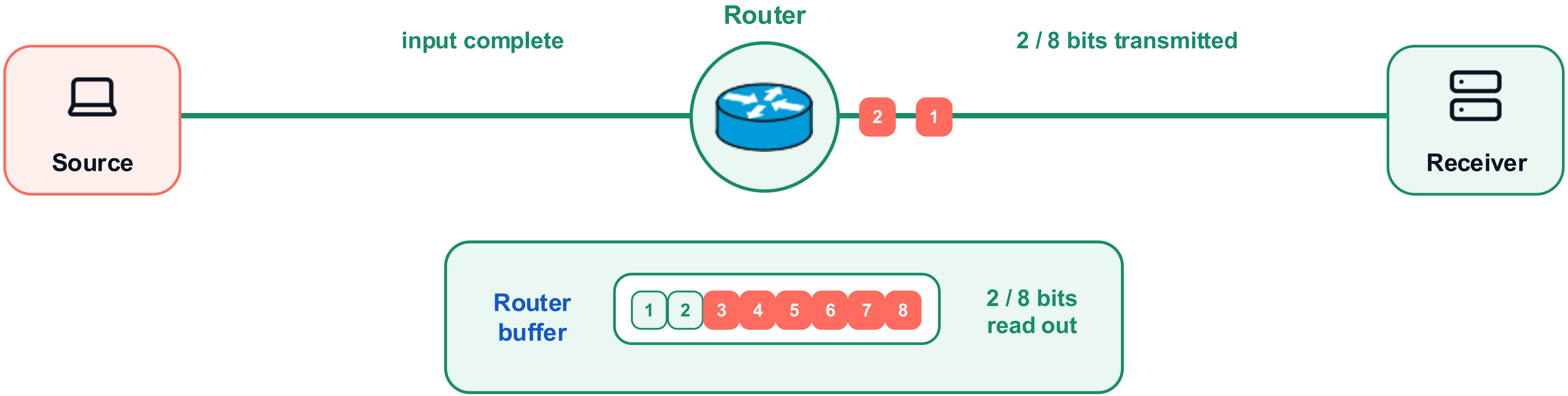


The complete packet was stored first; now the router sends one bit at a time.

The router transmits bit 2 onto the next link

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

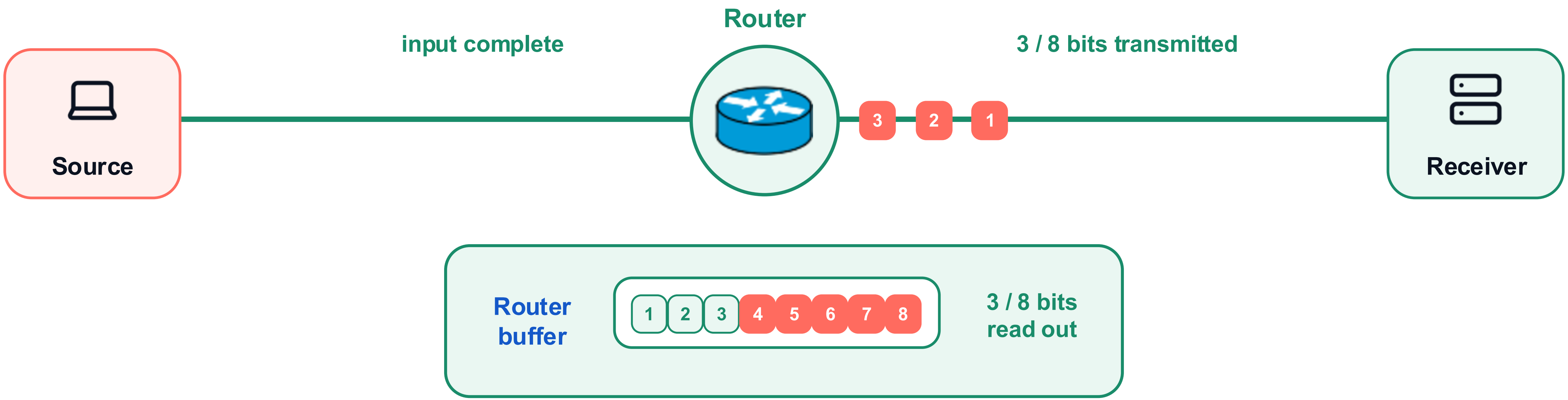


The complete packet was stored first; now the router sends one bit at a time.

The router transmits bit 3 onto the next link

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

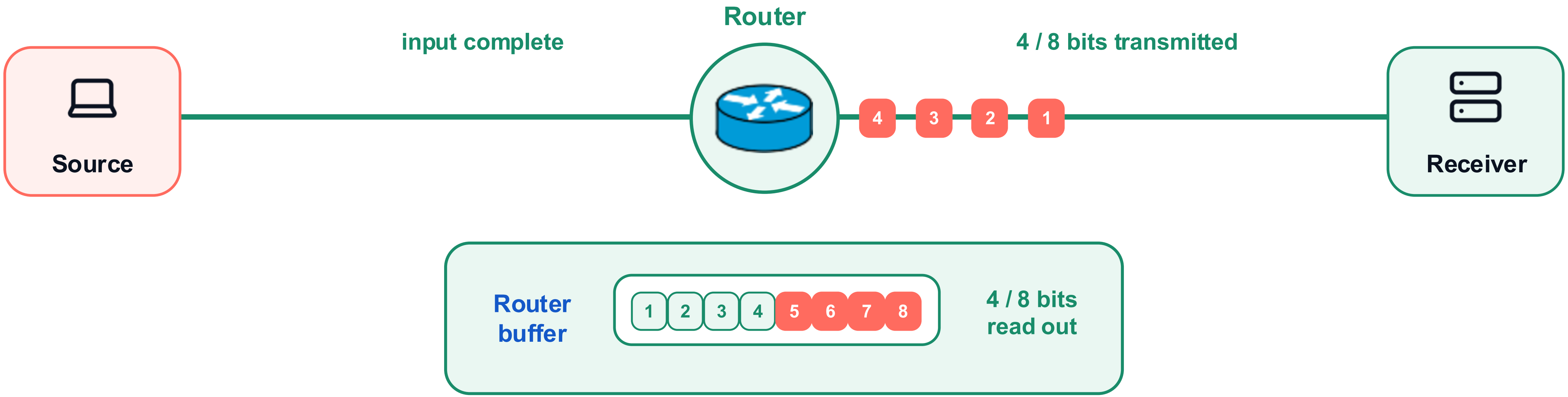


The complete packet was stored first; now the router sends one bit at a time.

The router transmits bit 4 onto the next link

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

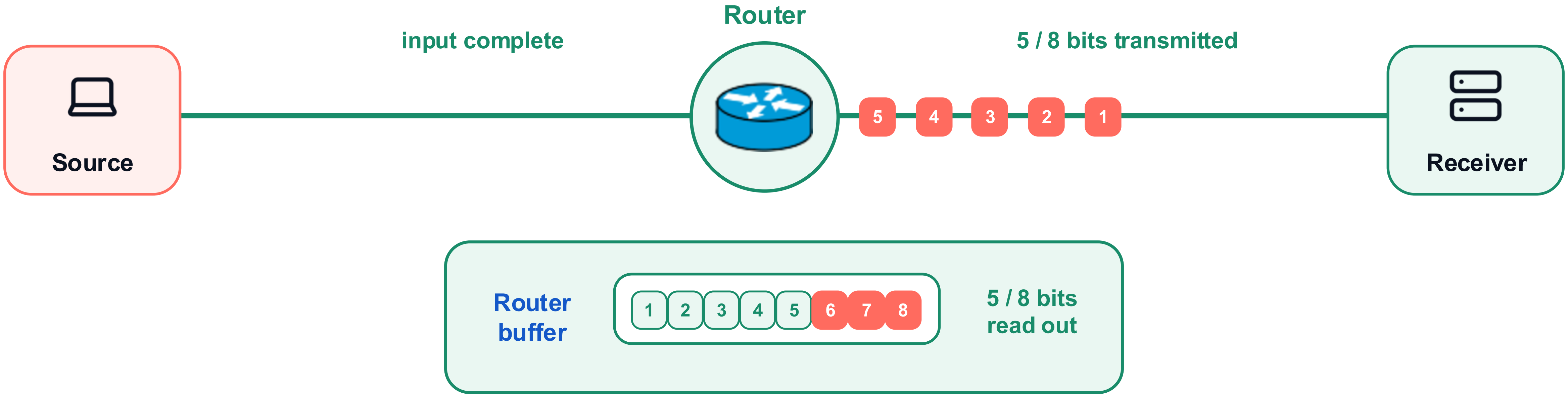


The complete packet was stored first; now the router sends one bit at a time.

The router transmits bit 5 onto the next link

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

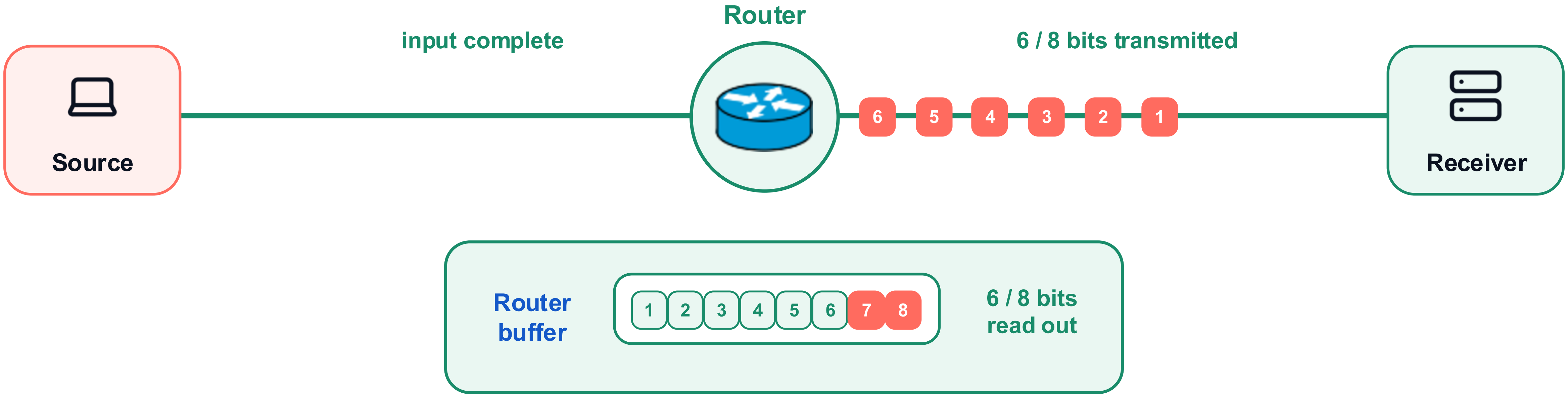


The complete packet was stored first; now the router sends one bit at a time.

The router transmits bit 6 onto the next link

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

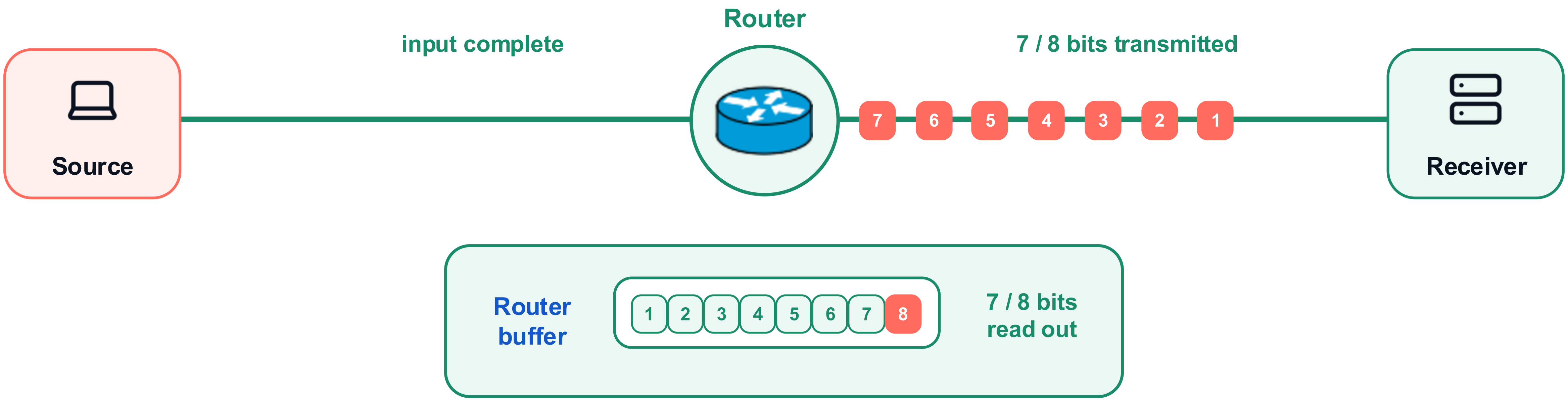


The complete packet was stored first; now the router sends one bit at a time.

The router transmits bit 7 onto the next link

1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet

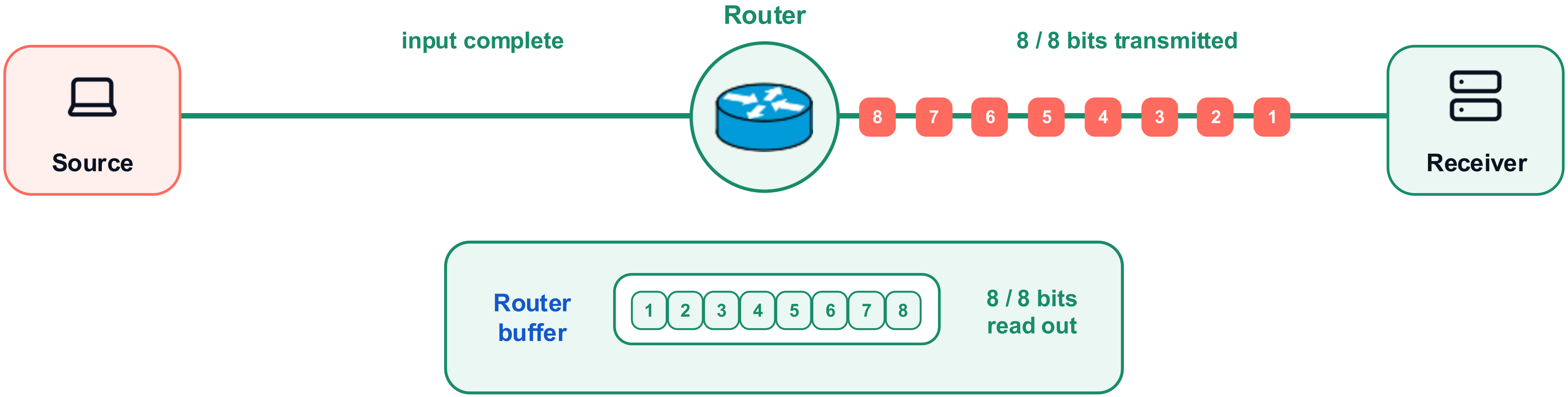


The complete packet was stored first; now the router sends one bit at a time.

All 8 bits have been transmitted onto the next link

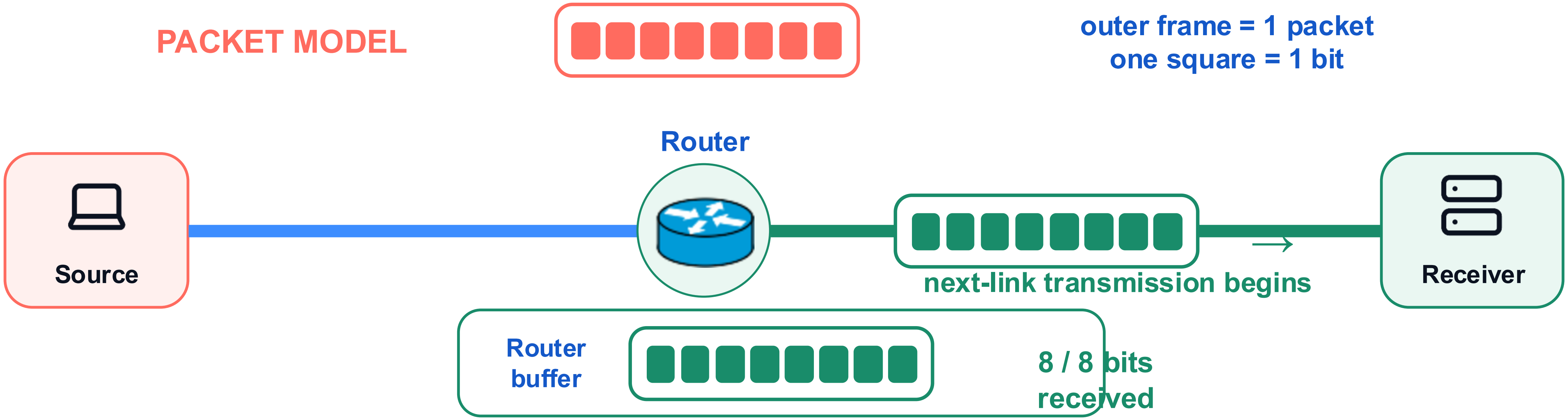
1 RECEIVE + STORE ALL 8 BITS → 2 TRANSMIT

one square = one bit · eight bits = one packet



Store first. Then forward.

Store-and-forward waits for all 8 bits

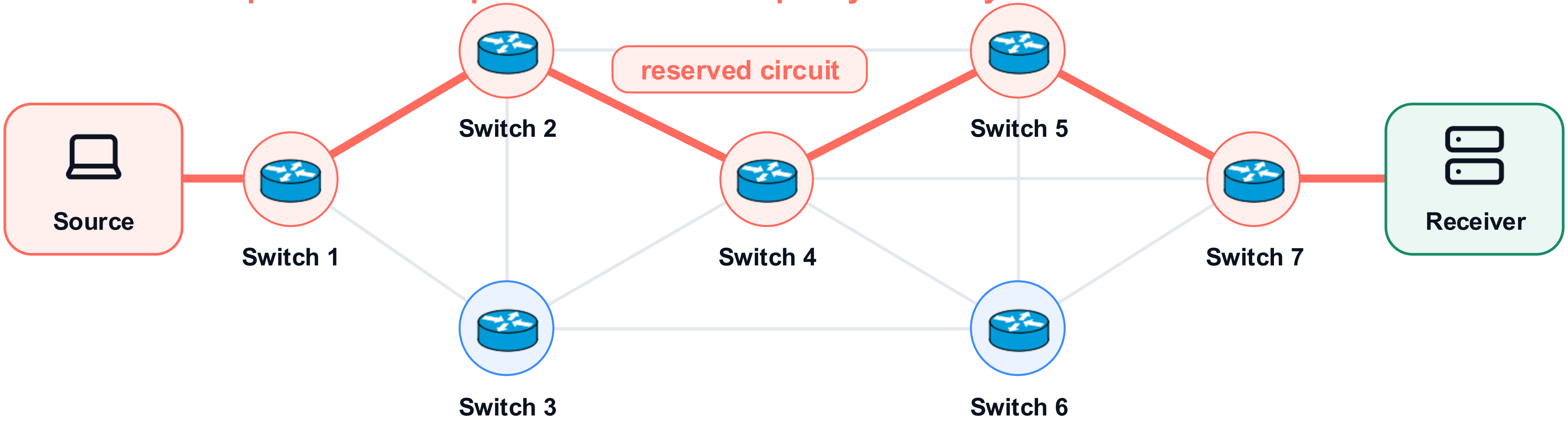


Cut-through comparison may start the next link before all 8 bits arrive

At 8/8 bits, the next-link transmission may begin.

Circuit switching reserves an end-to-end path

Setup chooses one path and reserves capacity on every red link before data flows.



1 set up the path → 2 reserve capacity on each link → 3 transmit

The circuit owns reserved capacity across several switches for the duration of the connection.

Packet switching shares link capacity on demand

Packet switching

- **Shares link capacity packet by packet**
- No setup; efficient for bursty traffic
- Queues can create variable delay and loss

Core structure | Routing + forwarding | Packet switching | Performance

Circuit switching reserves capacity before data flows

Packet switching

- **Shares link capacity packet by packet**
- No setup; efficient for bursty traffic
- Queues can create variable delay and loss

Circuit switching

- **Reserves capacity on every link**
- Requires setup before data flows
- Predictable service; idle capacity stays reserved

Packet and circuit switching make different tradeoffs

Packet switching

- **Shares link capacity packet by packet**
- No setup; efficient for bursty traffic
- Queues can create variable delay and loss

Circuit switching

- **Reserves capacity on every link**
- Requires setup before data flows
- Predictable service; idle capacity stays reserved

Packet switching favors statistical sharing; circuit switching favors reservation.

Why packet sharing is efficient

Why sharing works

- **Idle capacity can serve another user**
- Bursty users can share the same link
- No per-flow reservation is required

The same sharing can create congestion

Why sharing works

- **Idle capacity can serve another user**
- Bursty users can share the same link
- No per-flow reservation is required

When sharing breaks down

- **Simultaneous bursts build queues**
- Full buffers drop packets
- Congestion control must reduce the load

Packet switching is efficient, but congestion is real

Why sharing works

- **Idle capacity can serve another user**
- Bursty users can share the same link
- No per-flow reservation is required

When sharing breaks down

- **Simultaneous bursts build queues**
- Full buffers drop packets
- Congestion control must reduce the load

Packet switching is not a slam dunk; its strength and its risk both come from sharing.

Part 4 · Performance

04

Follow one packet: delay, loss, throughput.

Core structure | Routing + forwarding | Packet switching | Performance

Performance follows one packet in order

1 **Transmission**

push bits into link

2 **Propagation**

travel A → B

3 **Processing**

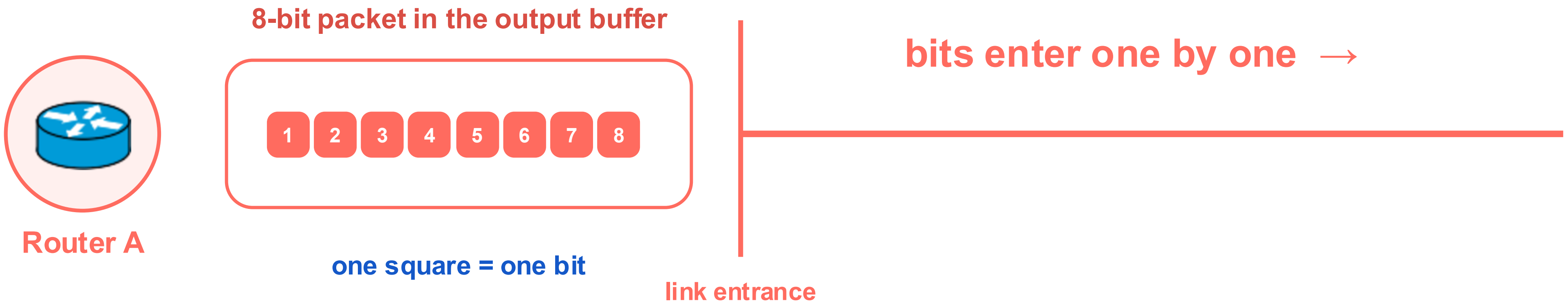
Router B examines packet

4 **Queueing**

wait behind packets

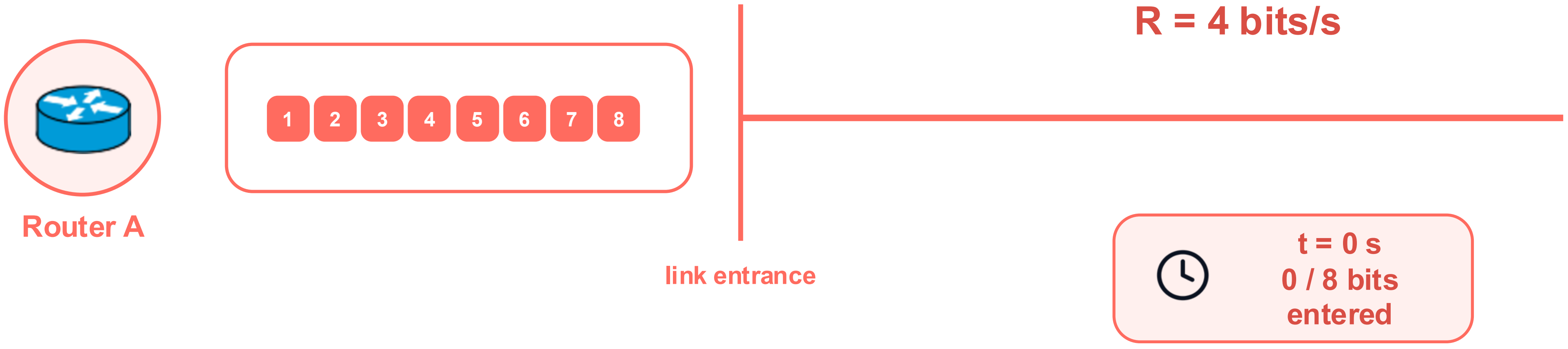
Follow the packet: onto the link, across the link, through the router, then into the buffer.

Start at Router A: put the packet's bits onto the link



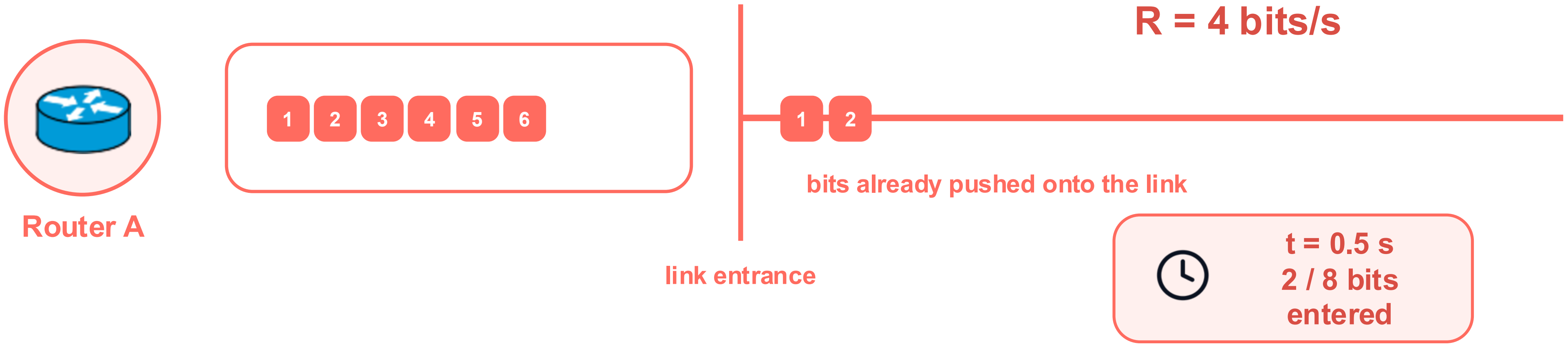
Transmission is the act of pushing the packet's bits onto the link.

The link accepts bits at transmission rate R



Transmission rate tells how many bits enter the link each second.

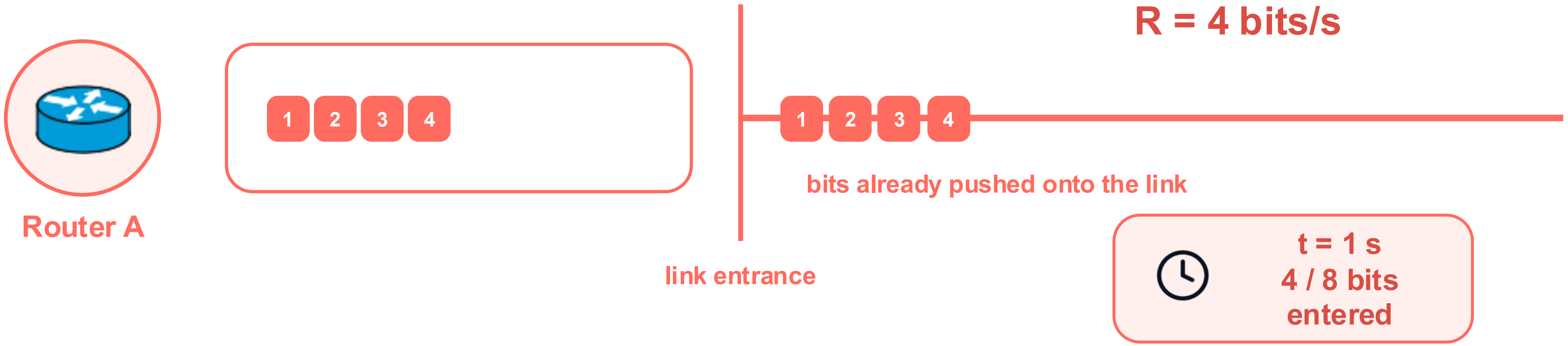
A faster link puts more bits onto the link each second



Transmission rate tells how many bits enter the link each second.

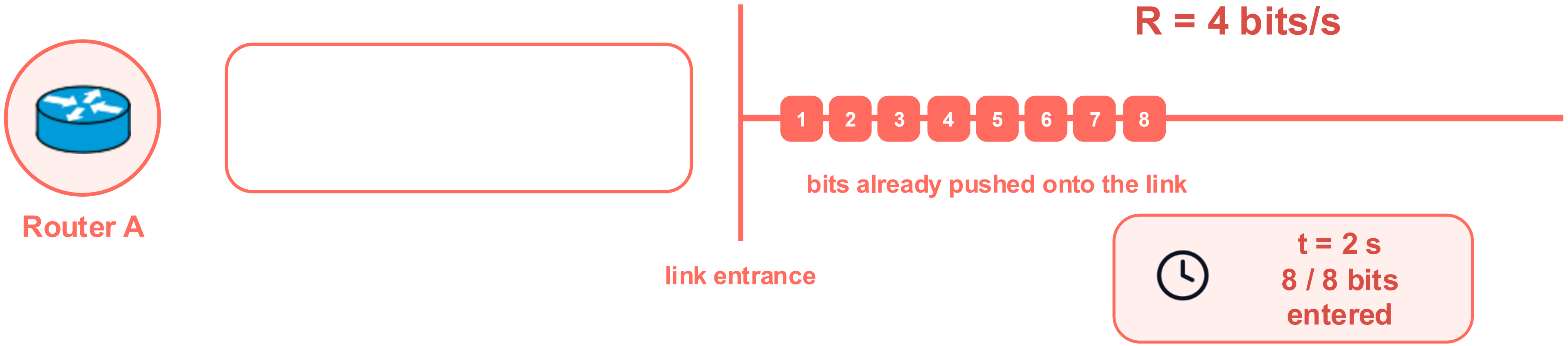
Core structure | Routing + forwarding | Packet switching | Performance

At R = 4 bits/s, four bits enter in one second



Transmission rate tells how many bits enter the link each second.

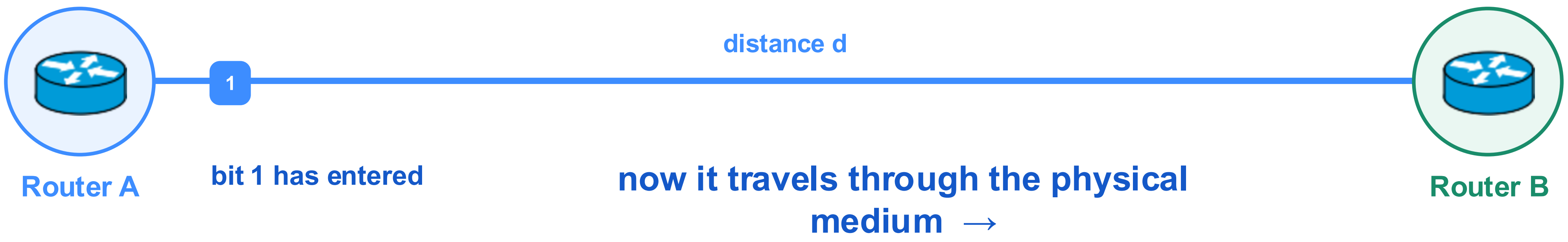
Transmission delay ends when the last bit enters the link



d_trans = L / R = 8 bits / 4 bits/s = 2 s

Higher transmission rate R → shorter transmission delay.

After a bit enters the link, it still must reach Router B



Transmission put the bit onto the link; propagation moves it across the link.

Core structure | Routing + forwarding | Packet switching | Performance

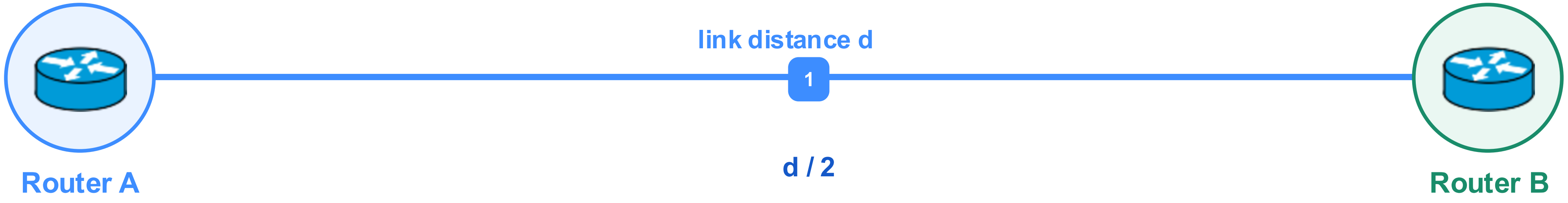
Propagation starts when the bit enters the physical link

propagation speed s ≈ speed of light in the medium

Propagation delay measures physical travel time from A to B.

Core structure | Routing + forwarding | Packet switching | Performance

The signal travels near the speed of light in the medium



propagation speed s ≈ speed of light in the medium

Propagation delay measures physical travel time from A to B.

Core structure | Routing + forwarding | Packet switching | Performance

Propagation delay ends when the bit reaches Router B

propagation speed s ≈ speed of light in the medium

d_prop = distance d / propagation speed s

Longer distance d → longer propagation delay.

Core structure | Routing + forwarding | Packet switching | Performance

Router B receives the complete packet

What should Router B do with this packet?

The packet has arrived; now the router must make a forwarding decision.

Processing delay is time spent examining the packet

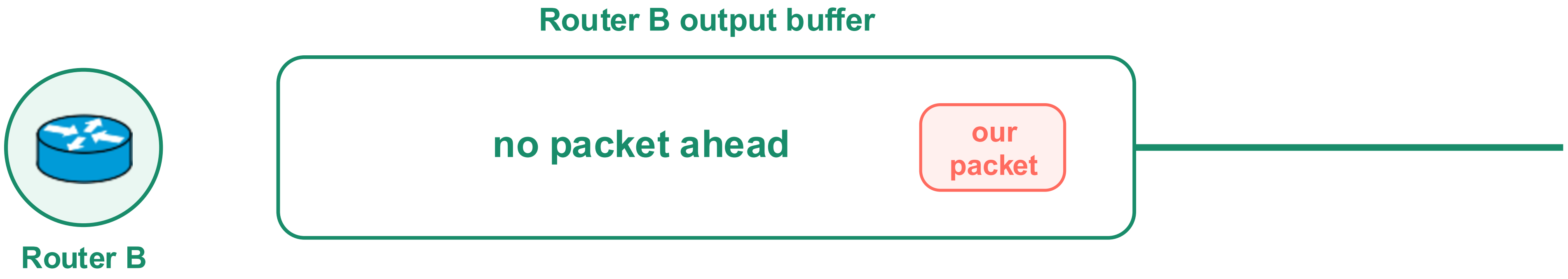
1 Check header and errors

2 Look up the next output

d_proc = time spent processing inside the router

Processing is computation inside Router B—not time spent on the link.

An empty output buffer means almost no queueing delay

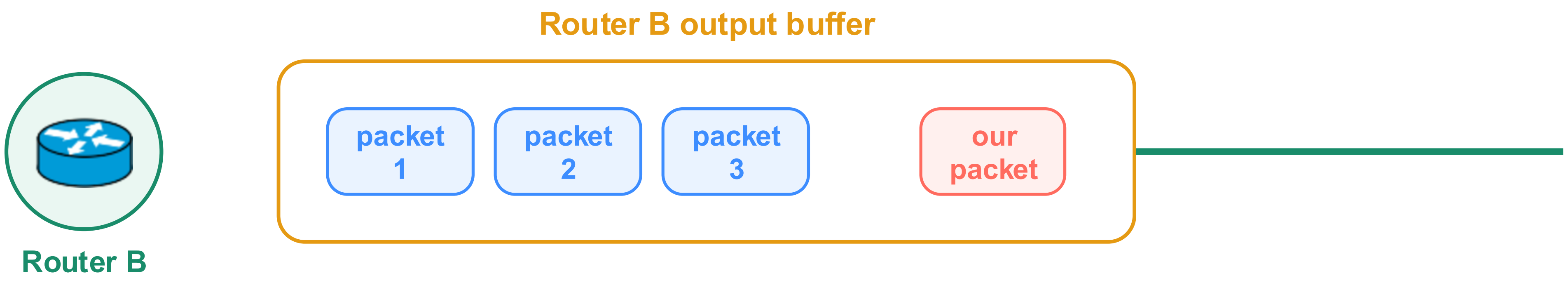


$d\_queue \approx 0$

No packets ahead means almost no queueing delay.

Core structure | Routing + forwarding | Packet switching | Performance

Other packets in the buffer create queueing delay



wait behind 3 packets

buffer waiting time = queueing delay d_queue

More packets ahead → longer queueing delay.

The four delays occur in the packet's travel order

1	2	3	4
Transmission	**Propagation**	**Processing**	**Queueing**
bits enter link	bits travel A → B	Router B examines	wait in buffer
L / R	d / s	d_proc	d_queue

One packet, one journey, four places where time is spent.

Core structure | Routing + forwarding | Packet switching | Performance

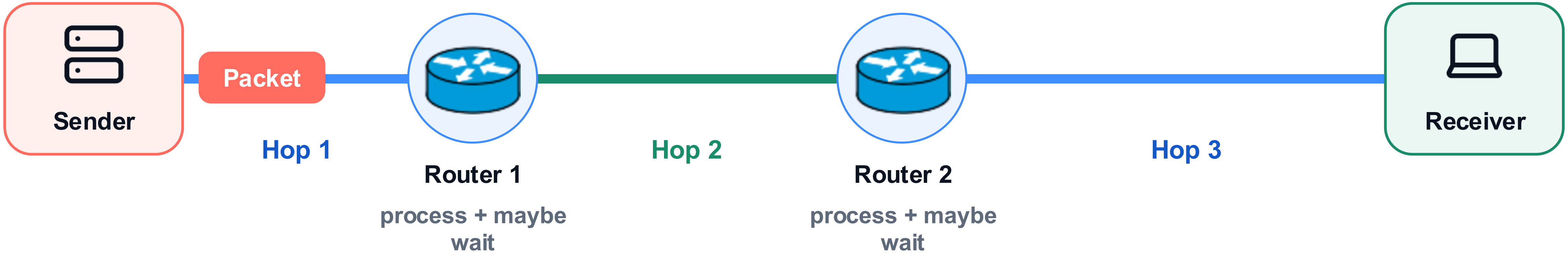
One-hop delay is the sum of the four components

$$d\_nodal = d\_trans + d\_prop + d\_proc + d\_queue$$

Faster link rate R	→	shorter transmission delay
Shorter distance d	→	shorter propagation delay
Less router work	→	shorter processing delay
Fewer packets ahead	→	shorter queueing delay

Each delay has a different physical cause—and a different way to reduce it.

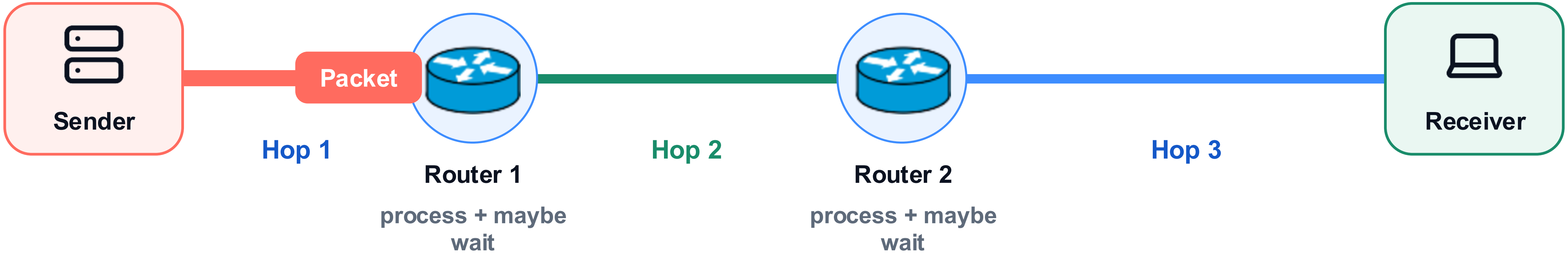
The same four steps repeat at every hop



At every router: process → queue → transmit → propagate.

Keep the route fixed and watch one packet advance one stage at a time.

The packet crosses the first link

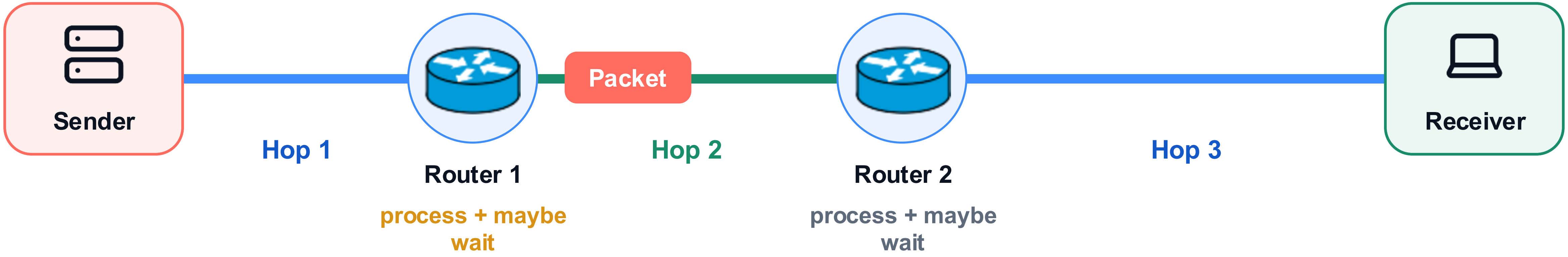


The packet pays transmission and propagation delay on Hop 1.

Keep the route fixed and watch one packet advance one stage at a time.

Core structure | Routing + forwarding | Packet switching | Performance

The packet may wait again at the next router

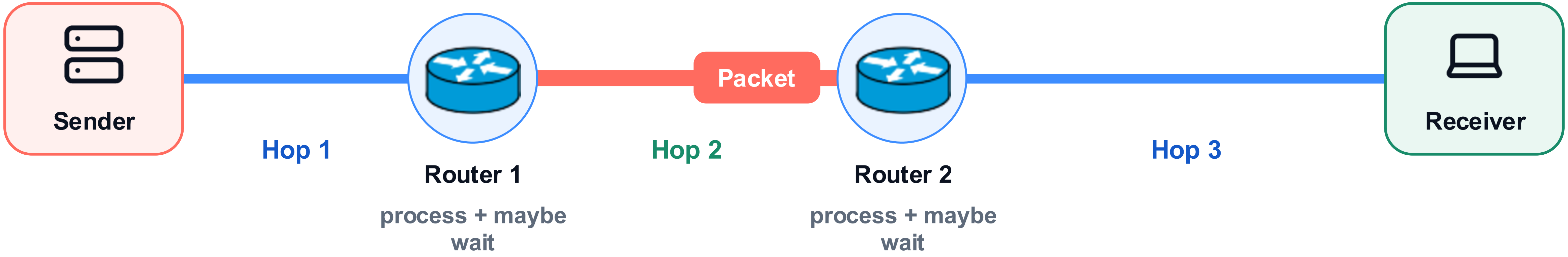


Router 1 processes the packet and may queue it again.

Keep the route fixed and watch one packet advance one stage at a time.

Core structure | Routing + forwarding | Packet switching | Performance

The packet crosses the next link

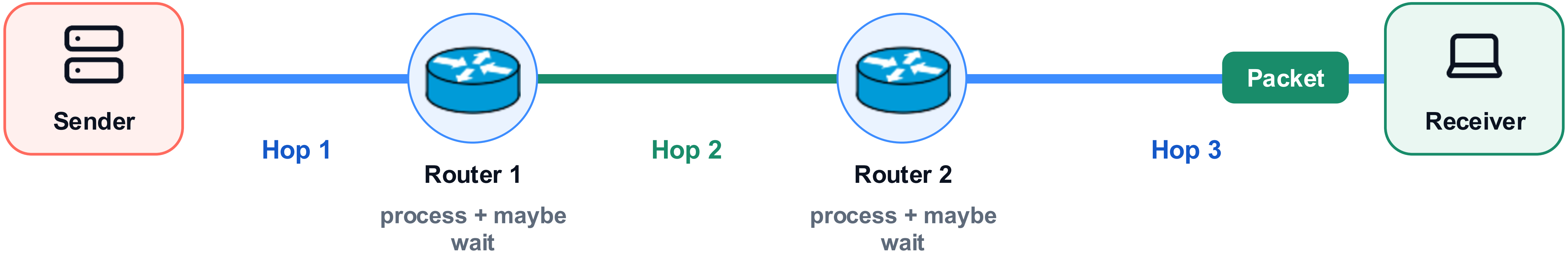


The same four-stage cycle begins again on Hop 2.

Keep the route fixed and watch one packet advance one stage at a time.

Core structure | Routing + forwarding | Packet switching | Performance

End-to-end delay adds the delay of every hop

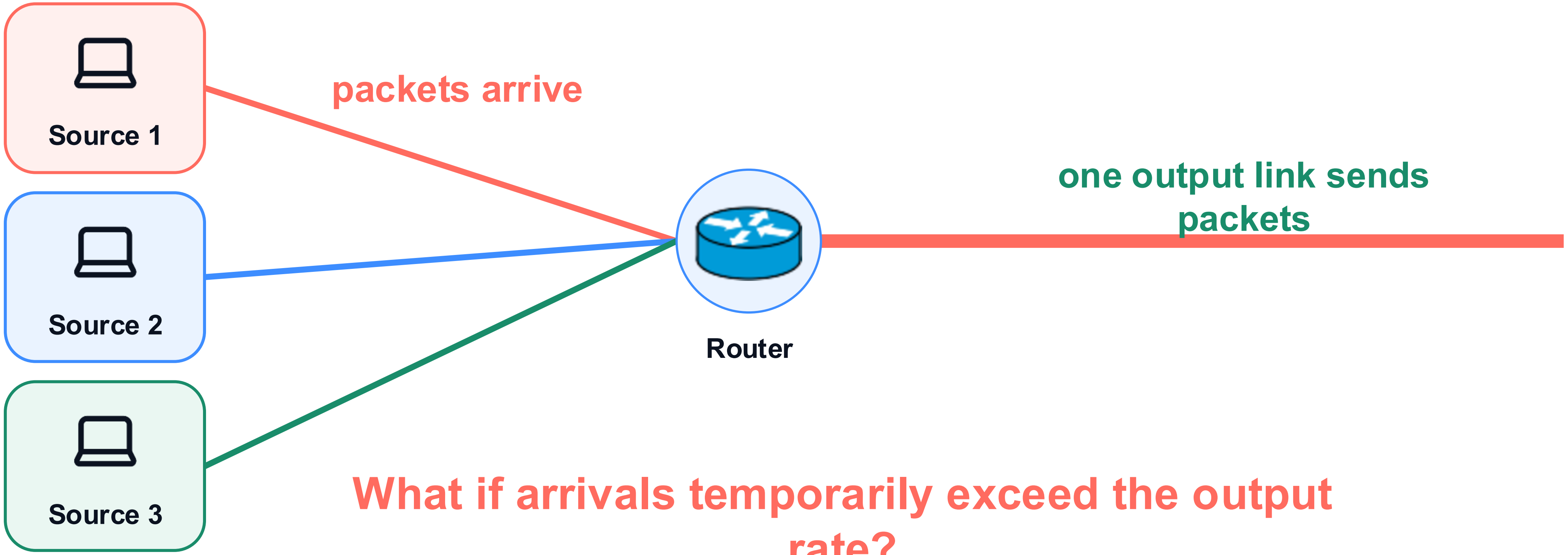


end-to-end delay = Σ (one-hop delays)

End-to-end delay grows by repeating the same local process across the selected route.

Core structure | Routing + forwarding | Packet switching | Performance

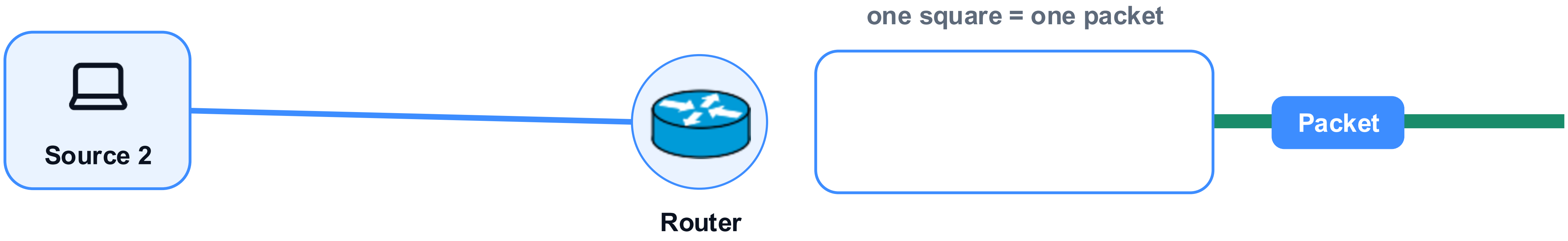
What changes when packets arrive too quickly?



What if arrivals temporarily exceed the output rate?

Keep the output link fixed; now increase the arriving traffic.

Below capacity, the output link keeps up



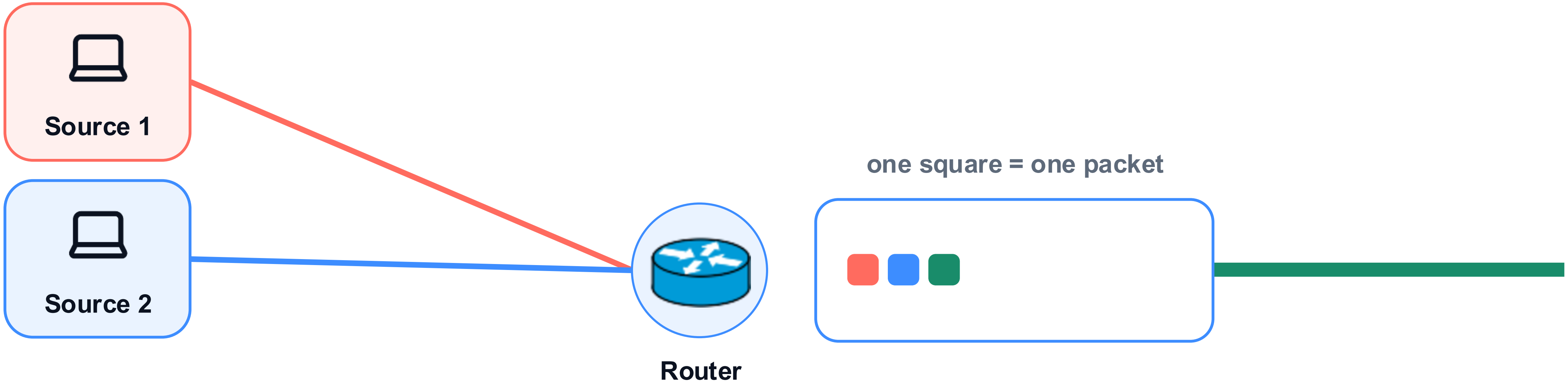
arrival rate ≤ output rate

Packets leave as fast as they arrive, so the queue stays short.

Queueing is small when service keeps up with arrivals.

Core structure | Routing + forwarding | Packet switching | Performance

A burst makes packets wait in the buffer

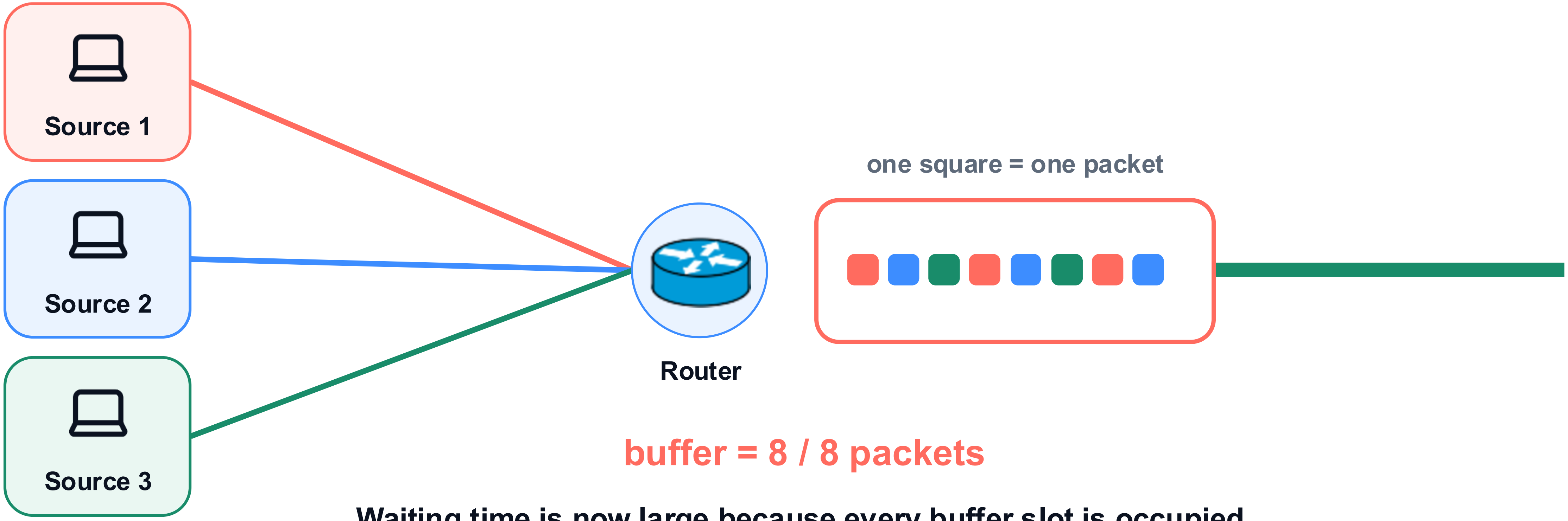


arrival rate > output rate for a short burst

Later packets wait behind packets that arrived first.

Queueing delay appears before packet loss.

A full buffer cannot accept another packet

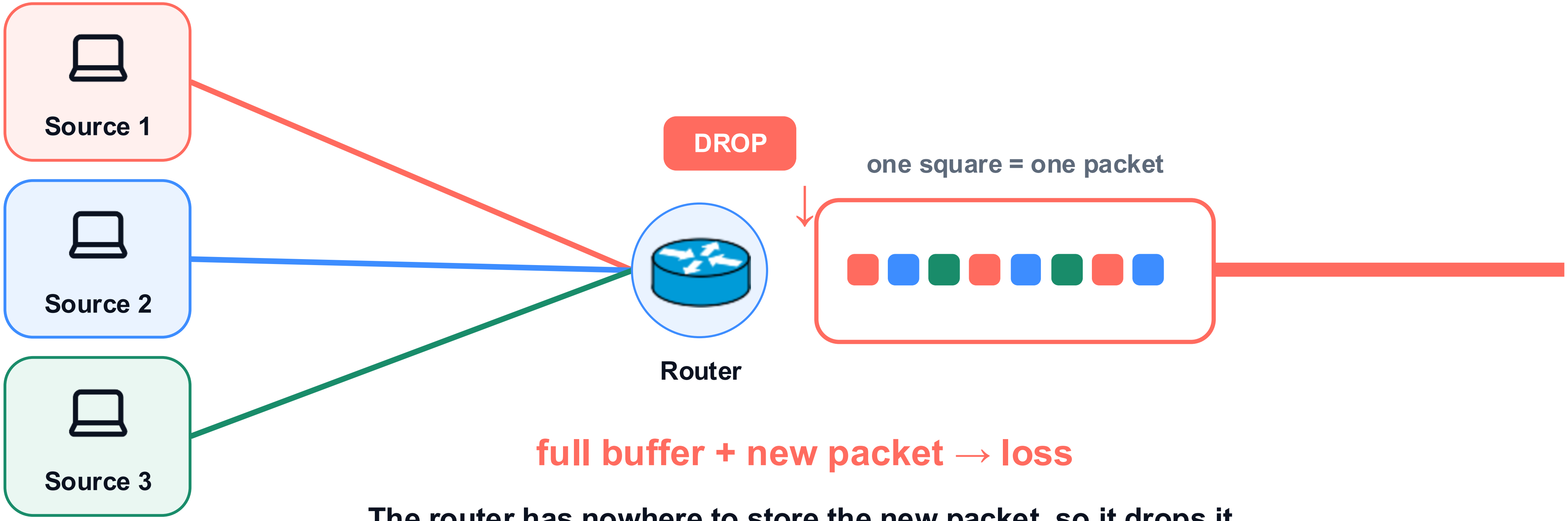


buffer = 8 / 8 packets

Waiting time is now large because every buffer slot is occupied.

A finite buffer can absorb a burst, but only temporarily.

The next arriving packet is dropped



full buffer + new packet → loss

The router has nowhere to store the new packet, so it drops it.

Loss is the next step after a finite buffer becomes full.

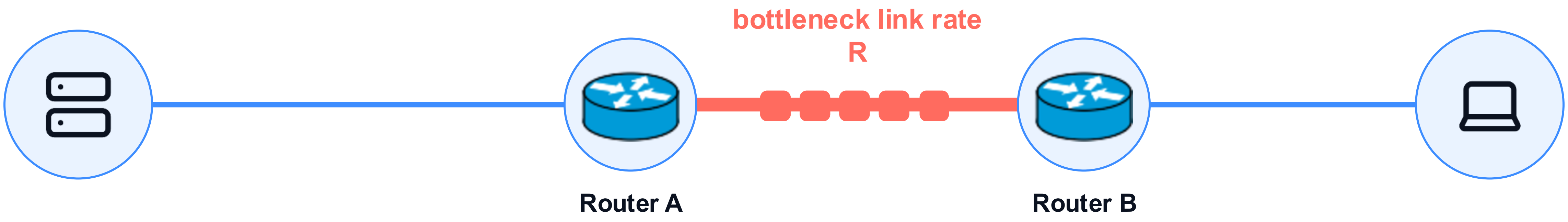
Throughput measures useful delivery

$$\text{throughput} = \text{delivered bits} / \text{elapsed time}$$

Throughput counts useful bits that actually reach the receiver.

Core structure | Routing + forwarding | Packet switching | Performance

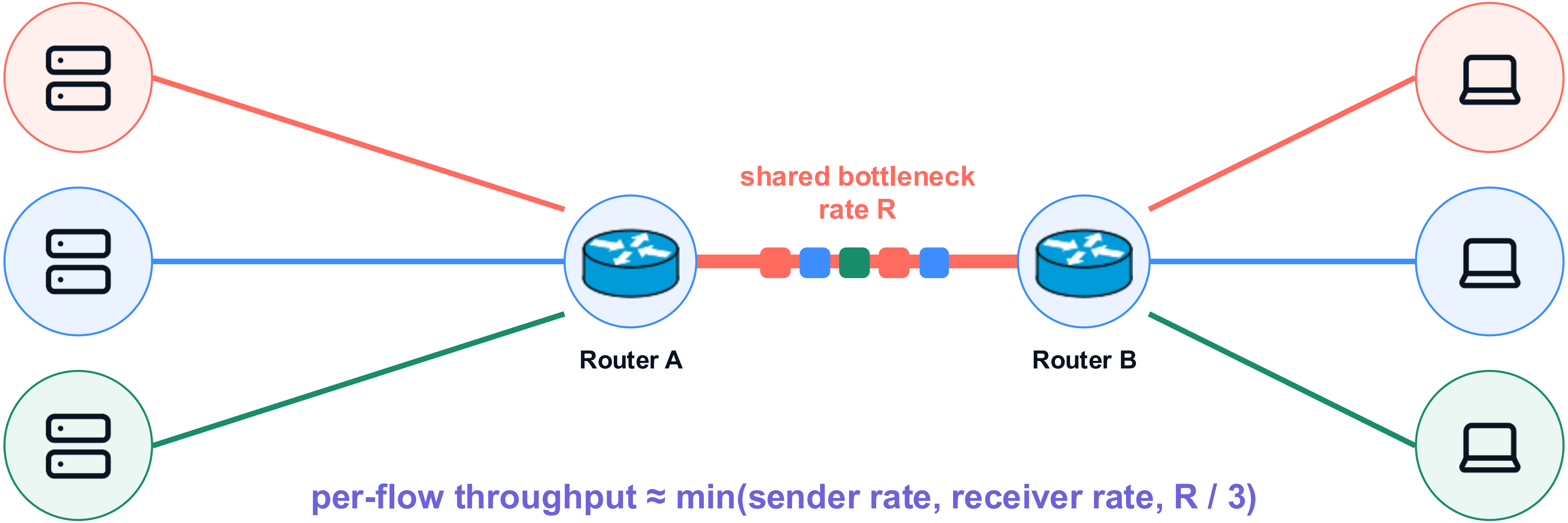
The slowest link limits one connection



throughput = min(sender rate, bottleneck rate, receiver rate)

One slow link can limit the entire end-to-end connection.

Sharing one bottleneck divides capacity



per-flow throughput ≈ min(sender rate, receiver rate, R / 3)

When three flows share one bottleneck, each receives only part of its capacity.

Core structure | Routing + forwarding | Packet switching | Performance

Below capacity, delivery stays smooth

DELAY

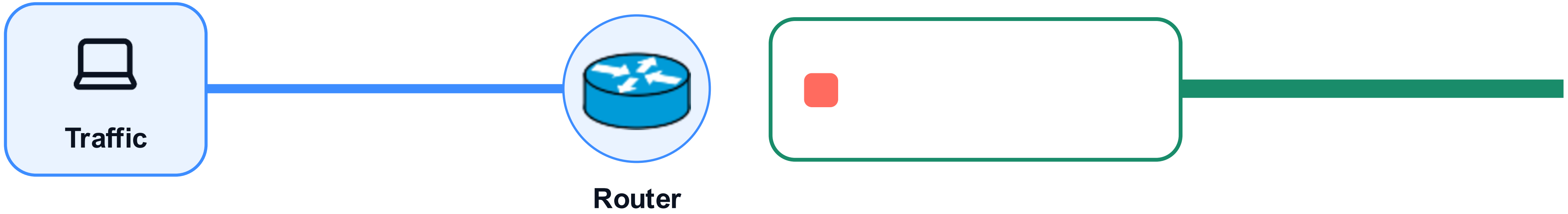
LOW

LOSS

NONE

THROUGHPUT

STEADY



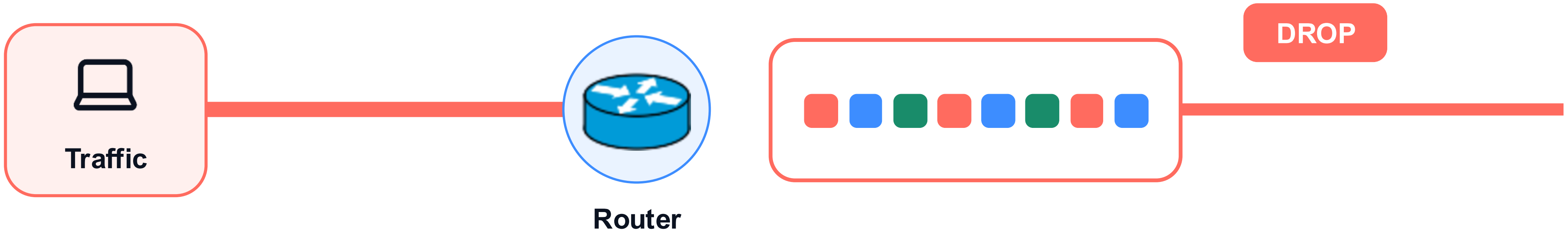
service keeps up with arrivals → queue stays short

When offered load stays below capacity, packets move with little waiting and no loss.

Core structure | Routing + forwarding | Packet switching | Performance

Congestion raises delay and loss, then caps throughput

DELAY	LOSS	THROUGHPUT
RISING	OCCURS	CAPPED



arrivals exceed service → queue grows → buffer fills

Congestion connects the three metrics: more waiting, possible loss, and no extra useful delivery.

The Internet core is shared, distributed, and constrained

Shared

Packets from many users reuse links and routers.

Distributed

Independent networks coordinate without one owner.

Constrained

Rates, distance, buffers, and contention shape performance.

The Internet core works because local actions compose into global communication.

Core structure | Routing + forwarding | Packet switching | Performance